



دانشگاه تهران
پردیس دانشکده های فنی
دانشکده مهندسی برق و کامپیوتر



پایان نامه برای دریافت درجه کارشناسی
در رشته مهندسی کامپیوتر گرایش نرم افزار

تشخیص گلوگاه های یادگیری در یک سامانه چندعامله مبتنی بر مدل زبانی بزرگ با
استفاده از ردیابی دانش و گراف های دانش

نام و نام خانوادگی:

شهزاد ممیز

شماره دانشجویی:

۸۱۰۱۰۰۲۷۲

استاد راهنما:

سرکار خانم دکتر فتانه تقی یاره

فروردین ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

تعهدنامه اصالت اثر

باسمه تعالی

اینجانب شهزاد ممیز تائید می کنم که مطالب مندرج در این پایان نامه حاصل تلاش اینجانب است و به دستاوردهای پژوهشی دیگران که در این نوشته از آنها استفاده شده است مطابق مقررات ارجاع گردیده است. این پایان نامه قبلاً برای احراز هیچ مدرک هم سطح یا بالاتر ارائه نشده است.

کلیه حقوق مادی و معنوی این اثر متعلق به دانشکده فنی دانشگاه تهران می باشد.

نام و نام خانوادگی دانشجو :

شهزاد ممیز

امضای دانشجو :

تشکر و قدردانی:

با نهایت احترام و ارادت، از تمامی عزیزانی که در این مسیر پرفراز و نشیب به من کمک کردند، صمیمانه تشکر می‌کنم. ابتدا از خانم دکتر فتانه تقی یاره، استاد عزیز که با صبر و حوصله به ارزیابی کارم پرداختند و در این مسیر کمک فراوانی به من کردند، تشکر می‌کنم. همچنین از آقای فرشاد حسامی، دانشجوی کارشناسی ارشد، بابت ارائه بازخوردهای تکمیلی در چارچوب علمی و پژوهشی، سپاسگزارم. در پایان، از مادر و پدر عزیزم که همواره با محبت، حمایت و دلگرمی‌های بی‌دریغ خود پشتیبان اصلی من در این مسیر بودند، از صمیم قلب قدردانی می‌کنم.

چکیده:

با گسترش استفاده از فناوری‌های هوشمند در آموزش، نیاز به سامانه‌هایی که بتوانند ساختار مفاهیم آموزشی، روابط پیش‌نیازی و روند یادگیری فراگیران را به‌صورت دقیق‌تر نمایش دهند، افزایش یافته است. در روش‌های معمول آموزشی، ارزیابی یادگیری بیشتر بر اساس آزمون‌های نهایی یا نمره‌های کلی انجام می‌شود و معمولاً مشخص نیست هر دانشجو در کدام مفهوم ضعف دارد، کدام مفهوم به مفهوم دیگر وابسته است و روند یادگیری در طول زمان چگونه تغییر کرده است. این مسئله باعث می‌شود معلم یا طراح آموزشی نتواند تصویر کاملی از وضعیت دانش فراگیران به‌دست آورد. همین‌جا متوجه می‌شویم کسب نمره ی ۱۸ در یک امتحان بیانگر همه اطلاعات نیست و از روی این نمره به تنهایی نمی‌توان نقاط قوت و ضعف دانشجویان را فهمید و جزئیات یادگیری دانشجویان را متوجه شد.

در این پروژه، یک سامانه تحت وب برای شبیه‌سازی یادگیری مبتنی بر گراف دانش طراحی و پیاده‌سازی شده است. در این سامانه، کاربر موضوع آموزشی، تعداد مفاهیم و تعداد تکرارهای شبیه‌سازی را وارد می‌کند. سپس سامانه با استفاده از مدل زبانی، مفاهیم اصلی مرتبط با موضوع و روابط وابستگی میان آن‌ها را تولید می‌کند. این مفاهیم به‌صورت گره و روابط پیش‌نیازی به‌صورت یال‌های جهت‌دار در گراف دانش نمایش داده می‌شوند. پس از آن، سه دانشجوی شبیه‌سازی‌شده با سطح‌های ضعیف، متوسط و قوی ایجاد می‌شوند و برای هر دانشجو یک گراف دانش اختصاصی ساخته می‌شود. در هر تکرار، دانشجویان توضیح مربوط به مفاهیم را دریافت کرده، به سؤالات تولیدشده پاسخ می‌دهند و پاسخ آن‌ها توسط مدل زبانی ارزیابی می‌شود.

معیار ارزیابی در این پروژه عددی بین صفر تا یک است که میزان کیفیت پاسخ دانشجو را نشان می‌دهد. این نمره برای به‌روزرسانی مقدار تسلط هر دانشجو بر هر مفهوم استفاده می‌شود. همچنین یک گراف دانش عمومی بر اساس میانگین میزان تسلط دانشجویان ساخته می‌شود تا وضعیت کلی یادگیری گروه نمایش داده شود. در پایان شبیه‌سازی، سامانه خروجی‌هایی شامل فهرست مفاهیم، جدول وابستگی‌ها، گراف‌های دانش در هر تکرار، گراف دانش نهایی، گراف دانشجوی ضعیف، نمودار روند یادگیری، جدول پاسخ‌های دانشجویان، نمودار درصد یادگیری مفاهیم و نمودار توزیع کلی یادگیری تولید می‌کند. این خروجی‌ها امکان تحلیل دقیق‌تر وضعیت یادگیری و شناسایی مفاهیم ضعیف‌تر را فراهم می‌سازند.

کلمات کلیدی:

گراف دانش، ردیابی دانش، شبیه‌سازی یادگیری، مدل زبانی، مدل‌های زبانی بزرگ، تحلیل آموزشی، سامانه چند عامله، گلوگاه آموزشی، ارزیابی دانشجو، یادگیری هوشمند.

فهرست مطالب

۸	فصل ۱: مقدمه و بیان مساله
۱۰	۱-۱ شرح مسئله تحقیق
۱۲	۲-۱ تعریف موضوع تحقیق
۱۴	۳-۱ اهداف و آرمان‌های کلی تحقیق
۱۶	۴-۱ روش انجام تحقیق
۱۸	۵-۱ ساختار پایان‌نامه
۲۰	فصل ۲: پیشینه ی تحقیق
۲۱	۲-۱ ردیابی دانش عمیق
۲۲	۲-۲ ردیابی دانش مبتنی بر توجه
۲۲	۲-۳ ردیابی دانش گراف‌محور
۲۳	۲-۴ مدل تعامل گرافی در ردیابی دانش
۲۳	۲-۵ گراف دانش در آموزش
۲۴	۲-۶ مدل‌های زبانی بزرگ و شخصی‌سازی یادگیری
۲۵	۲-۷ ساخت گراف دانش با کمک مدل‌های زبانی بزرگ
۲۵	۲-۸ گراف دانش و ردیابی ساختاری در مدل‌های زبانی
۲۵	۲-۹ جمع‌بندی پیشینه و جایگاه پژوهش حاضر
۲۶	فصل ۳: مفاهیم اولیه و پیش زمینه
۲۶	۳-۱ مفاهیم پایه
۲۷	۳-۱-۱ مقدمه‌ای بر ساختار سامانه

۲۷	۲-۱-۳ سیستم‌های غیرقطعی
۲۸	۱-۲-۱-۳ واحد تولید
۲۹	۲-۲-۱-۳ گراف دانش و نمایش ساختار مفهومی
۳۳	فصل ۴: طراحی و پیاده سازی
۳۳	۱-۴ روش پیشنهادی
۳۷	۲-۴ ابزارهای مورد نیاز
۳۸	۳-۴ معیار ارزیابی
۴۷	فصل ۵: نتایج
۶۸	فصل ۶: جمع‌بندی، نتیجه‌گیری و پیشنهادها
۶۹	۱-۶ جمع‌بندی
۷۰	۲-۶ نتیجه‌گیری
۷۱	۱-۲-۶ نوآوری / دستاوردها
۷۱	۲-۲-۶ محدودیتها
۷۲	۳-۲-۶ پیشنهادها
۷۴	فصل ۷: مراجع

فصل ۱: مقدمه و بیان مساله

در سال‌های اخیر، استفاده از روش‌های هوشمند در آموزش و یادگیری به یکی از محورهای مهم در طراحی سامانه‌های آموزشی تبدیل شده است. هدف اصلی این نوع سامانه‌ها آن است که فرایند آموزش صرفاً به ارائه محتوای ثابت محدود نماند، بلکه وضعیت یادگیری دانشجو، میزان تسلط او بر مفاهیم، نقاط ضعف، روابط پیش‌نیازی میان مفاهیم و روند پیشرفت او نیز به صورت ساخت یافته بررسی شود. در چنین رویکردی، آموزش یک فرایند پویا در نظر گرفته می‌شود؛ یعنی دانشجو در طول زمان با مفاهیم مختلف مواجه می‌شود، به پرسش‌ها پاسخ می‌دهد، بازخورد دریافت می‌کند و سطح تسلط او به تدریج به روزرسانی می‌شود.

در پروژه حاضر، یک سامانه شبیه‌سازی آموزشی طراحی و پیاده‌سازی شده است که هدف آن مدل‌سازی فرایند یادگیری چند دانشجوی فرضی روی یک موضوع مشخص است. این سامانه از ترکیب چند جزء اصلی تشکیل شده است: استخراج مفاهیم اصلی یک موضوع، تعیین روابط پیش‌نیازی میان مفاهیم، ساخت گراف دانش، تعریف دانشجویان با سطوح متفاوت، تولید پاسخ توسط دانشجویان شبیه‌سازی شده، ارزیابی پاسخ‌ها، به‌روزرسانی میزان تسلط، تولید نمودار گراف دانش^۱، تولید نمودار روند یادگیری و نمایش نتایج در قالب یک برنامه تحت وب. در واقع، سامانه فقط یک خروجی متنی تولید نمی‌کند، بلکه یک چرخه کامل از تولید برنامه آموزشی، اجرای شبیه‌سازی، ارزیابی و گزارش‌دهی را پوشش می‌دهد.

کدی که در این پروژه پیاده‌سازی شده است با زبان برنامه‌نویسی پایتون نوشته شده و برای بخش وب از چارچوب فلک استفاده می‌کند. برای ساخت و تحلیل گراف دانش از کتابخانه NetworkX، برای تولید نمودارها از Matplotlib، برای محاسبات عددی از NumPy، و برای ارتباط با مدل زبانی از کتابخانه OpenAI استفاده شده است. برنامه به گونه‌ای طراحی شده است که کاربر از طریق صفحه وب، موضوع آموزشی، تعداد مفاهیم و تعداد تکرارهای شبیه‌سازی را وارد می‌کند. سپس برنامه در پس‌زمینه اجرا می‌شود و میزان پیشرفت اجرای شبیه‌سازی از طریق یک نوار پیشرفت در صفحه وب نمایش داده می‌شود.

یکی از نکات مهم در این پروژه، استفاده از مدل زبانی برای دو وظیفه اصلی است. وظیفه اول، تولید ساختار اولیه درس است؛ یعنی مدل زبانی باید بر اساس موضوع ورودی، تعداد مشخصی مفهوم اصلی استخراج کند و روابط پیش‌نیازی میان آن‌ها را بازگرداند. وظیفه دوم، شبیه‌سازی پاسخ‌دهی دانشجویان و ارزیابی پاسخ‌ها است. به این ترتیب، مدل زبانی هم در نقش تولیدکننده محتوای آموزشی و هم در نقش دانشجوی شبیه‌سازی شده و ارزیاب به کار گرفته شده است. این موضوع باعث می‌شود سامانه بتواند برای موضوعات مختلف، بدون نیاز به تعریف دستی همه مفاهیم، ساختار آموزشی اولیه ایجاد کند.

گراف دانش در این پروژه نقش مرکزی دارد. هر مفهوم آموزشی به عنوان یک گره در گراف در نظر گرفته می‌شود و روابط پیش‌نیازی میان مفاهیم به صورت یال‌های جهت‌دار نمایش داده می‌شوند. برای هر گره، مقدار تسلط^۲ نگهداری می‌شود. این مقدار در ابتدا صفر است و در طول اجرای شبیه‌سازی، بر اساس پاسخ‌های دانشجویان و امتیازی که دریافت می‌کنند، به‌روزرسانی می‌شود. در پایان، گراف دانش نهایی همراه با مقدار تسلط هر مفهوم ترسیم می‌شود تا بتوان وضعیت کلی یادگیری را به صورت بصری مشاهده کرد.

در این گزارش، اجزای مختلف سامانه، روش طراحی، نحوه پیاده‌سازی، ساختار داده‌ها، روند اجرای شبیه‌سازی، معیارهای ارزیابی، خروجی‌ها، محدودیت‌ها و پیشنهادهای توسعه آینده به صورت منظم و مطابق ساختار پایان‌نامه توضیح داده می‌شود. متن گزارش بر اساس کد پیاده‌سازی شده نوشته شده است و از بیان نتایج عددی غیرواقعی خودداری می‌شود. بنابراین هر جا که نیاز به داده

تجربی وابسته به اجرای برنامه وجود داشته باشد، گزارش آن را به عنوان خروجی قابل تولید توسط سامانه توضیح می دهد و مقدار واقعی باید پس از اجرای برنامه با ورودی مشخص تکمیل شود.

۱-۱- شرح مسئله تحقیق

با پیشرفت سریع فناوری های مبتنی بر هوش مصنوعی، به ویژه مدل های زبانی بزرگ و سیستم های چندعاملی، امکان طراحی محیط های آموزشی هوشمند و تعاملی بیش از پیش فراهم شده است. در این محیط ها، عامل های یادگیرنده می توانند به صورت مستقل یا تعاملی مسیرهای یادگیری متفاوتی را طی کرده و از طریق تعامل با عامل های دیگر (مانند عامل معلم)، دانش خود را به مرور زمان توسعه دهند. با این حال، یکی از چالش های اساسی در چنین سیستم هایی، مدل سازی دقیق، پویا و قابل تفسیر از وضعیت دانش یادگیرندگان در طول فرآیند یادگیری است.

در روش های سنتی ارزیابی آموزشی، معمولاً سطح دانش یادگیرنده به صورت ایستا و مبتنی بر آزمون های مقطعی سنجیده می شود. این رویکرد نه تنها قادر به نمایش تغییرات تدریجی دانش در طول زمان نیست، بلکه نمی تواند روابط مفهومی بین مباحث مختلف را نیز به درستی مدل سازی کند. از سوی دیگر، روش های نوین مانند ردیابی دانش تلاش می کنند با استفاده از داده های تعامل یادگیرنده با سیستم، سطح تسلط او بر مفاهیم مختلف را در طول زمان تخمین بزنند. با این وجود، بسیاری از این روش ها فاقد ساختار معنایی غنی بوده و معمولاً روابط پیچیده بین مفاهیم را به صورت صریح در نظر نمی گیرند.

در این میان، گراف های دانش به عنوان ابزاری قدرتمند برای نمایش ساختارمند دانش و روابط بین مفاهیم مطرح شده اند. ترکیب ردیابی دانش با گراف دانش این امکان را فراهم می سازد که نه تنها میزان تسلط یادگیرنده بر هر مفهوم به صورت پویا مدل شود، بلکه وابستگی ها و پیش نیازهای مفهومی نیز به صورت ساخت یافته در نظر گرفته شوند. با این حال، چالش اصلی در این حوزه، ایجاد یک گراف دانشی پویا و قابل به روزرسانی برای هر یادگیرنده بر اساس تعاملات او و سپس تحلیل این گراف ها برای استخراج بینش های آموزشی است.

علاوه بر این، با ظهور مدل های زبانی بزرگ، امکان شبیه سازی رفتار یادگیرندگان به صورت عامل های هوشمند فراهم شده است. این مدل ها می توانند به عنوان دانش آموزان مجازی عمل کرده و به سوالات پاسخ دهند. اما یک مسئله کلیدی در این زمینه، چگونگی استخراج دانش مفهومی از پاسخ های متنی این مدل ها و نگاشت آن به ساختار گراف دانشی است. این فرآیند نیازمند طراحی روش هایی دقیق برای تحلیل متن، شناسایی مفاهیم، و به روزرسانی وزن های مربوط به میزان تسلط بر هر مفهوم می باشد.

مسئله مهم دیگر، انتقال از سطح فردی به سطح جمعی است. در یک محیط چندعاملی که شامل چندین یادگیرنده (واقعی یا شبیه سازی شده) است، هر عامل دارای گراف دانشی مختص به خود می باشد. چالش اینجاست که چگونه می توان این گراف های فردی را به صورت معنادار تجمیع کرده و یک گراف دانشی کلان ایجاد نمود که نمایانگر وضعیت کلی دانش در یک جامعه یادگیرنده باشد. چنین گرافی می تواند برای تحلیل های سطح بالا و تصمیم گیری های آموزشی مورد استفاده قرار گیرد.

در نهایت، یکی از مهم‌ترین مسائل مورد توجه در این تحقیق، شناسایی گلوگاه‌های آموزشی است. گلوگاه‌های آموزشی به مفاهیمی اطلاق می‌شود که ضعف در آن‌ها تأثیر قابل توجهی بر یادگیری سایر مفاهیم دارد. شناسایی این گره‌ها در گراف دانشی، نیازمند استفاده از معیارهای گرافی است. اما تعیین دقیق این گلوگاه‌ها در یک گراف پویا و چندلایه، به‌ویژه در حضور چندین عامل یادگیرنده، همچنان یک چالش باز محسوب می‌شود.

۱-۲- تعریف موضوع تحقیق

موضوع این تحقیق، طراحی و پیاده‌سازی یک سامانه شبیه‌سازی یادگیری مبتنی بر گراف دانش و مدل زبانی است. در این سامانه، کاربر یک موضوع آموزشی را وارد می‌کند و برنامه با کمک مدل زبانی، مفاهیم اصلی آن موضوع و روابط پیش‌نیازی میان آن‌ها را استخراج می‌کند. سپس این مفاهیم به‌صورت گره‌های یک گراف جهت‌دار مدل می‌شوند. هر گره دارای یک مقدار تسلط است که وضعیت یادگیری را نشان می‌دهد. در ادامه، چند دانشجوی فرضی با سطح‌های مختلف تعریف می‌شوند و روند یادگیری آن‌ها در چند تکرار شبیه‌سازی می‌شود.

- در این تعریف، «گراف دانش» ساختاری است که مفاهیم و ارتباطات میان آن‌ها را نشان می‌دهد. اگر یک مفهوم برای فهم مفهوم دیگر لازم باشد، رابطه پیش‌نیازی میان آن‌ها در گراف به‌صورت یال جهت‌دار نمایش داده می‌شود. این ساختار باعث می‌شود سامانه بتواند فقط نمره کلی دانشجو را بررسی نکند، بلکه وضعیت هر مفهوم را جداگانه تحلیل کند. چنین تحلیلی برای شناسایی نقاط ضعف و مفاهیم گلوگاهی اهمیت دارد.

- «مدل زبانی» در این پروژه ابزاری است که برای تولید محتوای متنی، پاسخ‌دهی و ارزیابی استفاده می‌شود. مدل زبانی ابتدا بر اساس دستور ورودی^۳ طراحی شده، مفاهیم و وابستگی‌ها را تولید می‌کند. سپس در نقش دانشجوی شبیه‌سازی شده ظاهر می‌شود و با توجه به سطح دانشجو پاسخ می‌دهد. همچنین در مرحله ارزیابی، پاسخ دانشجو را نسبت به سؤال بررسی می‌کند و یک نمره عددی بین صفر و یک برمی‌گرداند. در کد، ارتباط با مدل زبانی از طریق شیء client و تابع client.chat.completions.create انجام شده است.

- «دانشجوی شبیه‌سازی شده» در این پروژه موجودیتی نرم‌افزاری است که در کلاس Student تعریف شده است. هر دانشجو دارای سطح یادگیری، حافظه و گراف دانش فردی است. حافظه دانشجو شامل توضیحاتی است که در طول فرایند یادگیری دریافت کرده است. گراف دانش فردی او نیز شامل همان مفاهیم است، اما مقدار تسلط هر مفهوم برای هر دانشجو جداگانه نگهداری می‌شود. بنابراین سیستم می‌تواند عملکرد دانشجوی ضعیف، متوسط و قوی را جداگانه بررسی کند.

- «تسلط» یا mastery در این پروژه یک مقدار عددی بین صفر تا یک است که میزان یادگیری یک مفهوم را نشان می‌دهد. این مقدار ابتدا صفر در نظر گرفته می‌شود. پس از پاسخ دانشجو به سؤال مربوط به یک مفهوم، پاسخ ارزیابی می‌شود و یک نمره تولید می‌گردد. سپس مقدار تسلط قبلی و نمره جدید میانگین‌گیری می‌شوند تا مقدار تسلط جدید به دست آید. این روش ساده است، اما برای هدف شبیه‌سازی آموزشی پروژه مناسب است؛ زیرا امکان مشاهده تغییر تدریجی تسلط را فراهم می‌کند.

- «گزارش نهایی» در این پروژه شامل مجموعه‌ای از خروجی‌های متنی و تصویری است. تابع `build_simulation_report` نتایج اجرای شبیه‌سازی را به صورت یک گزارش متنی آماده می‌کند. این گزارش شامل تنظیمات شبیه‌سازی، مفاهیم استخراج‌شده، روابط پیش‌نیازی، ساختار دانش، تسلط نهایی، گلوگاه‌ها، تاریخچه یادگیری، پاسخ‌های دانشجویان و مسیر فایل‌های تصویری تولیدشده است. این گزارش می‌تواند مبنای تحلیل عملکرد سامانه و بررسی نتایج اجرای آن قرار گیرد.
- بنابراین موضوع تحقیق را می‌توان چنین خلاصه کرد: ساخت یک برنامه وب که با استفاده از مدل زبانی و گراف دانش، برای هر موضوع آموزشی یک ساختار مفهومی تولید کند، یادگیری دانشجویان فرضی را شبیه‌سازی نماید، پاسخ‌ها را ارزیابی کند، تسلط مفهومی را محاسبه کند و نتایج را به صورت گراف، نمودار و گزارش متنی نمایش دهد.

۱-۳- اهداف و آرمان‌های کلی تحقیق

هدف اصلی این پروژه، پیاده‌سازی یک سامانه عملی برای شبیه‌سازی فرایند یادگیری در محیط آموزشی است. این سامانه تلاش می‌کند با بهره‌گیری از مدل‌های زبانی بزرگ، گراف دانش و ردیابی تدریجی میزان تسلط، نمونه‌ای اولیه از یک سیستم آموزش هوشمند ارائه دهد. برای دستیابی به این هدف کلی، اهداف جزئی پروژه در چند محور اصلی تعریف می‌شوند.

نخستین هدف، استخراج و سازمان‌دهی مفاهیم آموزشی بر اساس موضوع ورودی کاربر است. در این بخش، کاربر یک موضوع آموزشی و تعداد مفاهیم موردنظر را وارد می‌کند و سامانه با استفاده از مدل زبانی، مفاهیم اصلی مرتبط با آن موضوع را استخراج می‌کند. خروجی این مرحله به صورت ساخت‌یافته و در قالب JSON تولید می‌شود تا برنامه بتواند آن را به صورت خودکار پردازش کند. این هدف، نقطه آغاز فرایند شبیه‌سازی است، زیرا کیفیت مفاهیم استخراج‌شده بر ساختار گراف دانش و تحلیل‌های بعدی تأثیر مستقیم دارد.

هدف دوم، ساخت گراف دانش و نمایش روابط مفهومی میان اجزای درس است. در این گراف، هر مفهوم به عنوان یک گره و هر رابطه پیش‌نیازی یا وابستگی مفهومی به عنوان یک یال جهت‌دار در نظر گرفته می‌شود. استفاده از گراف دانش باعث می‌شود ساختار درس فقط به صورت فهرستی از مفاهیم نمایش داده نشود، بلکه ارتباط میان مفاهیم نیز مشخص باشد. این موضوع اهمیت زیادی دارد، زیرا ضعف در یک مفهوم پایه می‌تواند بر یادگیری مفاهیم وابسته اثر بگذارد و از این طریق، امکان شناسایی بهتر مسیرهای دشوار یادگیری فراهم می‌شود.

هدف سوم، شبیه‌سازی یادگیرندگان با سطوح مختلف و ارزیابی تدریجی میزان تسلط آن‌ها است. در این پروژه، سه سطح دانشجو شامل ضعیف، متوسط و قوی تعریف شده‌اند و تفاوت آن‌ها از طریق تنظیمات مدل زبانی، دستور سیستمی و مقدار دما شبیه‌سازی می‌شود. پاسخ‌های تولیدشده توسط دانشجویان شبیه‌سازی‌شده با استفاده از تابع ارزیابی به نمره‌ای عددی بین صفر تا یک تبدیل می‌شوند. سپس مقدار تسلط هر دانشجو بر هر مفهوم، بر اساس امتیازهای جدید به‌روزرسانی می‌شود و گراف عمومی دانش نیز با میانگین‌گیری از میزان تسلط دانشجویان در هر مفهوم اصلاح می‌گردد. این هدف امکان بررسی روند یادگیری، مقایسه سطوح مختلف دانشجویان و تشخیص نقاط ضعف مفهومی را فراهم می‌کند.

هدف چهارم، ارائه خروجی‌های قابل فهم و تبدیل سامانه به یک ابزار تحت وب قابل استفاده برای کاربر است. خروجی‌های سامانه شامل گزارش متنی، تصویر گراف دانش، نمودار روند یادگیری، میزان تسلط نهایی و شکاف باقی‌مانده در مفاهیم مختلف است.

این خروجی‌ها به کاربر کمک می‌کنند تا وضعیت یادگیری را به صورت بصری و تحلیلی مشاهده کند. همچنین با تبدیل اسکرپت شبیه‌سازی به یک برنامه تحت وب، کاربر بدون نیاز به اجرای مستقیم کد می‌تواند موضوع، تعداد مفاهیم و تعداد تکرارها را از طریق فرم وارد کند و نتایج را در مرورگر ببیند. وجود نوار پیشرفت نیز تجربه کاربری را بهبود می‌دهد و کاربر را از وضعیت اجرای عملیات آگاه می‌سازد.

در مجموع، آرمان کلی این پروژه آن است که یک نمونه اولیه از سامانه آموزشی هوشمند طراحی و پیاده‌سازی شود؛ سامانه‌ای که بتواند در آینده توسعه یافته و به ابزاری برای تحلیل آموزشی، طراحی مسیر یادگیری، شناسایی گلوگاه‌های یادگیری، تشخیص نقاط ضعف دانشجویان و تولید محتوای آموزشی تطبیقی تبدیل شود.

۱-۴- روش انجام تحقیق

چارچوب پیشنهادی شامل چهار مرحله اصلی است:

۱. ردیابی دانش
۲. ساخت گراف دانش
۳. تجمیع گراف‌ها
۴. شناسایی گلوگاه آموزشی

هدف این چارچوب، ایجاد یک نمایش پویا و قابل تفسیر از دانش یادگیرندگان و تحلیل آن در سطح فردی و جمعی است.

در این پژوهش، یک چارچوب یکپارچه برای مدل‌سازی، استخراج و تحلیل دانش یادگیرندگان در محیط‌های آموزشی هوشمند ارائه می‌شود که بر پایه ترکیب سه مؤلفه کلیدی شامل ردیابی دانش، گراف‌های دانشی و مدل‌های زبانی بزرگ طراحی شده است. هدف اصلی این رویکرد، ایجاد یک نمایش پویا، ساخت یافته و قابل تفسیر از وضعیت دانش یادگیرندگان و استفاده از آن برای شناسایی گلوگاه‌های آموزشی در سطوح فردی و جمعی است. این چارچوب با الهام از رویکردهای اخیر در زمینه ترکیب ردیابی دانش با مدل‌های زبانی بزرگ برای شخصی‌سازی یادگیری توسعه یافته است و همچنین از پیشرفت‌های حاصل در استفاده از مدل‌های زبانی برای استخراج دانش ساخت یافته و ساخت گراف‌های دانشی بهره می‌گیرد. علاوه بر این، ایده استفاده از نمایش‌های سه‌تایی و ساختارهای گرافی برای بهبود تفسیرپذیری و تحلیل فرآیند استدلال، متأثر از پژوهش‌هایی است که دانش را به صورت ساخت یافته در فرآیند پردازش و تحلیل وارد می‌کنند.

شبیه‌سازی آموزش

در جلسه آموزشی، ابتدا عامل آموزشی یک مفهوم مشخص را بر اساس ساختار از پیش تعریف شده محتوای درسی ارائه می‌دهد. این آموزش می‌تواند شامل توضیح متنی، ارائه مثال و یا توضیح مفهومی باشد که توسط مدل زبانی بزرگ تولید می‌شود. پس از ارائه محتوا، عامل آموزشی مجموعه‌ای از سوالات هدفمند مرتبط با همان مفهوم را مطرح می‌کند تا میزان درک یادگیرنده از آن مفهوم سنجیده شود.

شبیه‌سازی یادگیرنده با استفاده از مدل‌های زبانی بزرگ

در این پژوهش، به منظور تولید داده‌های تعاملی و شبیه‌سازی رفتار یادگیرندگان در محیط آموزشی، از مدل‌های زبانی بزرگ به عنوان عامل شبیه‌ساز دانش‌آموز استفاده شده است. هدف نهایی سیستم، کاربرد در محیط‌های واقعی آموزشی برای کاربران انسانی است و استفاده از مدل زبانی به عنوان شبیه‌ساز برای تولید رفتار یادگیرنده در شرایط آزمایشگاهی در نظر گرفته می‌شود.

در این چارچوب، ابتدا محیط آموزشی به صورت یک سیستم چندعاملی مدل‌سازی می‌شود که در آن عامل‌های یادگیرنده و عامل‌های آموزشی در قالب یک ساختار تعاملی با یکدیگر ارتباط برقرار می‌کنند. عامل یادگیرنده با استفاده از مدل‌های زبانی بزرگ در برابر سوالات پرسیده شده پاسخ‌های متنی می‌دهد و عامل آموزشی نیز ابتدا مطالب را آموزش می‌دهد و سپس با پرسیدن سوالات، میزان یادگیری هرکدام از عامل‌های یادگیرنده مشخص می‌شود. در واقع دانش‌آموز و عامل یادگیرنده با استفاده از مدل‌های زبانی بزرگ شبیه‌سازی می‌شوند.

به منظور پیاده‌سازی عملی فرآیند شبیه‌سازی یادگیرنده، یک مکانیزم کنترل‌شده برای تعیین سطح دانش عامل‌ها طراحی شده است که در آن هر عامل بر اساس یک سطح از پیش تعریف‌شده (ضعیف، متوسط، قوی) رفتار متفاوتی در پاسخ‌گویی از خود نشان می‌دهد. این سطح دانش از طریق یک تابع تنظیم‌کننده سبک^۴ به مدل زبانی اعمال می‌شود که شامل دو مؤلفه اصلی است: (۱) توصیف رفتاری عامل و (۲) یک پارامتر کنترلی برای تنظیم میزان تصادفی بودن پاسخ‌ها.

در این ساختار، برای هر سطح از دانش، یک توصیف متنی (prompt) به مدل زبانی داده می‌شود که نقش و رفتار یادگیرنده را مشخص می‌کند. برای مثال، در سطح «ضعیف»، مدل به گونه‌ای هدایت می‌شود که پاسخ‌هایی کوتاه، مبهم، و در برخی موارد نادرست تولید کند. این امر باعث می‌شود که پاسخ‌های تولیدشده شامل خطاهای مفهومی و استدلال‌های ناقص باشد که نمایانگر رفتار یک دانش‌آموز با درک محدود از مفاهیم است. در سطح «متوسط»، مدل رفتاری بینابینی از خود نشان داده و تنها در برخی موارد دچار خطا می‌شود، در حالی که در سطح «قوی»، مدل پاسخ‌هایی دقیق، کامل و منسجم ارائه می‌دهد که نشان‌دهنده تسلط بالا بر مفاهیم است.

علاوه بر توصیف رفتاری عامل‌های یادگیرنده، در این پژوهش از پارامتر temperature به عنوان یک مؤلفه کلیدی برای کنترل میزان تنوع و عدم قطعیت در پاسخ‌های تولیدشده توسط مدل زبانی استفاده شده است. پارامتر temperature یکی از تنظیمات اصلی در مدل‌های زبانی است که نحوه نمونه‌برداری از توزیع احتمالاتی کلمات را تعیین می‌کند. این پارامتر تعیین می‌کند که پاسخ‌ها تا چه حد قطعی یا تصادفی باشند؛ به طوری که مقادیر بالاتر منجر به پاسخ‌های متنوع‌تر و گاه نادقیق‌تر، و مقادیر پایین‌تر باعث تولید پاسخ‌های دقیق‌تر و پایدارتر می‌شوند.

بر این اساس، برای شبیه‌سازی یادگیرندگان ضعیف از مقدار بالاتر (۰,۸)، برای یادگیرندگان متوسط از مقدار میانی (۰,۶) و برای یادگیرندگان قوی از مقدار پایین‌تر (۰,۳) استفاده شده است. این تنظیمات باعث می‌شود رفتار مدل زبانی از نظر میزان خطا و دقت، با سطح دانش یادگیرنده همخوانی داشته باشد.

در مجموع، این مکانیزم دوگانه (توصیف رفتاری به همراه پارامتر کنترلی) امکان شبیه‌سازی سیستماتیک و قابل تنظیم سطوح مختلف یادگیری را فراهم می‌سازد و به تولید داده‌هایی با تنوع و واقع‌گرایی بالا کمک می‌کند. این داده‌ها در ادامه برای تحلیل فرآیند یادگیری و ساخت گراف دانش مورد استفاده قرار می‌گیرند.

ارزیابی پاسخ‌ها و شبیه‌سازی امتیازدهی توسط عامل یاد دهنده

در مرحله شبیه‌سازی، میزان یادگیری عامل‌ها از طریق مدل‌های زبانی بزرگ، از بین ۰ تا ۱ امتیاز دهی می‌شود. در صورتی که پاسخ دقیق و درست باشد نمره ۱ (نمره کامل) را دریافت می‌کند و در صورتی که پاسخ اشتباه باشد، نمره ۰ را دریافت می‌کند (کمترین نمره) و در صورتی که پاسخ دانش آموز شبیه‌سازی شده، نه کاملاً درست و نه کاملاً غلط باشد بین ۰ و ۱ نمره‌ای از مدل‌های زبانی بزرگ که شبیه‌ساز یک معلم است، دریافت می‌کند. این ساختار امکان شبیه‌سازی رفتار یادگیرندگان در یک محیط آموزشی واقعی را فراهم می‌سازد و داده‌های تعاملی لازم برای تحلیل فرآیند یادگیری را ایجاد می‌کند.

ردیابی دانش

در این مرحله، تمامی تعاملات یادگیرنده با محیط آموزشی، شامل پاسخ‌های ارائه‌شده، امتیازهای عملکردی، تعداد مواجهه با هر مفهوم و توالی فعالیت‌های آموزشی، به‌عنوان ورودی به ماژول ردیابی دانش وارد می‌شود. هدف این ماژول، تخمین پویای وضعیت دانش هر یادگیرنده در طول زمان است. برای این منظور، برای هر یادگیرنده یک گراف دانش اختصاصی ایجاد می‌شود که در آن گره‌ها نشان‌دهنده مفاهیم آموزشی و یال‌ها بیانگر روابط پیش‌نیازی میان مفاهیم هستند.

در هر تکرار، یادگیرنده ابتدا با توضیح آموزشی مربوط به هر مفهوم مواجه می‌شود و سپس به پرسشی درباره همان مفهوم پاسخ می‌دهد. پاسخ تولیدشده توسط یک ارزیاب بررسی شده و بر اساس میزان درستی، کامل بودن و ارتباط با پرسش، نمره‌ای بین صفر تا یک دریافت می‌کند. این نمره به‌عنوان شواهد جدید برای به‌روزرسانی مقدار تسلط یادگیرنده بر آن مفهوم استفاده می‌شود. مقدار تسلط پیشین و نمره جدید به‌صورت وزن‌دار ترکیب می‌شوند تا تخمین دانش یادگیرنده به‌تدریج و بر اساس شواهد انباشته‌شده اصلاح شود.

علاوه بر عملکرد مستقیم یادگیرنده در هر مفهوم، روابط پیش‌نیازی میان مفاهیم نیز در فرایند ردیابی دانش لحاظ می‌شود. به این صورت که اگر یک مفهوم به مفاهیم پایه‌ای دیگری وابسته باشد، میزان تسلط یادگیرنده بر آن پیش‌نیازها نیز در محاسبه تسلط نهایی او بر مفهوم موردنظر اثرگذار خواهد بود. این ویژگی باعث می‌شود مدل، وابستگی‌های واقعی میان مفاهیم آموزشی را بهتر منعکس کند و تخمین دقیق‌تری از وضعیت یادگیری ارائه دهد.

همچنین برای هر یادگیرنده، حافظه‌ای مفهومی در نظر گرفته می‌شود که تعداد مواجهه‌های قبلی با هر مفهوم و میزان تثبیت آن در حافظه را ذخیره می‌کند. این حافظه باعث می‌شود پاسخ‌های یادگیرنده در تکرارهای بعدی متناسب با تجربه‌های قبلی او تغییر کند. در نتیجه، مدل می‌تواند روند یادگیری تدریجی، فراموشی نسبی، ضعف در پیش‌نیازها و تفاوت میان یادگیرندگان ضعیف، متوسط و قوی را بهتر شبیه‌سازی کند.

در پایان هر تکرار، گراف دانش هر یادگیرنده به‌روزرسانی شده و سپس از میانگین وضعیت یادگیرندگان، یک گراف دانش عمومی برای کل گروه ساخته می‌شود. این گراف عمومی نمایی کلی از وضعیت یادگیری گروه ارائه می‌دهد و امکان تحلیل‌هایی مانند میزان تسلط نهایی بر هر مفهوم، روند پیشرفت در طول زمان، مقایسه عملکرد سطوح مختلف یادگیرندگان و شناسایی مفاهیم دشوار را فراهم می‌سازد. به این ترتیب، ماژول ردیابی دانش نقش اصلی را در تبدیل تعاملات خام یادگیرنده به شاخص‌های تحلیلی قابل استفاده برای تصمیم‌گیری آموزشی ایفا می‌کند.

گراف دانش

در گام بعدی، به منظور غلبه بر محدودیت‌های نمایش برداری و ضمنی دانش در روش‌های ردیابی دانش، اطلاعات استخراج‌شده از تعاملات یادگیرنده به یک ساختار گراف تبدیل می‌شود. برای این منظور، پاسخ‌های متنی عامل یادگیرنده با استفاده از قابلیت‌های درک و استخراج دانش مدل‌های زبانی بزرگ تحلیل شده و مفاهیم کلیدی و روابط میان آن‌ها شناسایی می‌شوند. این فرآیند با تکیه بر توانایی مدل‌های زبانی در استخراج دانش ساخت‌یافته از منابع متنی و یکپارچه‌سازی آن‌ها انجام می‌شود. خروجی این مرحله وابستگی بین گره‌های مختلف را نشان می‌دهد که به تدریج یک گراف دانش برای هر یادگیرنده تشکیل می‌دهد. استفاده از این ساختار سه‌تایی باعث می‌شود دانش به صورت صریح، قابل تفسیر و ساخت‌یافته نمایش داده شود و امکان تحلیل روابط مفهومی فراهم گردد.

تجمیع گراف‌های دانشی

پس از ایجاد گراف‌های دانشی فردی، مرحله تجمیع این گراف‌ها به منظور تحلیل جمعی دانش آغاز می‌شود. در این مرحله، گراف‌های مربوط به یادگیرندگان مختلف با حفظ ساختار مفهومی مشترک در یک گراف کلان ادغام می‌شوند. این گراف کلان، نمایی جامع از وضعیت دانش در سطح یک گروه از یادگیرندگان ارائه می‌دهد و امکان تحلیل الگوهای یادگیری، شناسایی نقاط ضعف مشترک و بررسی توزیع دانش در سطح جمعی را فراهم می‌سازد.

شناسایی گلوگاه‌های آموزشی

در ادامه، این گراف با استفاده از روش‌های تحلیل شبکه مورد بررسی قرار می‌گیرد تا مفاهیم کلیدی و گره‌های تأثیرگذار در ساختار دانش شناسایی شوند. هدف اصلی در این مرحله، شناسایی گره‌هایی است که نقش حیاتی در انتقال دانش دارند و ضعف در آن‌ها می‌تواند منجر به اختلال در یادگیری سایر مفاهیم شود. این گره‌ها به عنوان گلوگاه‌های آموزشی در نظر گرفته می‌شوند.

تحلیل عمیق فرآیند یادگیری

در نهایت، با بهره‌گیری از ساختار گراف و امکان ردیابی مسیرهای دانش، می‌توان فرآیند یادگیری را به صورت عمیق‌تری تحلیل کرد. در این راستا، استفاده از مفاهیمی نظیر بازردیابی دانش امکان بررسی این موضوع را فراهم می‌کند که کدام مفاهیم و روابط بیشترین تأثیر را در شکل‌گیری پاسخ‌های یادگیرنده داشته‌اند. این قابلیت، علاوه بر افزایش تفسیرپذیری مدل، امکان ارائه بازخوردهای دقیق‌تر و طراحی مداخلات آموزشی هدفمند را فراهم می‌سازد. به طور کلی، چارچوب پیشنهادی با ترکیب قابلیت‌های مدل‌سازی پویا در ردیابی دانش، نمایش ساخت‌یافته در گراف‌های دانشی و توانایی‌های درک و تولید مدل‌های زبانی بزرگ، یک رویکرد جامع برای تحلیل و بهبود فرآیند یادگیری ارائه می‌دهد که می‌تواند در توسعه سیستم‌های آموزش تطبیقی و هوشمند نقش مؤثری ایفا کند.

۵-۱- ساختار پایان‌نامه

این گزارش در چند فصل تنظیم شده است.

در فصل اول، مقدمه، بیان مسئله، تعریف موضوع، اهداف و روش انجام تحقیق توضیح داده شد. این فصل چارچوب کلی پروژه را مشخص می‌کند و نشان می‌دهد سامانه طراحی‌شده چه مسئله‌ای را حل می‌کند و چرا استفاده از گراف دانش و مدل زبانی برای این مسئله مناسب است.

فصل دوم به پیشینه تحقیق اختصاص دارد. در این فصل، پژوهش‌های مرتبط با ردیابی دانش، گراف دانش، مدل‌های زبانی بزرگ و کاربرد آن‌ها در سامانه‌های آموزشی هوشمند بررسی می‌شود. هدف این فصل آن است که جایگاه پژوهش حاضر در میان

مطالعات قبلی مشخص شود و نشان داده شود که ترکیب ردیابی دانش، گراف دانش و مدل‌های زبانی چگونه می‌تواند برای تحلیل یادگیری و شناسایی گلوگاه‌های آموزشی مورد استفاده قرار گیرد.

فصل سوم به مفاهیم اولیه و پیش‌زمینه اختصاص دارد. در این فصل، مفاهیمی مانند برنامه وب، Flask، مدل زبانی، JSON، گراف جهت‌دار، گراف دانش، مدل دانشجو، میزان تسلط، سیستم غیرقطعی، تولید محتوا، ارزیابی پاسخ و نوار پیشرفت توضیح داده می‌شود. هدف فصل دوم آن است که خواننده قبل از ورود به جزئیات طراحی و پیاده‌سازی، با مفاهیم پایه‌ای مورد استفاده در پروژه آشنا شود.

فصل چهارم به مدل‌سازی، شبیه‌سازی و طراحی و پیاده‌سازی روش پیشنهادی می‌پردازد. در این فصل، ساختار کلی سامانه، جریان داده، مراحل اجرای شبیه‌سازی، تعریف دانشجویان، نحوه تولید curriculum، روش ارزیابی، معیارهای خروجی و تحلیل نتایج و همچنین جزئیات کد توضیح داده می‌شود. این فصل نشان می‌دهد که پروژه فقط یک برنامه وب ساده نیست، بلکه یک مدل شبیه‌سازی آموزشی دارد که در آن مفاهیم، دانشجویان، پاسخ‌ها و تسلط‌ها به صورت منظم مدیریت می‌شوند.

فصل پنجم به ارائه و تحلیل نتایج حاصل از اجرای سامانه پیشنهادی اختصاص دارد. در این فصل، خروجی‌هایی مانند گراف دانش، میزان تسلط دانشجویان بر مفاهیم، روند پیشرفت یادگیری، دشواری مفاهیم و نمودارهای تحلیلی بررسی می‌شود. هدف این فصل نشان دادن عملکرد سامانه در شبیه‌سازی فرایند یادگیری و استخراج شاخص‌های قابل تحلیل از تعاملات دانشجویان است.

فصل ششم شامل جمع‌بندی، نتیجه‌گیری، نوآوری‌ها، محدودیت‌ها و پیشنهادها است. در این فصل، دستاوردهای پروژه مرور می‌شود و مشخص می‌گردد سامانه تا چه اندازه اهداف تعریف‌شده را پوشش داده است. همچنین محدودیت‌هایی مانند وابستگی به کیفیت پاسخ مدل زبانی، احتمال خطا در JSON، زمان‌بر بودن فراخوانی‌های مدل و نیاز به امنیت بیشتر در نگهداری کلید API مطرح می‌شود.

فصل هفتم به مراجع اختصاص دارد. از آنجا که متن حاضر بر اساس کد پیاده‌سازی شده نوشته شده و منابع خارجی مشخصی توسط مجری پروژه ارائه نشده است، بخش مراجع باید پس از تعیین منابع رسمی مورد استفاده تکمیل شود. منابع مناسب می‌توانند شامل مستندات NumPy، Matplotlib، NetworkX، Flask و مستندات API مدل زبانی باشند.

فصل ۲: پیشینه ی تحقیق

در سال‌های اخیر، توسعه سامانه‌های آموزشی هوشمند به یکی از زمینه‌های مهم پژوهشی در علوم کامپیوتر، هوش مصنوعی و فناوری آموزشی تبدیل شده است. هدف اصلی این سامانه‌ها آن است که فرایند آموزش از حالت ثابت و یکسان برای همه یادگیرندگان خارج شود و بر اساس وضعیت واقعی دانش، نقاط ضعف، مسیر یادگیری و سطح تسلط هر فرد، محتوای مناسب‌تری ارائه گردد. در چنین رویکردی، یادگیرنده فقط بر اساس یک نمره نهایی یا نتیجه آزمون ارزیابی نمی‌شود، بلکه رفتار آموزشی او در طول زمان، پاسخ‌های قبلی، مفاهیم یادگرفته‌شده، مفاهیم ضعیف و ارتباط میان مفاهیم نیز مورد تحلیل قرار می‌گیرد.

یکی از مفاهیم کلیدی در این حوزه، **ردیابی دانش** یا Knowledge Tracing است. ردیابی دانش به مجموعه روش‌هایی گفته می‌شود که تلاش می‌کنند وضعیت دانش یادگیرنده را در طول زمان و بر اساس تعاملات او با سامانه آموزشی مدل‌سازی کنند. در این روش‌ها، سامانه بررسی می‌کند که یادگیرنده در گذشته به چه سؤال‌هایی پاسخ داده، عملکرد او چگونه بوده، و بر اساس این داده‌ها احتمال موفقیت او در پاسخ‌های آینده را پیش‌بینی می‌کند. اهمیت ردیابی دانش در این است که امکان مدل‌سازی پویا و تدریجی یادگیری را فراهم می‌سازد و می‌تواند مبنای تصمیم‌گیری در سامانه‌های آموزش تطبیقی قرار گیرد.

با این حال، روش‌های سنتی ردیابی دانش معمولاً با چند محدودیت مهم مواجه هستند. بسیاری از این روش‌ها دانش یادگیرنده را به صورت بردارهای عددی یا حالت‌های پنهان مدل می‌کنند و روابط مفهومی میان اجزای درس را به صورت صریح نشان نمی‌دهند. در حالی که در آموزش واقعی، مفاهیم از یکدیگر مستقل نیستند. برخی مفاهیم پیش‌نیاز مفاهیم دیگر هستند و ضعف در یک مفهوم پایه می‌تواند باعث اختلال در یادگیری چند مفهوم وابسته شود. به همین دلیل، استفاده از **گراف دانش** می‌تواند نقش مهمی در تحلیل ساختار مفهومی آموزش ایفا کند. در گراف دانش، مفاهیم به صورت گره و روابط میان آن‌ها به صورت یال نمایش داده می‌شوند. این ساختار امکان تحلیل دقیق‌تر مسیر یادگیری، تشخیص مفاهیم کلیدی و شناسایی گلوگاه‌های آموزشی را فراهم می‌کند.

از سوی دیگر، ظهور **مدل‌های زبانی بزرگ** فرصت جدیدی برای طراحی سامانه‌های آموزشی ایجاد کرده است. این مدل‌ها می‌توانند متن آموزشی تولید کنند، پاسخ یادگیرنده را تحلیل کنند، سؤال بسازند، بازخورد بدهند و حتی نقش دانشجوی شبیه‌سازی‌شده یا معلم را در یک محیط آموزشی ایفا کنند. ترکیب مدل‌های زبانی بزرگ با ردیابی دانش و گراف دانش می‌تواند بستری مناسب برای طراحی سامانه‌هایی فراهم کند که هم از نظر محتوایی انعطاف‌پذیر باشند و هم از نظر تحلیلی قابلیت تفسیر داشته باشند.

در این فصل، پژوهش‌های مرتبط با موضوع پایان‌نامه در چند محور بررسی می‌شوند: نخست، روش‌های ردیابی دانش عمیق و توجه‌محور؛ دوم، روش‌های گراف‌محور در ردیابی دانش؛ سوم، کاربرد گراف‌های دانش در آموزش؛ چهارم، استفاده از مدل‌های زبانی بزرگ در شخصی‌سازی یادگیری؛ و در نهایت، پژوهش‌هایی که مدل‌های زبانی بزرگ و گراف دانش را برای استدلال ساختاری ترکیب کرده‌اند. این بررسی نشان می‌دهد که پژوهش حاضر در نقطه تلاقی چند حوزه مهم قرار دارد و تلاش می‌کند از ظرفیت مشترک آن‌ها برای شناسایی گلوگاه‌های یادگیری استفاده کند.

۲-۱ ردیابی دانش عمیق

یکی از پژوهش‌های مهم و اثرگذار در حوزه ردیابی دانش، مقاله Deep Knowledge Tracing است که توسط Piech و همکاران ارائه شد [۱]. این پژوهش نقطه عطفی در گذار از مدل‌های کلاسیک و احتمالاتی به مدل‌های مبتنی بر یادگیری عمیق محسوب می‌شود. پیش از این، بسیاری از روش‌های ردیابی دانش بر مدل‌های ساده‌تری مانند Bayesian Knowledge Tracing تکیه داشتند که در آن‌ها وضعیت دانش یادگیرنده معمولاً به صورت مجموعه‌ای از متغیرهای محدود و وابسته به مهارت‌های مشخص مدل می‌شد. اگرچه این روش‌ها قابل تفسیر بودند، اما برای نمایش پیچیدگی‌های واقعی فرایند یادگیری محدودیت داشتند.

در مدل DKT، از شبکه‌های عصبی بازگشتی برای مدل‌سازی توالی تعاملات دانشجو استفاده شد. ایده اصلی این بود که پاسخ‌های گذشته دانشجو می‌توانند به عنوان یک توالی زمانی در نظر گرفته شوند و مدل بازگشتی می‌تواند الگوهای پنهان در این توالی را یاد بگیرد. در این رویکرد، نیازی نیست همه روابط میان مفاهیم به صورت دستی تعریف شوند؛ زیرا مدل می‌تواند از داده‌های آموزشی، نمایش‌هایی نهفته از وضعیت دانش یادگیرنده استخراج کند. نویسندگان نشان دادند که استفاده از شبکه‌های عصبی بازگشتی می‌تواند عملکرد پیش‌بینی پاسخ آینده دانشجو را نسبت به روش‌های قبلی بهبود دهد و حتی به طراحی بهتر مسیر آموزشی کمک کند.

اهمیت این مقاله برای پژوهش حاضر در آن است که مفهوم مدل‌سازی تدریجی دانش یادگیرنده را به صورت جدی وارد چارچوب یادگیری عمیق کرد. در پروژه حاضر نیز سطح تسلط هر دانشجو بر مفاهیم مختلف در طول تکرارهای آموزشی تغییر می‌کند و سامانه تلاش می‌کند این تغییرات را در قالب مقدار mastery نمایش دهد. با این حال، تفاوت مهم پژوهش حاضر با DKT این است که DKT عمدتاً بر نمایش نهفته و برداری دانش تکیه دارد، در حالی که در این پروژه، وضعیت دانش علاوه بر مقدار عددی، در قالب گراف دانش و روابط مفهومی نیز نمایش داده می‌شود. بنابراین، DKT از نظر ایده ردیابی تدریجی دانش الهام‌بخش است، اما برای تحلیل ساختاری گلوگاه‌های یادگیری کافی نیست.

۲-۲ ردیابی دانش مبتنی بر توجه

پس از معرفی روش‌های مبتنی بر شبکه‌های عصبی بازگشتی، پژوهش‌های جدیدتر تلاش کردند محدودیت‌های این مدل‌ها را کاهش دهند. یکی از این محدودیت‌ها، دشواری تفسیر وضعیت داخلی مدل و همچنین ضعف در توجه دقیق به تعاملات مهم گذشته بود. در همین راستا، مقاله Context-Aware Attentive Knowledge Tracing یا AKT توسط Ghosh و همکاران ارائه شد [۲]. این مدل از سازوکار attention استفاده می‌کند تا مشخص کند کدام تعاملات قبلی یادگیرنده برای پیش‌بینی عملکرد آینده او اهمیت بیشتری دارند.

در AKT، به جای آنکه همه پاسخ‌های قبلی دانشجو به صورت یکسان در نظر گرفته شوند، مدل تلاش می‌کند بر تعاملات مرتبط‌تر تمرکز کند. این موضوع در آموزش اهمیت زیادی دارد، زیرا همه پاسخ‌های قبلی دانشجو به یک اندازه برای مفهوم فعلی مهم نیستند. مثلاً اگر دانشجو در یک مفهوم پایه مشکل داشته باشد، پاسخ‌های مربوط به همان مفهوم یا مفاهیم نزدیک به آن برای پیش‌بینی عملکرد آینده اهمیت بیشتری دارند. AKT همچنین با استفاده از مؤلفه‌هایی الهام‌گرفته از مدل‌های شناختی و روان‌سنجی، تلاش می‌کند بخشی از تفسیرپذیری را به مدل بازگرداند. طبق گزارش مقاله، این مدل در چند مجموعه داده واقعی عملکرد بهتری نسبت به برخی روش‌های قبلی داشته و برای ارائه بازخورد و شخصی‌سازی آموزش مناسب‌تر است.

ارتباط AKT با پژوهش حاضر از چند جهت قابل توجه است. نخست اینکه هر دو به مسئله ردیابی دانش به عنوان یک فرایند پویا نگاه می‌کنند. دوم اینکه هر دو اهمیت زمینه و تاریخچه یادگیری را در نظر می‌گیرند. سوم اینکه AKT نشان می‌دهد برای طراحی سامانه‌های آموزشی مؤثر، فقط پیش‌بینی پاسخ آینده کافی نیست، بلکه تفسیرپذیری و امکان ارائه بازخورد نیز اهمیت دارد. در پروژه حاضر نیز تلاش شده است که نتیجه یادگیری فقط در قالب یک عدد کلی ارائه نشود، بلکه روند یادگیری، مقدار

تسلط هر مفهوم، گراف دانش، و وضعیت مفاهیم ضعیف نمایش داده شود. با این حال، AKT همچنان ساختار مفهومی را به صورت گراف صریح نمایش نمی‌دهد؛ در حالی که پژوهش حاضر از گراف دانش برای نمایش روابط پیش‌نیازی و تحلیل گلوگاه‌های آموزشی استفاده می‌کند.

۳-۲ ردیابی دانش گراف‌محور

با افزایش توجه به روابط میان مفاهیم آموزشی، پژوهشگران به استفاده از ساختارهای گرافی در ردیابی دانش روی آوردند. یکی از پژوهش‌های مهم در این زمینه، مقاله Graph-based Knowledge Tracing یا GKT است [۳]. در این مقاله، دانش آموزشی به صورت یک گراف مدل می‌شود که در آن گره‌ها نماینده مفاهیم و یال‌ها نماینده روابط میان آن‌ها هستند. سپس مسئله ردیابی دانش به صورت یک مسئله طبقه‌بندی در سطح گره و در طول زمان بازتعریف می‌شود. در واقع، مدل تلاش می‌کند وضعیت هر مفهوم را در گراف و در طول تعاملات یادگیرنده به‌روزرسانی کند.

مزیت اصلی GKT در این است که برخلاف مدل‌های کاملاً توالی‌محور، ساختار مفهومی درس را وارد مدل می‌کند. وقتی مفاهیم به صورت گراف نمایش داده می‌شوند، مدل می‌تواند اثر یک مفهوم بر مفاهیم دیگر را نیز در نظر بگیرد. برای مثال، اگر یادگیرنده در یک مفهوم پایه ضعف داشته باشد، این ضعف می‌تواند بر مفاهیم وابسته نیز اثر بگذارد. این نگاه برای تحلیل آموزشی بسیار مهم است، زیرا در بسیاری از موارد، مشکل اصلی دانشجو در سؤال فعلی نیست، بلکه در یکی از مفاهیم پیش‌نیاز قرار دارد. GKT با تبدیل ساختار دانش به گراف، امکان تحلیل چنین وابستگی‌هایی را فراهم می‌کند.

این مقاله از نزدیک‌ترین پژوهش‌ها به پایان‌نامه حاضر است، زیرا در هر دو، مفهوم و رابطه آموزشی نقش اصلی دارند. با این حال، تفاوت مهم این است که GKT بیشتر بر پیش‌بینی عملکرد دانشجو با استفاده از مدل گرافی تمرکز دارد، در حالی که پروژه حاضر هدف گسترده‌تری دارد: تولید مفاهیم با کمک مدل زبانی، ساخت گراف دانش، شبیه‌سازی چند دانشجو، ارزیابی پاسخ‌ها، به‌روزرسانی mastery و نمایش نتایج به صورت گراف و نمودار. بنابراین، GKT از نظر استفاده از گراف برای ردیابی دانش یک پایه مهم محسوب می‌شود، اما پژوهش حاضر تلاش می‌کند این ایده را با مدل‌های زبانی بزرگ و تحلیل گلوگاه‌های یادگیری ترکیب کند.

۴-۲ مدل تعامل گرافی در ردیابی دانش

پژوهش مهم دیگر در حوزه ردیابی دانش گراف‌محور، مقاله GIKT: A Graph-based Interaction Model for Knowledge Tracing است [۴]. این مقاله به یکی از ضعف‌های مدل‌های قبلی اشاره می‌کند: بسیاری از روش‌ها نمی‌توانند تعامل میان دانشجو، سؤال و مهارت را به صورت یکپارچه مدل کنند. در آموزش واقعی، پاسخ یک دانشجو فقط به سطح دانش عمومی او وابسته نیست، بلکه به ویژگی سؤال، مهارت‌های موردنیاز برای پاسخ، و ارتباط میان آن مهارت‌ها نیز بستگی دارد.

در GIKT، از شبکه‌های کانولوشنی گرافی برای مدل‌سازی روابط میان سؤال‌ها و مهارت‌ها استفاده می‌شود. این مدل تلاش می‌کند هم وابستگی‌های بلندمدت در تاریخچه پاسخ‌های دانشجو را در نظر بگیرد و هم رابطه میان سؤال و مفهوم را بهتر نمایش دهد. اهمیت این کار در آن است که برخی سؤال‌ها ممکن است به چند مهارت یا مفهوم وابسته باشند و مدل باید بتواند این وابستگی‌های چندگانه را در تحلیل خود وارد کند. مقاله GIKT نشان می‌دهد که استفاده از ساختار گرافی می‌تواند درک بهتری از تعاملات آموزشی ایجاد کند و پیش‌بینی عملکرد دانشجو را بهبود دهد.

GIKT برای پایان‌نامه حاضر اهمیت زیادی دارد، زیرا نشان می‌دهد که ردیابی دانش زمانی دقیق‌تر می‌شود که رابطه میان عناصر آموزشی به صورت ساخت‌یافته مدل شود. در پروژه حاضر نیز هر مفهوم به عنوان یک گره در گراف دانش تعریف می‌شود

و روابط پیش‌نیازی میان مفاهیم به‌صورت یال‌های جهت‌دار نمایش داده می‌شوند. همچنین پاسخ هر دانشجوی شبیه‌سازی‌شده برای هر مفهوم ارزیابی می‌شود و مقدار mastery همان مفهوم تغییر می‌کند. تفاوت اصلی این است که GIKT بیشتر بر مدل یادگیری ماشین برای پیش‌بینی پاسخ تمرکز دارد، اما پروژه حاضر علاوه بر تحلیل دانش، یک سامانه شبیه‌سازی آموزشی مبتنی بر مدل زبانی و گراف دانش طراحی کرده است. بنابراین، GIKT می‌تواند به‌عنوان یکی از مهم‌ترین پشتوانه‌های نظری برای بخش گراف‌محور پایان‌نامه استفاده شود.

۲-۵ گراف دانش در آموزش

در کنار پژوهش‌های مربوط به ردیابی دانش، حوزه دیگری که ارتباط مستقیم با پژوهش حاضر دارد، گراف دانش آموزشی است. مقاله مروری A Survey of Knowledge Graph Approaches and Applications in Education به‌صورت جامع کاربرد گراف‌های دانش در آموزش را بررسی می‌کند [۵]. این مقاله نشان می‌دهد که گراف دانش می‌تواند برای سازمان‌دهی محتوای آموزشی، نمایش روابط میان مفاهیم، پیشنهاد مسیر یادگیری، سیستم‌های توصیه‌گر آموزشی و تحلیل یادگیری استفاده شود.

اهمیت گراف دانش در آموزش از آنجا ناشی می‌شود که محتوای آموزشی ذاتاً ساختارمند است. مفاهیم یک درس معمولاً به‌صورت زنجیره‌ای یا شبکه‌ای به هم وابسته‌اند. برای مثال، در یک درس برنامه‌نویسی، فهم متغیرها پیش‌نیاز فهم ساختارهای شرطی و حلقه‌ها است. در یک درس ریاضی، فهم تابع پیش‌نیاز فهم مشتق و انتگرال است. اگر این روابط به‌صورت صریح در سامانه آموزشی مدل شوند، معلم یا سامانه هوشمند می‌تواند بهتر تشخیص دهد که ضعف دانشجو دقیقاً در کدام قسمت قرار دارد. مقاله مروری مذکور نشان می‌دهد که پژوهش درباره گراف دانش در آموزش در سال‌های اخیر رشد قابل توجهی داشته و این حوزه برای شخصی‌سازی یادگیری اهمیت فزاینده‌ای پیدا کرده است.

ارتباط این مقاله با پژوهش حاضر بسیار مستقیم است. در پروژه حاضر، گراف دانش فقط یک تصویر تزئینی نیست، بلکه هسته اصلی مدل‌سازی آموزشی است. مفاهیم به‌صورت گره، روابط پیش‌نیازی به‌صورت یال و مقدار تسلط به‌عنوان ویژگی هر گره ذخیره می‌شود. این ساختار باعث می‌شود بتوان وضعیت هر مفهوم را جداگانه تحلیل کرد و مفاهیمی را که mastery پایین‌تری دارند، به‌عنوان نقاط ضعف یا گلوگاه‌های احتمالی شناسایی نمود. بنابراین، پژوهش حاضر در ادامه مسیر مطالعات گراف دانش آموزشی قرار می‌گیرد، اما با این تفاوت که ساخت گراف و ارزیابی یادگیری را با استفاده از مدل زبانی بزرگ و شبیه‌سازی چندعامل انجام می‌دهد.

۲-۶ مدل‌های زبانی بزرگ و شخصی‌سازی یادگیری

با ظهور مدل‌های زبانی بزرگ، پژوهش‌های جدیدی در زمینه استفاده از این مدل‌ها در آموزش انجام شده است. یکی از پژوهش‌های مرتبط، مقاله TutorLLM است [۶]. این مقاله یک سامانه توصیه‌گر آموزشی مبتنی بر مدل زبانی بزرگ ارائه می‌کند که در آن از ردیابی دانش و بازیابی افزوده با تولید یا RAG برای شخصی‌سازی پیشنهادها یادگیری استفاده می‌شود. ایده اصلی TutorLLM این است که مدل زبانی به‌تنهایی اگرچه توانایی تولید پاسخ‌های گسترده دارد، اما ممکن است به سطح واقعی دانشجو توجه کافی نداشته باشد. بنابراین، برای شخصی‌سازی بهتر، باید وضعیت یادگیری دانشجو نیز وارد فرایند تولید پاسخ شود.

در TutorLLM، وضعیت یادگیری دانشجو با استفاده از مدل ردیابی دانش تخمین زده می‌شود و سپس اطلاعات مرتبط از منابع آموزشی بازیابی می‌گردد تا مدل زبانی بتواند پاسخ‌ها یا پیشنهادها آموزشی مناسب‌تری تولید کند. این رویکرد نشان می‌دهد که ترکیب LLM با KT و RAG می‌تواند نسبت به استفاده از مدل زبانی عمومی، شخصی‌سازی و کیفیت آموزشی

بهتری فراهم کند. مقاله گزارش می‌کند که این ترکیب توانسته رضایت کاربران و عملکرد آزمون را نسبت به استفاده از LLM عمومی بهبود دهد.

TutorLLM از نظر ایده کلی به پروژه حاضر بسیار نزدیک است، زیرا هر دو به ترکیب مدل زبانی و ردیابی دانش در آموزش می‌پردازند. با این حال، تفاوت مهمی وجود دارد. TutorLLM بیشتر بر توصیه آموزشی و بازایی محتوای مناسب تمرکز دارد، در حالی که پروژه حاضر بر شبیه‌سازی یادگیری، ساخت گراف دانش، ارزیابی پاسخ دانشجویان شبیه‌سازی شده و شناسایی گلوگاه‌های مفهومی تمرکز می‌کند. به بیان دیگر، TutorLLM نشان می‌دهد که مدل زبانی وقتی با وضعیت دانش دانشجو ترکیب شود، می‌تواند آموزش شخصی‌سازی شده‌تری ارائه دهد؛ اما پژوهش حاضر تلاش می‌کند این وضعیت دانش را به شکل قابل مشاهده و گرافی نیز نمایش دهد.

۲-۷ ساخت گراف دانش با کمک مدل‌های زبانی بزرگ

یکی از چالش‌های مهم در استفاده از گراف دانش، ساخت و نگهداری آن است. در روش‌های سنتی، ساخت گراف دانش معمولاً به تعریف دستی مفاهیم، روابط، هستی‌شناسی و استخراج دانش از منابع مختلف نیاز دارد. این فرایند زمان‌بر است و به دانش متخصصان حوزه وابسته است. مقاله مروری LLM-Empowered Knowledge Graph Construction به بررسی این موضوع می‌پردازد که چگونه مدل‌های زبانی بزرگ می‌توانند فرایند ساخت گراف دانش را متحول کنند. [7]

این مقاله توضیح می‌دهد که مدل‌های زبانی بزرگ می‌توانند در مراحل مختلف ساخت گراف دانش نقش داشته باشند؛ از جمله استخراج مفاهیم، استخراج روابط، تکمیل دانش، ادغام منابع و حتی کمک به طراحی ساختارهای مفهومی. در واقع، ظهور LLMها باعث شده است که ساخت گراف دانش از یک فرایند کاملاً دستی و قاعده‌محور، به سمت فرایندهای زبانی، مولد و انعطاف‌پذیر حرکت کند. این موضوع برای حوزه آموزش بسیار مهم است، زیرا در بسیاری از درس‌ها و موضوعات، ساخت دستی گراف مفاهیم کار دشواری است. اگر مدل زبانی بتواند مفاهیم اصلی و روابط پیش‌نیازی را پیشنهاد دهد، سامانه آموزشی می‌تواند سریع‌تر برای موضوعات مختلف گراف اولیه بسازد.

در پروژه حاضر نیز از همین ایده استفاده شده است. سامانه، موضوع آموزشی را از کاربر دریافت می‌کند و سپس با استفاده از مدل زبانی، مفاهیم اصلی و وابستگی‌های میان آن‌ها را تولید می‌کند. سپس این خروجی به صورت گراف دانش نمایش داده می‌شود. بنابراین، این مقاله از نظر نظری پشتوانه مهمی برای استفاده از LLM در ساخت گراف دانش محسوب می‌شود. البته در پروژه حاضر، ساخت گراف دانش در سطح ساده‌تر و کاربردی‌تر انجام شده است و هدف اصلی، ایجاد یک نمونه اولیه برای شبیه‌سازی یادگیری و تحلیل گلوگاه‌ها است.

۲-۸ گراف دانش و ردیابی ساختاری در مدل‌های زبانی

پژوهش KnowTrace یکی دیگر از منابع مرتبط با پایان‌نامه حاضر است [۸]. این مقاله در حوزه پاسخ‌گویی چندمرحله‌ای و RAG مطرح شده است، اما از نظر مفهومی ارتباط مهمی با پروژه حاضر دارد. KnowTrace به این مسئله می‌پردازد که در روش‌های معمول RAG، اطلاعات بازایی شده معمولاً به صورت متن‌های متعدد وارد زمینه مدل زبانی می‌شوند و این موضوع می‌تواند باعث افزایش بار زمینه‌ای و دشواری تشخیص ارتباط میان قطعات اطلاعات شود. برای حل این مشکل، KnowTrace تلاش می‌کند به جای انباشتن متن، دانش موردنیاز را به صورت سه‌تایی‌های ساخت‌یافته و یک گراف دانش مرتبط با پرسش سازمان‌دهی کند.

اهمیت KnowTrace در آن است که نشان می‌دهد ساختار گرافی می‌تواند فرایند استدلال مدل زبانی را قابل فهم‌تر و مؤثرتر کند. این مقاله همچنین مفهوم knowledge backtracing را مطرح می‌کند؛ یعنی بازردیابی اینکه کدام بخش‌های دانش در رسیدن به پاسخ نهایی نقش داشته‌اند. این ایده از نظر آموزشی بسیار مهم است، زیرا در تحلیل یادگیری نیز فقط دانستن پاسخ نهایی کافی نیست. باید مشخص شود کدام مفاهیم، روابط یا مسیرهای دانشی در شکل‌گیری پاسخ نقش داشته‌اند و ضعف در کدام بخش می‌تواند مانع یادگیری شود.

در پروژه حاضر، اگرچه KnowTrace به صورت مستقیم برای آموزش طراحی نشده، اما ایده آن با تحلیل ساختاری یادگیری ارتباط دارد. پروژه حاضر نیز تلاش می‌کند دانش را صرفاً به صورت متن یا نمره نمایش ندهد، بلکه مفاهیم و روابط را در قالب گراف سازمان‌دهی کند. همچنین هدف شناسایی گلوگاه‌های یادگیری به نوعی با ایده backtracing مرتبط است؛ زیرا هر دو به دنبال فهم این هستند که کدام بخش از ساختار دانش در نتیجه نهایی نقش مهم‌تری دارد. بنابراین، KnowTrace می‌تواند به عنوان پشتوانه‌ای برای استفاده از گراف دانش در افزایش تفسیرپذیری و تحلیل مسیرهای دانشی در کنار مدل‌های زبانی بزرگ مطرح شود.

۹-۲ جمع‌بندی پیشینه و جایگاه پژوهش حاضر

بررسی پژوهش‌های پیشین نشان می‌دهد که حوزه ردیابی دانش از مدل‌های ساده و احتمالاتی به سمت مدل‌های عمیق، توجه‌محور و گراف‌محور حرکت کرده است. مقاله DKT نشان داد که شبکه‌های عصبی بازگشتی می‌توانند توالی تعاملات دانشجو را مدل کرده و برای پیش‌بینی عملکرد آینده استفاده شوند [۱]. سپس AKT با استفاده از سازوکار توجه، تلاش کرد تعاملات مهم‌تر گذشته را بهتر تشخیص دهد و قابلیت تفسیرپذیری را افزایش دهد [۲]. در ادامه، روش‌هایی مانند GKT و GIKT نشان دادند که وارد کردن ساختار گرافی مفاهیم، سؤال‌ها و مهارت‌ها می‌تواند مدل‌سازی دانش را دقیق‌تر و از نظر آموزشی معنادارتر کند. [3,4]

از طرف دیگر، پژوهش‌های مربوط به گراف دانش آموزشی نشان داده‌اند که گراف دانش می‌تواند برای نمایش ساختار درس، روابط پیش‌نیازی، توصیه مسیر یادگیری و تحلیل آموزشی به کار رود [۵]. این مسئله با هدف پژوهش حاضر کاملاً همسو است، زیرا در این پروژه نیز ساختار مفاهیم آموزشی در قالب گراف مدل می‌شود و مقدار تسلط هر مفهوم در طول شبیه‌سازی تغییر می‌کند.

پژوهش‌های جدیدتر درباره مدل‌های زبانی بزرگ نیز نشان می‌دهند که LLM ها می‌توانند در آموزش نقش مهمی ایفا کنند، اما استفاده مؤثر از آن‌ها نیازمند ترکیب با روش‌هایی مانند ردیابی دانش، RAG و گراف دانش است TutorLLM. نمونه‌ای از ترکیب LLM، KT و RAG برای شخصی‌سازی یادگیری است [۶]. همچنین پژوهش‌های مربوط به ساخت گراف دانش با کمک LLM نشان می‌دهند که مدل‌های زبانی می‌توانند در استخراج مفاهیم و روابط نقش مهمی داشته باشند [۷]. در نهایت، KnowTrace نشان می‌دهد که ساختاردهی دانش در قالب گراف می‌تواند استدلال مدل زبانی را شفاف‌تر و قابل ردیابی‌تر کند. [8]

با وجود این پیشرفت‌ها، هنوز خلأیی در ترکیب یکپارچه این حوزه‌ها وجود دارد. بسیاری از پژوهش‌ها یا بر پیش‌بینی عملکرد دانشجو تمرکز کرده‌اند، یا بر ساخت گراف دانش، یا بر استفاده از مدل زبانی برای توصیه آموزشی. اما پژوهش حاضر تلاش می‌کند این اجزا را در قالب یک سامانه واحد ترکیب کند: مدل زبانی برای تولید مفاهیم و روابط، گراف دانش برای نمایش ساختار مفهومی، شبیه‌سازی چند دانشجو برای ایجاد تعاملات آموزشی، ارزیابی پاسخ‌ها برای محاسبه mastery، و تحلیل گراف برای شناسایی مفاهیم ضعیف و گلوگاه‌های یادگیری.

بنابراین، جایگاه پژوهش حاضر را می‌توان در نقطه اتصال سه حوزه دانست: ردیابی دانش، گراف دانش آموزشی و مدل‌های زبانی بزرگ. نوآوری اصلی پروژه در این نیست که یک مدل جدید یادگیری عمیق برای KT ارائه کند، بلکه در طراحی یک چارچوب کاربردی و قابل مشاهده است که می‌تواند فرایند یادگیری را شبیه‌سازی کند، وضعیت مفاهیم را در گراف نمایش دهد و مسیرهای ضعف یادگیری را برای تحلیل آموزشی آشکار سازد.

فصل ۳: مفاهیم اولیه و پیش زمینه

برای درک کامل پروژه، لازم است مفاهیم پایه‌ای مورد استفاده در سامانه توضیح داده شوند. این پروژه از چند فناوری و چند مفهوم نظری تشکیل شده است و هرکدام نقش مشخصی در عملکرد نهایی برنامه دارند. اگرچه خروجی نهایی برای کاربر به شکل یک صفحه وب، یک گزارش متنی و چند نمودار دیده می‌شود، اما پشت این خروجی‌ها مجموعه‌ای از اجزای فنی در حال کار هستند. این اجزا شامل Flask برای پیاده‌سازی وب، مدل زبانی برای تولید و ارزیابی محتوا، JSON برای تبادل داده ساخت‌یافته، NetworkX برای ساخت گراف دانش، Matplotlib برای رسم نمودارها، NumPy برای محاسبات عددی، و threading برای اجرای پس‌زمینه هستند.

در این فصل، ابتدا معماری کلی پروژه از نظر مفهومی توضیح داده می‌شود. سپس گراف دانش و نحوه نمایش مفاهیم آموزشی در قالب گره و یال بررسی می‌شود. پس از آن، نقش مدل زبانی در تولید مفاهیم، پاسخ‌دهی دانشجویان و ارزیابی پاسخ‌ها تشریح می‌شود. در ادامه، مفهوم سیستم غیرقطعی بررسی می‌شود، زیرا خروجی مدل زبانی همیشه کاملاً ثابت و قابل پیش‌بینی نیست. سپس مدل دانشجو، حافظه، میزان تسلط و روش به‌روزرسانی آن توضیح داده می‌شود.

یکی از مفاهیم مهم در این پروژه، جداسازی مراحل مختلف شبیه‌سازی است. برنامه به‌صورت مرحله‌ای عمل می‌کند: ابتدا مفاهوم‌ها را می‌سازد، سپس گراف را شکل می‌دهد، بعد دانشجویان را تعریف می‌کند، سپس یادگیری را انجام می‌دهد، پاسخ‌ها را ارزیابی می‌کند، گراف را به‌روزرسانی می‌کند و در نهایت گزارش می‌سازد. این تقسیم‌بندی باعث می‌شود هر بخش از پروژه وظیفه مشخصی داشته باشد و تحلیل و توسعه برنامه ساده‌تر شود.

از نظر آموزشی، پروژه بر این فرض بنا شده است که یادگیری را می‌توان تا حدی به‌صورت تغییر در مقدار تسلط روی مفاهیم مدل کرد. هر مفهوم یک مقدار عددی دارد که نشان می‌دهد دانشجویان تا چه اندازه آن را یاد گرفته‌اند. اگر پاسخ دانشجو خوب باشد، مقدار تسلط افزایش می‌یابد. اگر پاسخ ضعیف باشد، مقدار تسلط کمتر رشد می‌کند. این مدل ساده است، اما برای نمایش ایده کلی شبیه‌سازی یادگیری مناسب است.

از نظر نرم‌افزاری، پروژه نیازمند مدیریت زمان اجرای طولانی است. چون در هر تکرار و برای هر مفهوم و هر دانشجو، چندین فراخوانی به مدل زبانی انجام می‌شود، اجرای برنامه ممکن است زمان‌بر باشد. به همین دلیل، برنامه از thread جداگانه و سیستم job استفاده می‌کند. این طراحی باعث می‌شود صفحه وب بتواند هم‌زمان وضعیت اجرای برنامه را از سرور بپرسد و نوار پیشرفت را نمایش دهد.

بنابراین فصل دوم، پایه نظری و فنی لازم برای فهم فصل‌های بعدی را فراهم می‌کند. در فصل‌های بعدی، همین مفاهیم در قالب طراحی و پیاده‌سازی واقعی پروژه استفاده خواهند شد.

۳-۱- بخش اول : مفاهیم پایه

پروژه حاضر بر پایه چند مفهوم کلیدی ساخته شده است. نخستین مفهوم، برنامه وب است. برنامه وب نرم‌افزاری است که کاربر از طریق مرورگر با آن ارتباط برقرار می‌کند. در این پروژه، کاربر از طریق صفحه اصلی، موضوع آموزشی و پارامترهای شبیه‌سازی را وارد می‌کند. سپس برنامه در سمت سرور عملیات لازم را انجام می‌دهد و نتیجه را دوباره در مرورگر نمایش می‌دهد. برای ساخت این بخش از Flask استفاده شده است.

فلسک یک چارچوب سبک برای توسعه برنامه‌های وب در زبان پایتون^۲ است. در این پروژه، فلسک به‌عنوان بخش اصلی رابط وب استفاده شده و وظیفه دریافت ورودی از کاربر، ارسال آن به بخش‌های پردازشی و نمایش نتایج را بر عهده دارد.

در کد پروژه، ابتدا شیء اصلی برنامه با نام app ساخته می‌شود. سپس مسیرهای مختلف وب^۴ تعریف می‌گردند تا مشخص شود هر درخواست کاربر، مانند نمایش صفحه اصلی، شروع شبیه‌سازی یا دریافت وضعیت پیشرفت، توسط کدام بخش از برنامه پاسخ داده شود. این ساختار باعث می‌شود اجزای سامانه به‌صورت منظم از هم جدا شوند و برنامه به شکل یک سامانه وب تعاملی عمل کند.

مفهوم دوم، مدل زبانی است. مدل زبانی در این پروژه وظیفه تولید متن و ارزیابی متن را بر عهده دارد. تابع generate_curriculum از مدل زبانی می‌خواهد مفاهیم و وابستگی‌ها را استخراج کند. متد answer در کلاس Student از مدل زبانی برای تولید پاسخ دانشجو استفاده می‌کند. تابع evaluate نیز پاسخ دانشجو را نمره‌گذاری می‌کند. بنابراین مدل زبانی در سه نقش ظاهر می‌شود: تولیدکننده ساختار آموزشی، شبیه‌ساز دانشجو و ارزیاب پاسخ. pp

مفهوم سوم، JSON است. چون پاسخ مدل زبانی باید توسط برنامه پردازش شود، لازم است خروجی آن ساختاریافته باشد. در پرامپت تابع generate_curriculum از مدل خواسته می‌شود خروجی را فقط در قالب JSON تولید کند. ساختار مورد انتظار شامل دو کلید concepts و dependencies است. کلید concepts فهرست مفاهیم را نگه می‌دارد و کلید dependencies فهرست یال‌های پیش‌نیازی را شامل می‌شود. تابع extract_json برای استخراج JSON از پاسخ مدل طراحی شده است.

مفهوم چهارم، گراف دانش است. گراف دانش ساختاری است که در آن مفاهیم آموزشی و روابط میان آن‌ها مدل می‌شوند. در این پروژه، گراف دانش با استفاده از nx.DiGraph() ساخته می‌شود. این نوع گراف جهت‌دار است؛ یعنی یال‌ها جهت دارند و می‌توانند رابطه پیش‌نیازی را نشان دهند. اگر یک یال از A به B وجود داشته باشد، معنای آن این است که A در مسیر یادگیری قبل از B قرار می‌گیرد یا برای فهم B اهمیت دارد.

مفهوم پنجم، میزان تسلط است. در برنامه، برای هر گره یک مقدار mastery ذخیره می‌شود. این مقدار نشان‌دهنده وضعیت یادگیری آن مفهوم است. در ابتدا مقدار همه مفاهیم صفر است. پس از پاسخ دانشجو و ارزیابی پاسخ، مقدار mastery به‌روزرسانی می‌شود. در گراف عمومی، مقدار mastery هر مفهوم برابر میانگین mastery همان مفهوم در میان دانشجویان است.

مفهوم ششم، شبیه‌سازی چنددانشجویی است. برنامه دانشجویان را با سطوح‌های مختلف می‌سازد. این دانشجویان با نام‌های $S_1, S_2, \dots, S_n$ تعریف شده‌اند و سطح آن‌ها ضعیف، متوسط و قوی است. تفاوت سطوح‌ها از طریق `style` و `temperature` در ارتباط با مدل زبانی ایجاد می‌شود. این طراحی باعث می‌شود پاسخ‌های دانشجویان یکسان نباشد و بتوان تفاوت کیفیت پاسخ را مشاهده کرد.

مفهوم هفتم، نمودار و تصویرسازی است. پروژه دو خروجی تصویری تولید می‌کند. نخست، تصویر گراف دانش که در آن گره‌ها، یال‌ها و مقدار `mastery` هر گره نمایش داده می‌شوند. دوم، نمودار روند یادگیری که تغییر `mastery` مفاهیم را در طول تکرارها نشان می‌دهد. این نمودارها با `Matplotlib` ساخته می‌شوند و در مسیر `static/generated` ذخیره می‌گردند.

مفهوم هشتم، اجرای پس‌زمینه است. چون اجرای شبیه‌سازی ممکن است زمان‌بر باشد، برنامه از `thread` استفاده می‌کند. هر شبیه‌سازی به‌عنوان یک `job` ثبت می‌شود و در پس‌زمینه اجرا می‌گردد. صفحه وب می‌تواند وضعیت `job` را از سرور دریافت کند و نوار پیشرفت را به‌روزرسانی کند. این طراحی برای برنامه‌های تعاملی اهمیت زیادی دارد.

سیستم‌های آموزشی هوشمند (ITS) سیستم‌هایی هستند که با استفاده از هوش مصنوعی تلاش می‌کنند فرآیند یادگیری را برای هر دانشجو شخصی‌سازی کنند. این سیستم‌ها قادرند سطح دانش کاربر را تحلیل کرده و محتوای آموزشی مناسب ارائه دهند.

۳-۱-۱- مقدمه‌ای بر ساختار سامانه

ساختار سامانه از چند لایه تشکیل شده است. لایه اول، رابط کاربری وب است. این لایه در فایل `HTML` پیاده‌سازی می‌شود و وظیفه دریافت ورودی از کاربر، نمایش نوار پیشرفت و نمایش نتیجه نهایی را دارد. لایه دوم، بک‌اند `Flask` است که درخواست‌ها را دریافت و پردازش می‌کند. لایه سوم، منطق شبیه‌سازی است که شامل تولید مفاهیم، ساخت دانشجو، اجرای یادگیری و ارزیابی است. لایه چهارم، ابزارهای تحلیلی و تصویری هستند که خروجی گراف و نمودار تولید می‌کنند.

جریان کلی سامانه از ورود کاربر آغاز می‌شود. کاربر موضوع آموزشی را وارد می‌کند؛ برای مثال، یک درس، یک مبحث دانشگاهی یا یک موضوع علمی. سپس تعداد مفاهیم و تعداد تکرارهای شبیه‌سازی را مشخص می‌کند. پس از ارسال فرم، برنامه یک `job` جدید ایجاد می‌کند. ایجاد `job` باعث می‌شود اجرای شبیه‌سازی قابل پیگیری باشد. هر `job` دارای وضعیت، درصد پیشرفت، پیام فعلی، نتیجه، گزارش و خطا است.

بعد از ایجاد `thread.job` جداگانه‌ای برای اجرای شبیه‌سازی ساخته می‌شود. این `thread` تابع `run_simulation_job` را اجرا می‌کند. در این تابع، ابتدا `run_simulation` اجرا می‌شود. تابع `run_simulation` مرکز اصلی منطق پروژه است و تقریباً همه مراحل مهم در آن هماهنگ می‌شوند. پس از پایان اجرای این تابع، گزارش متنی با `build_simulation_report` ساخته می‌شود و نتیجه در دیکشنری `job` ذخیره می‌گردد.

در طول اجرای شبیه‌سازی، تابع `update_job_progress` چندین بار صدا زده می‌شود. این تابع درصد پیشرفت و پیام فعلی `job` را تغییر می‌دهد. برای مثال، در ابتدای کار پیام «Generating curriculum concepts» تنظیم می‌شود. سپس هنگام ساخت دانشجویان، شروع شبیه‌سازی، ارزیابی پاسخ‌ها، تولید گراف، تولید نمودار و آماده‌سازی گزارش، پیام و درصد پیشرفت تغییر می‌کند. این اطلاعات برای نمایش نوار پیشرفت در صفحه وب استفاده می‌شود.

پس از پایان کار، مسیر `/result/<job_id>` صفحه نتیجه را نمایش می‌دهد. در این صفحه، داده‌های موجود در `result` و `report` به قالب HTML داده می‌شوند. نتیجه شامل داده‌های ساخت یافته است و گزارش شامل متن کامل شبیه‌سازی است. تصاویر نیز در مسیر `static` ذخیره شده‌اند و می‌توانند در صفحه نتیجه نمایش داده شوند.

این ساختار نشان می‌دهد که پروژه به‌صورت ماژولار طراحی شده است. هر بخش وظیفه مشخصی دارد. بخش وب مسئول دریافت و نمایش است. بخش مدل زبانی مسئول تولید و ارزیابی متن است. بخش گراف مسئول نگهداری روابط مفهومی و تسلط است. بخش نمودار مسئول تصویرسازی است. بخش `job` نیز مسئول مدیریت اجرای طولانی و اطلاع‌رسانی وضعیت است.

۳-۱-۲- سیستم‌های غیرقطعی

یکی از نکات مهم در این پروژه، وجود رفتار غیرقطعی در خروجی مدل زبانی است. سیستم غیرقطعی سیستمی است که برای یک ورودی کاملاً مشابه، ممکن است همیشه خروجی کاملاً یکسان تولید نکند. در پروژه حاضر، این رفتار به‌ویژه در سه بخش دیده می‌شود: تولید مفاهیم و وابستگی‌ها، پاسخ‌دهی دانشجویان، و ارزیابی پاسخ‌ها. علت اصلی این رفتار، ماهیت مدل زبانی و پارامترهایی مانند `temperature` است.

در تابع `generate_curriculum`، مدل زبانی با `temperature` برابر ۰٫۳ فراخوانی می‌شود. این مقدار نسبتاً پایین است، بنابراین خروجی باید تا حدی پایدار و دقیق باشد. با این حال، حتی در `temperature` پایین نیز ممکن است مدل در اجراهای مختلف، مفاهیم کمی متفاوت یا وابستگی‌های متفاوت تولید کند. به همین دلیل، کد تلاش می‌کند خروجی را در قالب JSON دریافت کند و سپس آن را پاک‌سازی نماید. اگر وابستگی‌ای شامل مفهومی باشد که در فهرست مفاهیم وجود ندارد، آن وابستگی حذف می‌شود.

در کلاس `Student`، رفتار غیرقطعی عمده‌تر برای شبیه‌سازی تفاوت سطح دانشجویان استفاده شده است. دانشجوی ضعیف با `temperature` برابر ۰٫۸ پاسخ می‌دهد. این مقدار باعث می‌شود پاسخ‌ها تنوع و احتمال خطای بیشتری داشته باشند. دانشجوی متوسط با `temperature` برابر ۰٫۶ و دانشجوی قوی با `temperature` برابر ۰٫۳ تعریف شده است. بنابراین سطح دانشجو فقط یک برچسب نیست، بلکه بر نحوه تولید پاسخ اثر می‌گذارد.

در تابع `temperature.evaluate` برابر صفر در نظر گرفته شده است. این انتخاب برای کاهش عدم قطعیت در ارزیابی انجام شده است. هدف این است که ارزیابی پاسخ تا حد امکان پایدار باشد و مدل فقط یک عدد بین صفر تا یک برگرداند. با این حال، همچنان ممکن است مدل به‌جای عدد خالص، متن اضافه تولید کند. به همین دلیل، کد از عبارت منظم استفاده می‌کند تا عدد معتبر را از خروجی استخراج کند. اگر استخراج عدد با خطا مواجه شود، مقدار صفر برگردانده می‌شود.

رفتار غیرقطعی مدل زبانی هم مزیت دارد و هم محدودیت. مزیت آن این است که شبیه‌سازی واقعی‌تر می‌شود، زیرا دانشجویان واقعی هم همیشه پاسخ یکسان نمی‌دهند. همچنین برای موضوعات مختلف، سیستم می‌تواند ساختارهای متفاوتی تولید کند. اما محدودیت آن این است که تکرارپذیری نتایج کاهش می‌یابد. ممکن است یک اجرای برنامه مفاهیمی تولید کند که با اجرای دیگر کمی متفاوت باشند. بنابراین برای گزارش نتایج تجربی دقیق، باید ورودی‌ها و خروجی‌های یک اجرای مشخص ذخیره و همان‌ها تحلیل شوند.

در این پروژه، بخشی از مدیریت عدم قطعیت از طریق کنترل `temperature`، پاک‌سازی وابستگی‌ها، اعتبارسنجی ورودی‌ها، استخراج JSON و مدیریت خطا انجام شده است. با این حال، برای نسخه‌های آینده می‌توان روش‌های قوی‌تری اضافه کرد؛ مانند اعتبارسنجی تعداد مفاهیم، درخواست بازتولید پاسخ در صورت JSON نامعتبر، استفاده از schema مشخص، ذخیره seed در بخش‌های قابل کنترل، و اضافه کردن ارزیابی چندباره برای کاهش خطای نمره‌دهی.

۳-۱-۲-۱- واحد تولید

واحد تولید در این پروژه شامل بخش‌هایی است که محتوای اولیه یا خروجی‌های قابل استفاده را تولید می‌کنند. مهم‌ترین واحد تولید، تابع `generate_curriculum` است. این تابع موضوع آموزشی و تعداد مفاهیم را دریافت می‌کند و از مدل زبانی می‌خواهد دقیقاً همان تعداد مفهوم اصلی را استخراج کند. علاوه بر مفاهیم، روابط پیش‌نیازی میان آن‌ها نیز باید تولید شود. این خروجی، نقطه شروع کل شبیه‌سازی است.

در `prompt` استفاده‌شده در این تابع، به مدل گفته می‌شود که خروجی فقط در قالب JSON باشد. این موضوع اهمیت زیادی دارد، زیرا اگر مدل پاسخ آزاد تولید کند، برنامه نمی‌تواند به راحتی آن را پردازش کند. قالب مورد انتظار شامل دو بخش است: `dependencies` و `concepts`. بخش اول آرایه‌ای از نام مفاهیم است و بخش دوم آرایه‌ای از زوج‌های مفهومی است که رابطه پیش‌نیازی را نشان می‌دهد.

پس از دریافت پاسخ مدل، تابع `extract_json` اجرا می‌شود. این تابع با استفاده از عبارت منظم، بخش JSON موجود در متن را استخراج می‌کند. این طراحی به این دلیل انجام شده است که مدل ممکن است گاهی JSON را داخل بلوک `markdown` یا همراه با متن اضافی برگرداند. تابع تلاش می‌کند اولین ساختار JSON معتبر را پیدا کند و سپس آن را با `json.loads` به دیکشنری Python تبدیل کند.

پس از تبدیل خروجی به داده Python، مفاهیم و وابستگی‌ها استخراج می‌شوند. سپس وابستگی‌ها پاک‌سازی می‌شوند. اگر یک وابستگی آرایه‌ای با طول دو نباشد، حذف می‌شود. اگر دو سر وابستگی در مجموعه مفاهیم وجود نداشته باشند، آن وابستگی معتبر نیست و حذف می‌شود. این مرحله باعث می‌شود گراف دانش فقط شامل یال‌هایی باشد که به مفاهیم واقعی اشاره می‌کنند.

واحد تولید دیگر، تابع `build_curriculum` است. این تابع برای هر مفهوم یک توضیح و یک سؤال می‌سازد. توضیح به شکل «`Explain concept in subject simply`» ساخته شده و سؤال به شکل «`What is concept in subject?`» تولید می‌شود. این بخش در نسخه فعلی ساده است و برای شبیه‌سازی کافی در نظر گرفته شده است. در نسخه‌های آینده می‌توان آن را گسترش داد تا توضیح‌های واقعی‌تر، تمرین‌های متنوع‌تر و پرسش‌های سطح‌بندی‌شده تولید کند.

واحد تولید تصویری نیز شامل دو تابع `draw_knowledge_graph` و `plot_learning_history` است. تابع اول گراف دانش را با رنگ آمیزی بر اساس `mastery` رسم می کند. تابع دوم روند تغییر `mastery` مفاهیم را در طول تکرارها نمایش می دهد. این دو خروجی به کاربر کمک می کنند نتیجه شبیه سازی را بهتر درک کنند.

در نهایت، تابع `build_simulation_report` نیز نوعی واحد تولید گزارش است. این تابع داده های ساخت یافته نتیجه را به متن قابل خواندن تبدیل می کند. گزارش شامل همه بخش های اصلی شبیه سازی است و می تواند برای بررسی دستی خروجی یا مستندسازی اجرای برنامه استفاده شود.

۳-۱-۲-۲- واحد ارزیابی و به روزرسانی

واحد ارزیابی و به روزرسانی یکی از بخش های مرکزی پروژه است. پس از آن که دانشجو به سؤال مربوط به یک مفهوم پاسخ داد، لازم است پاسخ او به یک مقدار عددی تبدیل شود تا بتوان وضعیت یادگیری را محاسبه کرد. این کار در تابع `evaluate` انجام می شود. ورودی این تابع، سؤال و پاسخ دانشجو است. تابع یک پرامپت می سازد و از مدل زبانی می خواهد فقط یک نمره از صفر تا یک تولید کند.

استفاده از بازه صفر تا یک باعث می شود محاسبات ساده و قابل مقایسه باشند. عدد صفر نشان دهنده پاسخ کاملاً نامناسب و عدد یک نشان دهنده پاسخ کاملاً مناسب است. مقادیر بین صفر و یک نیز کیفیت نسبی پاسخ را نشان می دهند. پس از دریافت پاسخ مدل، برنامه با عبارت منظم عدد معتبر را استخراج می کند. اگر استخراج با مشکل مواجه شود، مقدار صفر برگردانده می شود تا برنامه متوقف نشود.

در این کد، مقدار `mastery` با استفاده از میانگین گیری وزن دار بین تسلط قبلی دانشجو و نمره جدید پاسخ به روزرسانی می شود. اگر دانشجو برای اولین بار در یک مفهوم ارزیابی شود، همان مقدار `score` به عنوان مقدار اولیه `mastery` ثبت می شود. اما اگر قبلاً برای آن مفهوم شواهدی وجود داشته باشد، فرمول به روزرسانی به صورت زیر است:

$$mastery_{new} = \frac{((mastery_{old} \times weight_{old}) + (score \times weight_{new}))}{(weight_{old} + weight_{new})}$$

این فرمول نوعی میانگین گیری ساده است. مزیت آن این است که مقدار جدید هم به عملکرد گذشته و هم به پاسخ جدید وابسته است. اگر دانشجو قبلاً تسلط کمی داشته باشد اما پاسخ خوبی بدهد، مقدار تسلط افزایش پیدا می کند. اگر پاسخ ضعیف باشد، مقدار تسلط رشد کمتری خواهد داشت. این روش بسیار پیچیده نیست، اما برای نمایش روند تغییر تسلط در یک شبیه سازی آموزشی مناسب است.

در این پژوهش، مقدار تسلط یادگیرنده بر هر مفهوم با استفاده از میانگین گیری وزن دار میان تسلط پیشین و نمره پاسخ جدید به روزرسانی می شود. برای جلوگیری از نوسان های شدید ناشی از یک پاسخ منفرد، وزن بیشتری به مقدار تسلط قبلی اختصاص داده شده است. از سوی دیگر، برای آنکه مدل بتواند یادگیری جدید و تغییرات عملکردی دانشجو را نیز منعکس کند، نمره پاسخ

جدید با وزن مستقل وارد محاسبه می‌شود. در این پیاده‌سازی، وزن تسلط قبلی برابر با ۲ و وزن پاسخ جدید برابر با ۱ در نظر گرفته شده است. این انتخاب باعث می‌شود حدود دو سوم مقدار تسلط جدید از سابقه یادگیری و حدود یک سوم آن از عملکرد جدید دانشجو تأثیر بگیرد. بنابراین، مدل نه بیش از حد ناپایدار است و نه نسبت به پیشرفت یادگیرنده کند و غیرحساس عمل می‌کند.

پس از به‌روزرسانی mastery فردی، برنامه بازخوردی به حافظه دانشجو اضافه می‌کند. این بازخورد شامل score و پیام بهبود برای همان مفهوم است. اضافه شدن بازخورد به حافظه باعث می‌شود در پاسخ‌های بعدی، دانشجو به‌صورت متنی سابقه یادگیری و بازخورد را در پرامپت خود داشته باشد. در نتیجه، حافظه نقش ساده‌ای در شبیه‌سازی یادگیری ایفا می‌کند.

در پایان هر تکرار، گراف عمومی دانش به‌روزرسانی می‌شود. این کار در متد update از کلاس GeneralKG انجام می‌شود. برای هر مفهوم، mastery آن مفهوم در گراف سه دانشجو خوانده می‌شود و میانگین آن‌ها محاسبه می‌گردد. سپس این میانگین در گراف عمومی ذخیره می‌شود. بنابراین گراف عمومی نماینده وضعیت کلی گروه دانشجویان است.

تاریخچه یادگیری نیز در دیکشنری history ذخیره می‌شود. برای هر مفهوم، لیستی از مقادیر mastery در تکرارهای مختلف نگهداری می‌شود. این داده بعداً برای رسم نمودار روند یادگیری استفاده می‌شود. بنابراین واحد ارزیابی فقط نمره‌دهی لحظه‌ای انجام نمی‌دهد، بلکه داده لازم برای تحلیل روند یادگیری را نیز تولید می‌کند.

۳-۱-۳- گراف دانش و نمایش ساختار مفهومی

گراف دانش در این پروژه ابزاری برای نمایش رابطه میان مفاهیم آموزشی است. هر مفهوم به‌صورت یک گره در گراف ذخیره می‌شود. هر رابطه پیش‌نیازی نیز به‌صورت یک یال جهت‌دار بین دو گره ثبت می‌شود. استفاده از گراف جهت‌دار برای این پروژه مناسب است، زیرا روابط آموزشی معمولاً جهت دارند. برای مثال، اگر مفهوم A پیش‌نیاز مفهوم B باشد، رابطه از A به B تعریف می‌شود و عکس آن الزاماً درست نیست.

در کلاس GeneralKG، ابتدا یک گراف جهت‌دار با `nx.DiGraph()` ساخته می‌شود. سپس برای هر مفهوم، یک گره به گراف اضافه می‌شود و مقدار mastery آن صفر قرار داده می‌شود. سپس یال‌های وابستگی با `add_edges_from` به گراف اضافه می‌شوند. این کار ساختار اولیه دانش را ایجاد می‌کند.

دانشجویان نیز هرکدام گراف دانش فردی دارند. در کلاس Student، هنگام ساخت هر دانشجو، یک گراف جهت‌دار ساخته می‌شود و همه مفاهیم با mastery صفر به آن اضافه می‌شوند. در نسخه فعلی، یال‌های وابستگی در گراف فردی دانشجو اضافه نشده‌اند و تمرکز گراف فردی بر نگهداری mastery مفاهیم است. در مقابل، گراف عمومی هم مفاهیم و هم وابستگی‌ها را نگهداری می‌کند.

برای نمایش ساختار گراف، تابع `draw_knowledge_graph` طراحی شده است. این تابع ابتدا اندازه شکل را مشخص می‌کند، سپس موقعیت گره‌ها را با `nx.spring_layout` محاسبه می‌کند. برای هر گره، برجستگی ساخته می‌شود که شامل نام مفهوم و مقدار mastery آن است. مقدار mastery همچنین برای رنگ‌آمیزی گره‌ها استفاده می‌شود. هرچه مقدار mastery متفاوت باشد، رنگ گره نیز بر اساس نگاشت رنگی تغییر می‌کند.

یال‌ها با فلش رسم می‌شوند تا جهت رابطه مشخص باشد. برچسب‌ها با فونت کوچک‌تر و حالت ضخیم نمایش داده می‌شوند. در پایان، عنوان نمودار، حذف محور، تنظیم layout و ذخیره تصویر انجام می‌شود. تصویر خروجی در مسیر static/generated ذخیره می‌شود تا توسط صفحه وب قابل نمایش باشد.

علاوه بر گراف، ساختار دانش به شکل دیکشنری نیز تولید می‌شود. تابع build_knowledge_structure برای هر مفهوم، فرزندان آن را پیدا می‌کند؛ یعنی مفاهیمی که از آن مفهوم به آن‌ها یال وجود دارد. این ساختار در گزارش متنی نمایش داده می‌شود. نمایش دیکشنری باعث می‌شود کاربر علاوه بر تصویر گراف، بتواند وابستگی‌ها را به صورت متنی نیز بخواند.

گراف دانش مزیت مهمی برای تحلیل آموزشی دارد. اگر یک مفهوم mastery پایینی داشته باشد و در گراف پیش‌نیاز چند مفهوم دیگر باشد، می‌تواند یک گلوگاه مهم باشد. در نسخه فعلی، گلوگاه‌ها فقط بر اساس کمترین mastery انتخاب می‌شوند، اما در توسعه آینده می‌توان اهمیت گرافی هر مفهوم را نیز در نظر گرفت؛ برای مثال، مفهومی که mastery پایین و تعداد خروجی زیاد دارد، ممکن است اولویت بیشتری برای آموزش مجدد داشته باشد.

فصل ۴: طراحی و پیاده سازی

در این فصل، مدل سازی و طراحی سامانه ای که در این پروژه پیاده سازی شده است، به صورت کامل تشریح می شود. هدف اصلی این پروژه طراحی و شبیه سازی یک سامانه آموزشی مبتنی بر «گراف دانش» است که بتواند برای یک موضوع درسی مشخص، مفاهیم اصلی آن را استخراج کند، روابط پیش نیازی میان مفاهیم را تشکیل دهد، چند دانشجوی مجازی با سطوح یادگیری متفاوت را شبیه سازی کند، میزان یادگیری آن ها را در طول چند تکرار آموزشی ارزیابی نماید و در نهایت نتایج یادگیری را به صورت نمودارها، گراف ها و جدول های قابل تحلیل نمایش دهد.

در این سامانه، ورودی اصلی کاربر شامل سه بخش است: موضوع درسی، تعداد مفاهیم مورد نیاز و تعداد تکرارهای شبیه سازی. پس از دریافت این ورودی ها، سامانه با استفاده از یک مدل زبانی از طریق API، مفاهیم اصلی مربوط به موضوع را تولید می کند و روابط وابستگی یا پیش نیازی بین آن مفاهیم را به دست می آورد. این روابط به صورت یال های جهت دار در یک گراف دانش نمایش داده می شوند. در این گراف، هر گره نشان دهنده یک مفهوم آموزشی و هر یال نشان دهنده رابطه پیش نیازی یا وابستگی میان دو مفهوم است. برای مثال اگر مفهومی پیش نیاز مفهوم دیگر باشد، در گراف یک یال جهت دار از مفهوم اول به مفهوم دوم ایجاد می شود.

در طراحی این پروژه، برای نمایش ساختار مفاهیم و روابط بین آن ها از کتابخانه NetworkX استفاده شده است. این کتابخانه امکان ساخت گراف های جهت دار، افزودن گره ها و یال ها و نگهداری ویژگی هایی مانند میزان تسلط یا mastery را فراهم می کند. در کنار آن، کتابخانه Matplotlib برای ترسیم تصویری گراف ها و نمودارهای تحلیلی مورد استفاده قرار گرفته است. از آن جایی که پروژه در محیط وب و از طریق Flask اجرا می شود، حالت Agg برای Matplotlib فعال شده است تا رسم نمودارها بدون نیاز به محیط گرافیکی مستقیم سیستم انجام شود. این موضوع برای اجرای برنامه در سمت سرور اهمیت دارد.

در این پروژه، فرآیند یادگیری فقط به یک دانشجوی محدود نشده است، بلکه سه نوع دانشجوی شبیه سازی شده در نظر گرفته شده اند: دانشجوی ضعیف، دانشجوی متوسط و دانشجوی قوی. هر یک از این دانشجویان دارای سبک پاسخ دهی متفاوت هستند. این تفاوت از طریق تابع get_style در کلاس Student تعریف شده است. دانشجوی ضعیف معمولاً پاسخ های کوتاه، مبهم، ناقص یا اشتباه می دهد. دانشجوی متوسط گاهی پاسخ درست و گاهی پاسخ ناقص تولید می کند. دانشجوی قوی پاسخ هایی واضح تر و صحیح تر ارائه می دهد. این تفاوت رفتاری با استفاده از پیام سیستمی و مقدار دمای مدل زبانی کنترل می شود. بنابراین در این پروژه، سطح دانشجو فقط یک برچسب ساده نیست، بلکه در نحوه تولید پاسخ توسط مدل اثر می گذارد.

فرآیند شبیه سازی در چند مرحله انجام می شود. ابتدا مفاهیم و وابستگی های آموزشی تولید می شوند. سپس برای هر دانشجو یک گراف دانش جداگانه ساخته می شود. در هر گراف، گره ها همان مفاهیم تولید شده هستند و مقدار اولیه تسلط هر مفهوم برابر صفر قرار داده می شود. پس از آن، در هر تکرار آموزشی، دانشجویان با توضیح مربوط به هر مفهوم مواجه می شوند. این توضیح توسط تابع build_curriculum ساخته می شود و شامل معرفی کلی مفهوم، نقش آن، مثال ها و ارتباط آن با سایر مفاهیم است. سپس برای هر مفهوم، یک سؤال از دانشجو پرسیده می شود. پاسخ دانشجو توسط مدل زبانی تولید شده و سپس دوباره با استفاده از مدل زبانی ارزیابی می شود. نتیجه ارزیابی عددی بین صفر تا یک است. این عدد به عنوان نمره پاسخ دانشجو در نظر گرفته می شود.

یکی از بخش های مهم طراحی این سامانه، نحوه به روز رسانی میزان تسلط است. در این پروژه، تسلط هر دانشجو بر یک مفهوم با ترکیب مقدار قبلی تسلط و نمره جدید محاسبه می شود. به عبارت دیگر، مقدار جدید تسلط برابر میانگین مقدار قبلی و نمره

فعلی است. این روش باعث می‌شود که یادگیری به صورت تدریجی مدل شود و یک پاسخ منفرد به تنهایی کل وضعیت یادگیری دانشجو را تعیین نکند. سپس یک گراف دانش عمومی نیز ساخته می‌شود که میانگین میزان تسلط دانشجویان را برای هر مفهوم نشان می‌دهد. این گراف عمومی نماینده وضعیت کلی کلاس یا گروه یادگیرندگان است.

در کنار مدل سازی یادگیری، طراحی سامانه شامل بخش مدیریت اجرای غیرهمزمان نیز هست. چون ارتباط با مدل زبانی، تولید پاسخ‌ها، ارزیابی پاسخ‌ها و رسم نمودارها ممکن است زمان‌بر باشد، در این پروژه از `threading` استفاده شده است. زمانی که کاربر شبیه سازی را آغاز می‌کند، یک شناسه یکتا با استفاده از `uuid` برای آن اجرا ساخته می‌شود و عملیات اصلی در یک نخ پس زمینه انجام می‌گیرد. وضعیت پیشرفت کار در دیکشنری `jobs` ذخیره می‌شود و با استفاده از `jobs_lock` از تداخل همزمانی جلوگیری می‌شود. در سمت کاربر، صفحه `HTML` به صورت دوره‌ای وضعیت شبیه سازی را از مسیر `<job_id>/status/` دریافت می‌کند و نوار پیشرفت را به روزرسانی می‌نماید. این طراحی باعث می‌شود کاربر در طول اجرای شبیه سازی از وضعیت برنامه مطلع باشد و احساس توقف یا عدم پاسخ گویی سامانه نداشته باشد.

بنابراین فصل سوم به بررسی منطق مدل سازی و شبیه سازی این سامانه می‌پردازد. در این فصل، روش پیشنهادی، ابزارهای استفاده شده، معیارهای ارزیابی، نوع خروجی‌ها و شیوه تحلیل نتایج توضیح داده می‌شود. تمرکز این فصل بر این است که نشان دهد سامانه چگونه از یک ورودی ساده مانند نام یک موضوع درسی، یک ساختار آموزشی مبتنی بر گراف دانش می‌سازد، یادگیری چند دانشجوی فرضی را شبیه سازی می‌کند و خروجی‌هایی قابل استفاده برای تحلیل آموزشی تولید می‌نماید.

۴-۱ روش پیشنهادی

روش پیشنهادی این پروژه بر پایه ترکیب سه بخش اصلی طراحی شده است: تولید محتوای آموزشی و مفاهیم با استفاده از مدل زبانی، نمایش ساختار دانش با استفاده از گراف جهت دار، و شبیه سازی یادگیری دانشجویان با سطوح مختلف توانایی. این سه بخش در کنار یکدیگر، سامانه‌ای را ایجاد می‌کنند که می‌تواند از یک موضوع درسی، یک مدل آموزشی قابل مشاهده و قابل تحلیل بسازد.

در گام نخست، کاربر از طریق رابط کاربری وب، موضوع آموزشی موردنظر خود را وارد می‌کند. علاوه بر موضوع، تعداد مفاهیم اصلی و تعداد تکرارهای شبیه سازی نیز توسط کاربر تعیین می‌شود. در کد، مقدار تعداد مفاهیم با تابع `safe_int` کنترل می‌شود تا کمتر از ۲ و بیشتر از ۱۵ نباشد. تعداد تکرارها نیز بین ۱ تا ۱۰ محدود شده است. این محدودیت‌ها باعث می‌شوند برنامه از نظر زمانی و محاسباتی قابل کنترل باقی بماند و ورودی‌های نامعتبر باعث اختلال در اجرای سامانه نشوند. اگر کاربر موضوع را وارد نکند، مسیر `/start` یک پاسخ خطا به صورت `JSON` بازمی‌گرداند و اجرای شبیه سازی شروع نمی‌شود.

پس از دریافت ورودی، تابع `generate_curriculum` فراخوانی می‌شود. این تابع با استفاده از مدل زبانی، مفاهیم اصلی موضوع را تولید می‌کند. در متن دستور یا `prompt`، از مدل خواسته شده است که دقیقاً به تعداد مشخص شده، مفاهیم اصلی یادگیری را استخراج کند و روابط پیش‌نیازی میان آن‌ها را تعریف نماید. خروجی مورد انتظار مدل باید فقط `JSON` معتبر باشد و توضیح اضافی یا `Markdown` نداشته باشد. ساختار خروجی شامل دو بخش است: `concepts` برای فهرست مفاهیم و `dependencies` برای روابط وابستگی میان مفاهیم. هر وابستگی به صورت یک آرایه دوتایی نمایش داده می‌شود که عنصر اول پیش‌نیاز و عنصر دوم مفهوم مقصد است.

از آنجایی که خروجی مدل زبانی ممکن است همیشه دقیقاً مطابق قالب خواسته شده نباشد، در این پروژه تابع `extract_json` برای استخراج `JSON` معتبر طراحی شده است. این تابع ابتدا متن خروجی را پاک سازی می‌کند و اگر مدل پاسخ را داخل بلوک‌هایی مانند `json` قرار داده باشد، آن‌ها را حذف می‌کند. سپس تلاش می‌کند متن باقی مانده را به عنوان `JSON` بخواند.

اگر این کار موفق نباشد، با استفاده از عبارت منظم، اولین ساختار شبیه شیء JSON را از متن استخراج می‌کند. این بخش برای افزایش پایداری برنامه در برابر پاسخ‌های غیرکاملاً منظم مدل اهمیت دارد.

بعد از استخراج مفاهیم، برنامه اطمینان حاصل می‌کند که تعداد مفاهیم دقیقاً مطابق مقدار درخواست شده باشد. اگر مدل مفاهیم بیشتری تولید کرده باشد، فهرست مفاهیم به تعداد موردنظر کوتاه می‌شود. اگر تعداد مفاهیم کمتر از مقدار موردنیاز باشد، برنامه مفاهیم جایگزین با نام‌هایی مانند Concept 1، Concept 2 و مانند آن اضافه می‌کند. سپس روابط وابستگی نیز پاک‌سازی می‌شوند. فقط روابطی پذیرفته می‌شوند که به صورت یک لیست دوتایی باشند، هر دو سر رابطه در فهرست مفاهیم موجود باشند و رابطه از یک مفهوم به خودش نباشد. این مرحله باعث می‌شود گراف نهایی فقط شامل گره‌ها و یال‌های معتبر باشد و روابط خارج از فهرست مفاهیم وارد ساختار گراف نشوند.

پس از تولید مفاهیم و وابستگی‌ها، چند دانشجوی شبیه‌سازی شده ساخته می‌شوند. این کار در تابع `run_simulation` انجام می‌شود. از کلاس `Student` با نام‌های `S1`، `S2`، `S3`، ...، `Sn` شیء^۵ ساخته می‌شوند. سطح این دانشجویان می‌تواند ضعیف، متوسط یا قوی باشد. هر دانشجو دارای یک حافظه متنی و یک گراف دانش اختصاصی است. در سازنده کلاس `Student`، ابتدا برای هر مفهوم یک گره در گراف ایجاد می‌شود و ویژگی `mastery` آن برابر صفر قرار می‌گیرد. سپس روابط وابستگی تولیدشده به عنوان یال‌های جهت‌دار به گراف دانش دانشجو اضافه می‌شوند. بنابراین هر دانشجو از نظر ساختار مفاهیم، گرافی مشابه دارد، اما مقدار تسلط او روی هر مفهوم در طول شبیه‌سازی متفاوت خواهد شد.

در کنار گراف‌های اختصاصی دانشجویان، یک گراف عمومی نیز با استفاده از کلاس `GeneralKG` ساخته می‌شود. این گراف نمایانگر وضعیت کلی گروه دانشجویان است. در ابتدا، این گراف نیز شامل همان مفاهیم و روابط وابستگی است و مقدار تسلط همه مفاهیم در آن صفر است. اما بعد از هر تکرار آموزشی، تابع `update` در کلاس `GeneralKG` اجرا می‌شود و مقدار تسلط هر مفهوم در گراف عمومی بر اساس میانگین تسلط همان مفهوم در گراف دانشجویان محاسبه می‌گردد. در نتیجه، گراف عمومی تصویری میانگین از وضعیت یادگیری کل گروه ارائه می‌دهد.

در مرحله بعد، محتوای آموزشی اولیه برای مفاهیم ساخته می‌شود. این کار توسط تابع `build_curriculum` انجام می‌شود. این تابع برای هر مفهوم، یک توضیح و یک سؤال ایجاد می‌کند. توضیح هر مفهوم به این صورت است که مفهوم موردنظر متعلق به موضوع واردشده است و باید تعریف، نقش، مثال‌ها و ارتباط آن با سایر مفاهیم مطالعه شود. سؤال نیز با قالب «`What is concept in subject`؟» ساخته می‌شود. به این ترتیب برای هر مفهوم یک واحد آموزشی ساده شامل توضیح و پرسش وجود دارد. این بخش در واقع محتوای پایه‌ای است که در هر تکرار به دانشجویان داده می‌شود.

فرآیند شبیه‌سازی درون حلقه‌ای به تعداد تکرارها^۶ انجام می‌شود. در هر تکرار، ابتدا همه دانشجویان برای همه مفاهیم، توضیح مربوط به آن مفهوم را دریافت می‌کنند. متد `learn` در کلاس `Student` این توضیح را به حافظه متنی دانشجو اضافه می‌کند. حافظه دانشجو به صورت یک رشته متنی ذخیره می‌شود و شامل مفاهیم و توضیحاتی است که دانشجو تاکنون دریافت کرده است. سپس برای هر مفهوم، سؤال مربوط به همان مفهوم از هر سه دانشجو پرسیده می‌شود. پاسخ هر دانشجو با متد `answer` تولید می‌شود. در این متد، ابتدا سبک پاسخ‌دهی دانشجو با تابع `get_style` مشخص می‌شود و سپس مدل زبانی با دریافت حافظه دانشجو و سؤال، پاسخ یک‌خطی تولید می‌کند.

تفاوت دانشجویان در همین بخش به صورت مستقیم اعمال می‌شود. برای دانشجوی ضعیف، پیام سیستمی به مدل می‌گوید که نقش یک دانشجوی بسیار ضعیف را داشته باشد و پاسخ‌هایی کوتاه، مبهم، ناقص یا اشتباه بدهد. همچنین دمای مدل برای او ۰٫۸ تنظیم شده است. برای دانشجوی متوسط، پیام سیستمی بیان می‌کند که پاسخ‌ها گاهی درست و گاهی ناقص هستند و

^۵ object
^۶ iterations

دما ۰.۶ است. برای دانشجوی قوی، از مدل خواسته می‌شود پاسخ‌هایی درست و روشن ارائه دهد و دمای مدل ۰.۳ است. بنابراین هر سطح دانشجو با ترکیبی از دستور نقش و دمای تولید پاسخ مدل‌سازی شده است.

بعد از دریافت پاسخ دانشجو، تابع evaluate اجرا می‌شود. این تابع سؤال و پاسخ دانشجو را به مدل زبانی می‌دهد و از آن می‌خواهد فقط یک عدد بین صفر تا یک برگرداند. این عدد نشان‌دهنده کیفیت پاسخ دانشجو است. اگر پاسخ کاملاً درست باشد، انتظار می‌رود نمره به یک نزدیک‌تر باشد و اگر پاسخ نادرست یا ناقص باشد، نمره کمتر شود. در کد، پس از دریافت متن خروجی مدل، با استفاده از عبارت منظم اولین عدد معتبر بین صفر و یک استخراج می‌شود. اگر استخراج عدد با خطا مواجه شود، تابع مقدار صفر را برمی‌گرداند. این طراحی باعث می‌شود سامانه حتی در صورت پاسخ نامنظم مدل ارزیاب نیز دچار توقف کامل نشود.

در این کد، مقدار mastery با استفاده از میانگین‌گیری وزن‌دار بین تسلط قبلی دانشجو و نمره جدید پاسخ به‌روزرسانی می‌شود. اگر دانشجو برای اولین بار در یک مفهوم ارزیابی شود، همان مقدار score به‌عنوان مقدار اولیه mastery ثبت می‌شود. اما اگر قبلاً برای آن مفهوم شواهدی وجود داشته باشد، فرمول به‌روزرسانی به‌صورت زیر است:

$$\text{mastery}_{\text{new}} = \frac{((\text{mastery}_{\text{old}} \times \text{weight}_{\text{old}}) + (\text{score} \times \text{weight}_{\text{new}}))}{(\text{weight}_{\text{old}} + \text{weight}_{\text{new}})}$$

این فرمول نوعی میانگین‌گیری ساده است. مزیت آن این است که مقدار جدید هم به عملکرد گذشته و هم به پاسخ جدید وابسته است. اگر دانشجو قبلاً تسلط کمی داشته باشد اما پاسخ خوبی بدهد، مقدار تسلط افزایش پیدا می‌کند. اگر پاسخ ضعیف باشد، مقدار تسلط رشد کمتری خواهد داشت. این روش بسیار پیچیده نیست، اما برای نمایش روند تغییر تسلط در یک شبیه‌سازی آموزشی مناسب است.

در این پژوهش، مقدار تسلط یادگیرنده بر هر مفهوم با استفاده از میانگین‌گیری وزن‌دار میان تسلط پیشین و نمره پاسخ جدید به‌روزرسانی می‌شود. برای جلوگیری از نوسان‌های شدید ناشی از یک پاسخ منفرد، وزن بیشتری به مقدار تسلط قبلی اختصاص داده شده است. از سوی دیگر، برای آنکه مدل بتواند یادگیری جدید و تغییرات عملکردی دانشجو را نیز منعکس کند، نمره پاسخ جدید با وزن مستقل وارد محاسبه می‌شود. در این پیاده‌سازی، وزن تسلط قبلی برابر با ۲ و وزن پاسخ جدید برابر با ۱ در نظر گرفته شده است. این انتخاب باعث می‌شود حدود دو سوم مقدار تسلط جدید از سابقه یادگیری و حدود یک سوم آن از عملکرد جدید دانشجو تأثیر بگیرد. بنابراین، مدل نه بیش از حد ناپایدار است و نه نسبت به پیشرفت یادگیرنده کند و غیرحساس عمل می‌کند.

این روش باعث می‌شود که مقدار تسلط به‌صورت تدریجی تغییر کند. اگر دانشجو در یک تکرار نمره بالایی دریافت کند، میزان تسلط او افزایش می‌یابد؛ اما این افزایش کاملاً ناگهانی نیست، چون با مقدار قبلی میانگین گرفته می‌شود. همچنین اگر در یک تکرار پاسخ ضعیفی بدهد، مقدار تسلط کاهش یا رشد کمتری خواهد داشت. این شیوه ساده است، اما برای نمایش روند یادگیری و مقایسه بین دانشجویان قابل استفاده است.

تمام پاسخ‌ها و نمره‌ها در فهرست `answer_logs` ذخیره می‌شوند. برای هر پاسخ، شماره تکرار، نام دانشجو، سطح دانشجو، مفهوم، سؤال، پاسخ و نمره ذخیره می‌شود. این داده‌ها در خروجی نهایی در جدول `Student Answers` نمایش داده می‌شوند و امکان بررسی جزئی رفتار هر دانشجو را فراهم می‌کنند. به عنوان مثال، از این جدول می‌توان دید که دانشجوی ضعیف در هر مفهوم چه پاسخی داده، دانشجوی قوی چه پاسخی داده و نمره هر پاسخ چقدر بوده است. این جدول یکی از مهم‌ترین خروجی‌های تحلیلی سامانه است، زیرا فقط نتیجه نهایی را نشان نمی‌دهد، بلکه مسیر پاسخ‌دهی و ارزیابی را نیز ثبت می‌کند.

بعد از پایان ارزیابی همه دانشجویان در هر تکرار، گراف عمومی با استفاده از وضعیت جدید دانشجویان به‌روزرسانی می‌شود. سپس مقدار تسلط عمومی هر مفهوم در دیکشنری `history` ذخیره می‌شود. این دیکشنری برای هر مفهوم، فهرستی از مقادیر تسلط در تکرارهای مختلف نگهداری می‌کند. بعداً از همین داده برای رسم نمودار روند یادگیری استفاده می‌شود. همچنین در پایان هر تکرار، تصویر گراف دانش عمومی ذخیره می‌شود و مسیر آن در فهرست `iteration_graphs` قرار می‌گیرد. به این ترتیب، کاربر می‌تواند تغییرات گراف دانش را در طول تکرارهای مختلف مشاهده کند.

در پایان کل شبیه‌سازی، چند خروجی تصویری و تحلیلی تولید می‌شود. ابتدا گراف دانش نهایی عمومی رسم می‌شود. سپس نمودار روند یادگیری با استفاده از داده‌های `history` تولید می‌گردد. همچنین گراف دانش مربوط به دانشجوی ضعیف به صورت جداگانه رسم می‌شود. علت نمایش گراف دانشجوی ضعیف این است که وضعیت یادگیری او معمولاً نسبت به سایر دانشجویان چالش‌برانگیزتر است و تحلیل آن می‌تواند برای معلم اهمیت بیشتری داشته باشد. علاوه بر این، تابع `generate_teacher_analysis` اجرا می‌شود و درصد یادگیری هر مفهوم و نمودارهای ستونی و دایره‌ای را تولید می‌کند. در این تابع، آستانه یادگیری برابر ۰.۶ در نظر گرفته شده است. هر پاسخی که نمره آن حداقل ۰.۶ باشد، به عنوان حالت «یادگرفته‌شده» محسوب می‌شود.

بنابراین روش پیشنهادی این پروژه، یک چرخه کامل از تولید مفاهیم، ساخت گراف دانش، آموزش، پرسش، پاسخ، ارزیابی، به‌روزرسانی تسلط، تولید نمودار و نمایش نتیجه را پیاده‌سازی می‌کند. این چرخه با استفاده از مدل زبانی و ابزارهای گرافانی انجام شده و نتیجه آن به صورت یک سامانه تعاملی تحت وب ارائه می‌شود. ویژگی مهم این روش آن است که فقط به نمایش مفاهیم بسنده نمی‌کند، بلکه وضعیت یادگیری دانشجویان و تغییرات آن در طول زمان را نیز مدل‌سازی می‌کند.

۴-۲ ابزارهای مورد نیاز

در این تحقیق از ابزارهای زیر استفاده شده است:

- پایتون
- Qwen 3.5 API
- NetworkX برای گراف دانش
- Matplotlib برای رسم کردن شکل‌ها و نمودارها

در این پروژه برای طراحی و پیاده‌سازی سامانه شبیه‌ساز گراف دانش آموزشی، از مجموعه‌ای از ابزارها، کتابخانه‌ها و فناوری‌های نرم‌افزاری استفاده شده است. هر یک از این ابزارها نقش مشخصی در فرآیند کلی سامانه دارند و در کنار یکدیگر باعث می‌شوند برنامه بتواند ورودی کاربر را دریافت کند، با مدل زبانی ارتباط برقرار نماید، گراف دانش بسازد، شبیه‌سازی یادگیری را اجرا کند، خروجی‌های تصویری تولید نماید و آن‌ها را در یک رابط کاربری وب نمایش دهد.

اولین ابزار اصلی در این پروژه زبان برنامه‌نویسی Python است. بخش اصلی منطق برنامه در فایل `app.py` نوشته شده است. Python به دلیل سادگی، وجود کتابخانه‌های قدرتمند برای پردازش داده، هوش مصنوعی، رسم نمودار و توسعه وب، گزینه

مناسبی برای این پروژه محسوب می‌شود. در این کد، Python برای مدیریت مسیرهای وب، ارتباط با API مدل زبانی، ساختاردهی داده‌ها، مدیریت نخ‌های اجرایی، پردازش JSON، ساخت گراف، محاسبه میانگین‌ها و ذخیره خروجی‌های تصویری استفاده شده است.

برای ایجاد برنامه وب، از چارچوب Flask استفاده شده است. Flask یک فریم‌ورک سبک برای توسعه برنامه‌های تحت وب در Python است. در این پروژه، شیء اصلی برنامه با دستور `app` ساخته شده و سپس چند مسیر یا `route` برای مدیریت درخواست‌های کاربر تعریف شده است. مسیر / صفحه اصلی را نمایش می‌دهد. مسیر `/start` شبیه‌سازی را آغاز می‌کند. مسیر `/status/<job_id>` وضعیت اجرای شبیه‌سازی را برمی‌گرداند و مسیر `/result/<job_id>` نتیجه نهایی را به کاربر نمایش می‌دهد. استفاده از Flask باعث شده است که منطق شبیه‌سازی در سمت سرور اجرا شود و کاربر فقط از طریق فرم وب با آن تعامل داشته باشد.

برای ساخت و مدیریت گراف دانش، از کتابخانه NetworkX استفاده شده است. این کتابخانه امکان ایجاد گراف‌های جهت‌دار، افزودن گره‌ها، افزودن یال‌ها و نگهداری ویژگی‌های هر گره را فراهم می‌کند. در این پروژه، از `nx.DiGraph()` برای ساخت گراف جهت‌دار استفاده شده است. جهت‌دار بودن گراف اهمیت زیادی دارد، زیرا روابط آموزشی معمولاً دارای جهت هستند. اگر مفهوم A پیش‌نیاز مفهوم B باشد، این رابطه با یک یال از A به B نمایش داده می‌شود. همچنین مقدار تسلط هر دانشجو بر هر مفهوم به‌عنوان ویژگی `mastery` در گره مربوط به آن مفهوم ذخیره می‌شود.

برای رسم گراف‌ها و نمودارها، از کتابخانه Matplotlib استفاده شده است. این کتابخانه در چند بخش پروژه نقش دارد. تابع `draw_knowledge_graph` از Matplotlib و NetworkX برای رسم گراف دانش استفاده می‌کند. در این تصویر، گره‌ها با رنگی متناسب با مقدار تسلط نمایش داده می‌شوند و برچسب هر گره شامل نام مفهوم و مقدار تسلط آن است. تابع `plot_learning_history` روند تغییر میزان تسلط مفاهیم را در طول تکرارها ترسیم می‌کند. تابع `generate_teacher_analysis` نیز نمودار ستونی درصد یادگیری هر مفهوم و نمودار دایره‌ای توزیع کلی یادگیری را تولید می‌کند. در ابتدای کد، دستور `matplotlib.use("Agg")` قرار داده شده است تا Matplotlib بتواند تصاویر را در محیط سرور و بدون نیاز به نمایشگر گرافیکی ذخیره کند.

برای محاسبات عددی، از کتابخانه NumPy استفاده شده است. نقش اصلی NumPy در این پروژه محاسبه میانگین مقادیر تسلط دانشجویان است. در کلاس `GeneralKG`، تابع `update` برای هر مفهوم، نمرات تسلط آن مفهوم را از گراف دانشجویان جمع‌آوری می‌کند و با استفاده از `np.mean` میانگین آن‌ها را محاسبه می‌نماید. این مقدار میانگین سپس به‌عنوان میزان تسلط عمومی آن مفهوم در گراف کلی ذخیره می‌شود.

یکی دیگر از ابزارهای اصلی پروژه، API مدل زبانی است که از طریق کتابخانه OpenAI فراخوانی می‌شود. در تابع `get_client`، یک کلاینت OpenAI با کلید API و آدرس پایه سرویس ساخته می‌شود. مدل مورد استفاده در متغیر `MODEL` مشخص شده و مقدار پیش‌فرض آن `gpt-3.5-turbo` است. در این پروژه، مدل زبانی سه نقش اصلی دارد: تولید مفاهیم و وابستگی‌ها، تولید پاسخ دانشجویان شبیه‌سازی‌شده و ارزیابی پاسخ دانشجویان. بنابراین مدل زبانی فقط برای تولید متن استفاده نشده، بلکه در فرآیند آموزشی و ارزیابی نیز نقش دارد.

برای مدیریت متغیرهای محیطی، از کتابخانه `dotenv` و تابع `load_dotenv` استفاده شده است. این تابع امکان خواندن متغیرهای ذخیره‌شده در فایل `.env` را فراهم می‌کند. در کد، مقدار مدل از متغیر محیطی `GAPGPT_MODEL` و آدرس پایه API از متغیر محیطی `GAPGPT_BASE_URL` خوانده می‌شود. در نسخه فعلی کد، کلید API به‌صورت مستقیم داخل تابع `get_client` قرار داده شده است. از نظر فنی، این روش باعث می‌شود برنامه بتواند به سرویس مدل زبانی متصل

شود، اما از نظر امنیتی، بهتر است در نسخه نهایی پروژه، کلید API فقط در فایل env ذخیره شود و در متن کد یا گزارش منتشر نشود.

برای تولید شناسه یکتا برای هر اجرای شبیه‌سازی، از کتابخانه uuid استفاده شده است. زمانی که کاربر شبیه‌سازی را آغاز می‌کند، یک job_id جدید با uuid.uuid4() ساخته می‌شود. این شناسه برای ردیابی وضعیت اجرای همان شبیه‌سازی استفاده می‌شود. به دلیل این طراحی، اگر چند شبیه‌سازی مختلف در زمان‌های متفاوت اجرا شوند، هر کدام شناسه جداگانه دارند و نتیجه آن‌ها با یکدیگر اشتباه نمی‌شود.

برای اجرای شبیه‌سازی در پس‌زمینه، از کتابخانه threading استفاده شده است. مسیر start/ بعد از دریافت ورودی، یک نخ جدید ایجاد می‌کند و تابع background_worker را در آن اجرا می‌کند. این کار باعث می‌شود درخواست اولیه کاربر سریع پاسخ بگیرد و صفحه وب بتواند با استفاده از شناسه شبیه‌سازی، وضعیت پیشرفت را بررسی کند. اگر شبیه‌سازی در همان درخواست اصلی اجرا می‌شد، کاربر تا پایان عملیات هیچ بازخوردی دریافت نمی‌کرد و احتمالاً صفحه وب ظاهراً متوقف به نظر می‌رسید. استفاده از نخ پس‌زمینه همراه با نوار پیشرفت، تجربه کاربری بهتری ایجاد کرده است.

برای جلوگیری از تداخل در دسترسی به داده‌های مشترک، از jobs_lock استفاده شده است. دیکشنری jobs اطلاعات همه اجراهای فعال یا تمام‌شده را نگهداری می‌کند. از آنجایی که نخ پس‌زمینه و درخواست‌های وضعیت ممکن است هم‌زمان به این دیکشنری دسترسی داشته باشند، استفاده از قفل باعث می‌شود تغییرات به شکل امن‌تری انجام شوند. توابع set_job_error.update_job_progress و set_job_done همگی هنگام تغییر وضعیت یک شبیه‌سازی از این قفل استفاده می‌کنند.

در سمت رابط کاربری، فایل index.html استفاده شده است. این فایل شامل ساختار HTML، کدهای CSS برای ظاهر صفحه و کدهای JavaScript برای ارسال فرم و دریافت وضعیت پیشرفت است. فرم صفحه شامل سه ورودی است: موضوع، تعداد مفاهیم و تعداد تکرارها. بعد از ارسال فرم، JavaScript با استفاده از fetch درخواست POST به مسیر start/ ارسال می‌کند. سپس اگر شبیه‌سازی شروع شود، تابع checkStatus هر یک ثانیه یک‌بار مسیر <job_id>/status/ را فراخوانی می‌کند و درصد پیشرفت، پیام وضعیت و نوار پیشرفت را به‌روزرسانی می‌کند. وقتی وضعیت به done برسد، مرورگر به مسیر <job_id>/result/ منتقل می‌شود و خروجی‌ها نمایش داده می‌شوند.

بنابراین ابزارهای استفاده‌شده در این پروژه به‌صورت هماهنگ در چند لایه عمل می‌کنند. Python و Flask لایه اصلی اجرا و وب را تشکیل می‌دهند. OpenAI API نقش تولید و ارزیابی محتوای هوشمند را دارد. NetworkX و Matplotlib لایه گراف و تصویرسازی را تشکیل می‌دهند. NumPy برای محاسبات عددی استفاده شده است. Threading و UUID برای مدیریت اجرای غیرهمزمان و شناسایی هر شبیه‌سازی به کار رفته‌اند. CSS، HTML و JavaScript نیز رابط کاربری، فرم ورودی، نوار پیشرفت و نمایش نتایج را فراهم کرده‌اند.

۳-۴ معیار ارزیابی

در این پروژه، معیار ارزیابی یادگیری بر پایه نمره‌ای عددی بین صفر تا یک طراحی شده است. این نمره توسط تابع evaluate محاسبه می‌شود. در این تابع، سؤال مربوط به یک مفهوم و پاسخ تولیدشده توسط دانشجو به مدل زبانی داده می‌شود و از مدل خواسته می‌شود فقط یک عدد از صفر تا یک برگرداند. عدد صفر نشان‌دهنده پاسخ کاملاً ضعیف یا نادرست و عدد یک نشان‌دهنده

پاسخ کاملاً صحیح و مناسب است. مقادیر بین صفر و یک نیز درجات مختلفی از ناقص بودن، نیمه درست بودن یا کامل بودن پاسخ را نشان می‌دهند.

فرآیند ارزیابی در کد به این شکل انجام می‌شود که ابتدا یک prompt شامل دو بخش ساخته می‌شود: بخش اول سؤال و بخش دوم پاسخ دانشجو. سپس از مدل خواسته می‌شود: «Give a score from 0 to 1 only Return only one number». این دستور باعث می‌شود خروجی ارزیاب تا حد امکان فقط شامل یک عدد باشد. پس از دریافت پاسخ مدل، متن خروجی با استفاده از عبارت منظم بررسی می‌شود و اولین عدد معتبر در بازه صفر تا یک استخراج می‌گردد. اگر استخراج عدد موفق باشد، همان عدد به صورت float بازگردانده می‌شود. در غیر این صورت، مقدار پیش فرض صفر برگردانده می‌شود. این مکانیزم باعث می‌شود که اگر مدل خروجی نامناسب یا غیر عددی تولید کند، برنامه از کار نیفتد و ارزیابی با مقدار محافظه کارانه صفر ادامه پیدا کند.

این نمره عددی چند کاربرد اصلی در سامانه دارد. کاربرد اول آن به روزرسانی میزان تسلط دانشجو بر مفهوم است. هر مفهوم در گراف دانش هر دانشجو دارای ویژگی mastery است. مقدار اولیه این ویژگی صفر است. پس از هر پاسخ و ارزیابی، مقدار جدید تسلط با استفاده از میانگین مقدار قبلی و نمره جدید محاسبه می‌شود. بنابراین اگر دانشجو برای یک مفهوم پاسخ خوبی بدهد، مقدار تسلط او روی آن مفهوم افزایش می‌یابد. اگر پاسخ ضعیف باشد، مقدار تسلط رشد کمی خواهد داشت یا در سطح پایینی باقی می‌ماند.

کاربرد دوم نمره ارزیابی، محاسبه وضعیت کلی گراف دانش عمومی است. پس از پایان ارزیابی دانشجویان در هر تکرار، گراف عمومی با میانگین گرفتن از مقادیر تسلط دانشجویان به روزرسانی می‌شود. در این حالت، برای هر مفهوم، مقادیر تسلط آن مفهوم در گراف دانشجوی ضعیف، متوسط و قوی خوانده می‌شود و میانگین آن‌ها به عنوان میزان تسلط کلی آن مفهوم در کلاس ذخیره می‌شود. این معیار امکان مشاهده وضعیت میانگین یادگیری را فراهم می‌کند. به این ترتیب، گراف عمومی فقط وضعیت یک دانشجو را نمایش نمی‌دهد، بلکه نماینده وضعیت کلی سه دانشجوی شبیه سازی شده است.

کاربرد سوم معیار ارزیابی، تولید تحلیل معلم است. در تابع generate_teacher_analysis، یک آستانه یادگیری تعریف شده است. مقدار این آستانه برابر ۰.۶ است. بر اساس این معیار، اگر نمره یک پاسخ برابر یا بیشتر از ۰.۶ باشد، آن پاسخ به عنوان یادگرفته شده یا Learned در نظر گرفته می‌شود. اگر نمره کمتر از ۰.۶ باشد، پاسخ در دسته Not Learned قرار می‌گیرد. سپس برای هر مفهوم، تعداد پاسخ‌هایی که نمره آن‌ها از این آستانه عبور کرده است شمارش می‌شود و نسبت آن به کل پاسخ‌های مربوط به همان مفهوم به درصد تبدیل می‌شود. این درصد در دیکشنری concept_mastery_percent ذخیره می‌شود و در جدول تحلیل معلم نمایش داده می‌شود.

این معیار درصدی برای معلم اهمیت زیادی دارد، زیرا فقط میانگین عددی تسلط را نشان نمی‌دهد، بلکه مشخص می‌کند چه درصدی از پاسخ‌های مربوط به یک مفهوم به سطح قابل قبول یادگیری رسیده‌اند. برای مثال، اگر در یک مفهوم بیشتر پاسخ‌ها نمره‌ای بالاتر از ۰.۶ داشته باشند، می‌توان گفت آن مفهوم در میان دانشجویان بهتر یاد گرفته شده است. اگر درصد یادگیری یک مفهوم پایین باشد، نشان می‌دهد آن مفهوم نیاز به آموزش بیشتر، توضیح بهتر یا تمرین اضافی دارد. البته در گزارش حاضر، مقدار واقعی این درصدها ذکر نمی‌شود، زیرا این مقادیر به اجرای واقعی برنامه و داده‌های خروجی وابسته هستند.

علاوه بر درصد یادگیری مفاهیم، تابع generate_teacher_analysis توزیع کلی یادگیری را نیز محاسبه می‌کند. در این بخش، همه نمره‌های ثبت شده در answer_logs بررسی می‌شوند. تعداد پاسخ‌هایی که نمره آن‌ها حداقل ۰.۶ است، در متغیر learned ذخیره می‌شود و تعداد پاسخ‌هایی که کمتر از ۰.۶ هستند، در متغیر not_learned قرار می‌گیرد. سپس این دو مقدار در نمودار دایره‌ای نمایش داده می‌شوند. این نمودار نشان می‌دهد که در کل شبیه سازی، چه بخشی از پاسخ‌ها در سطح قابل قبول یادگیری قرار گرفته‌اند و چه بخشی هنوز به سطح مطلوب نرسیده‌اند.

یکی دیگر از معیارهای غیرمستقیم ارزیابی، روند تغییر تسلط در طول تکرارها است. در دیکشنری `history`، برای هر مفهوم، مقدار تسلط عمومی آن مفهوم در پایان هر تکرار ذخیره می‌شود. تابع `plot_learning_history` این مقادیر را به صورت نمودار خطی رسم می‌کند. محور افقی این نمودار شماره تکرار و محور عمودی میانگین تسلط است. این نمودار نشان می‌دهد که تسلط روی هر مفهوم در طول زمان چگونه تغییر کرده است. اگر مقدار تسلط یک مفهوم در تکرارهای بعدی افزایش یابد، می‌توان نتیجه گرفت که فرآیند یادگیری برای آن مفهوم رو به بهبود بوده است. اگر تغییرات کم باشد یا مقدار تسلط پایین باقی بماند، نشان‌دهنده دشواری آن مفهوم یا ضعف پاسخ‌دهی دانشجویان در آن بخش است.

بنابراین معیارهای ارزیابی در این پروژه چندلایه هستند. در پایین‌ترین سطح، هر پاسخ دانشجو یک نمره عددی بین صفر تا یک دریافت می‌کند. در سطح بعدی، این نمره برای به‌روزرسانی مقدار تسلط دانشجو روی مفهوم استفاده می‌شود. در سطح گروهی، میانگین تسلط دانشجویان برای ساخت گراف عمومی محاسبه می‌شود. در سطح تحلیلی، آستانه ۰.۶ برای تشخیص یادگرفته‌شدن یا نشدن پاسخ‌ها استفاده می‌شود. در نهایت، نمودارهای ستونی، دایره‌ای و خطی به معلم کمک می‌کنند وضعیت یادگیری را از چند زاویه مختلف بررسی کند.

۴-۴ پیاده سازی کد

در این فصل، پیاده‌سازی عملی سامانه شبیه‌ساز گراف دانش آموزشی توضیح داده می‌شود. در فصل قبل، روش مدل‌سازی و منطق شبیه‌سازی معرفی شد. در این فصل تمرکز بر این است که این منطق چگونه در قالب کد اجرایی پیاده‌سازی شده و چگونه کاربر از طریق رابط وب با سامانه تعامل می‌کند. پیاده‌سازی پروژه در دو فایل اصلی انجام شده است: فایل `app.py` که شامل منطق سمت سرور، ارتباط با مدل زبانی، شبیه‌سازی یادگیری، تولید گراف‌ها و مدیریت مسیرهای `Flask` است؛ و فایل `index.html` که شامل رابط کاربری، فرم ورودی، نمایش نتایج، نوار پیشرفت و کد `JavaScript` برای ارتباط با سرور است.

فایل `app.py` با وارد کردن کتابخانه‌های موردنیاز شروع می‌شود. کتابخانه‌های `threading`، `uuid`، `json`، `re`، `os` و `traceback` برای مدیریت مسیرها، پردازش متن، کار با `JSON`، تولید شناسه یکتا، اجرای پس‌زمینه و مدیریت خطا استفاده شده‌اند.

در ادامه، از `Flask` ابزارهای موردنیاز مانند `Flask.render_template`، `Flask.request`، `Flask.jsonify`، `Flask.redirect` و `Flask.url_for` وارد شده‌اند. این ابزارها برای ساخت برنامه وب، دریافت داده از فرم، ارسال پاسخ `JSON`، هدایت کاربر به صفحه نتیجه و تولید آدرس فایل‌های استاتیک استفاده می‌شوند. همچنین `load_dotenv` برای خواندن متغیرهای محیطی و `OpenAI` برای ارتباط با مدل زبانی وارد شده‌اند. پس از فراخوانی `load_dotenv`، شیء اصلی `Flask` با ساخته می‌شود.

در بخش تنظیمات اولیه، متغیر `MODEL` تعریف شده است. مقدار این متغیر از محیط خوانده می‌شود و اگر مقدار مشخصی وجود نداشته باشد، مدل پیش‌فرض `gpt-3.5-turbo` استفاده می‌شود. سپس دیکشنری `jobs` و قفل `jobs_lock` تعریف شده‌اند. دیکشنری `jobs` برای ذخیره وضعیت هر اجرای شبیه‌سازی استفاده می‌شود. چون اجرای شبیه‌سازی در نخ جداگانه انجام می‌شود و هم‌زمان درخواست‌های وضعیت نیز به این دیکشنری دسترسی دارند، قفل `jobs_lock` برای جلوگیری از تداخل استفاده شده است.

تابع `get_client` مسئول ساخت کلاینت ارتباط با مدل زبانی است. این تابع کلید `API` و آدرس پایه سرویس را دریافت می‌کند و سپس شیء `OpenAI` را می‌سازد. در این پروژه، مدل زبانی از طریق یک آدرس پایه مشخص فراخوانی می‌شود. اگر کلید

API موجود نباشد، تابع خطا ایجاد می‌کند. از نظر پیاده‌سازی، این تابع باعث می‌شود ساخت کلاینت مدل زبانی در یک نقطه متمرکز باشد و توابع مختلف برنامه برای تولید مفاهیم، پاسخ دانشجو و ارزیابی پاسخ‌ها بتوانند از همین تابع استفاده کنند.

تابع `safe_int` برای کنترل ورودی‌های عددی کاربر پیاده‌سازی شده است. از آنجایی که داده‌های فرم به‌صورت رشته دریافت می‌شوند، این تابع تلاش می‌کند مقدار ورودی را به عدد صحیح تبدیل کند. اگر تبدیل ناموفق باشد، مقدار پیش‌فرض استفاده می‌شود. سپس اگر حداقل یا حداکثر مشخص شده باشد، مقدار عددی در همان بازه محدود می‌شود. این تابع در مسیر `start/` برای کنترل تعداد مفاهیم و تعداد تکرارها استفاده شده است. به این ترتیب، اگر کاربر مقدار نامعتبر وارد کند، برنامه از مقدار پیش‌فرض استفاده می‌کند و از اجرای غیرقابل کنترل جلوگیری می‌شود.

توابع `set_job_error`، `update_job_progress` و `set_job_done` برای مدیریت وضعیت شبیه‌سازی نوشته شده‌اند. تابع اول درصد پیشرفت و پیام فعلی را در دیکشنری `jobs` به‌روزرسانی می‌کند. درصد پیشرفت بین صفر تا صد محدود شده است. تابع `set_job_error` زمانی استفاده می‌شود که در اجرای شبیه‌سازی خطایی رخ دهد. این تابع وضعیت کار را `error` قرار می‌دهد، پیام خطا را ذخیره می‌کند و درصد پیشرفت را صدمی می‌کند. تابع `set_job_done` نیز زمانی اجرا می‌شود که شبیه‌سازی با موفقیت تمام شده باشد. این تابع وضعیت را `done` قرار می‌دهد، نتیجه نهایی را ذخیره می‌کند و پیام پایان موفقیت‌آمیز را ثبت می‌نماید.

تابع `extract_json` برای پردازش خروجی مدل زبانی استفاده می‌شود. مدل زبانی باید مفاهیم و وابستگی‌ها را در قالب JSON برگرداند، اما ممکن است گاهی خروجی را داخل بلوک Markdown قرار دهد یا متن اضافی تولید کند. این تابع ابتدا متن را پاک‌سازی می‌کند، نشانه‌های `json` و را حذف می‌کند و سپس تلاش می‌کند آن را به JSON تبدیل کند. اگر تبدیل مستقیم موفق نباشد، با استفاده از عبارت منظم، بخشی از متن که شبیه شیء JSON است استخراج می‌شود. این طراحی باعث افزایش مقاومت برنامه در برابر خطاهای قالب‌بندی خروجی مدل می‌شود.

تابع `generate_curriculum` یکی از اصلی‌ترین بخش‌های پیاده‌سازی است. این تابع موضوع و تعداد مفاهیم را دریافت می‌کند، یک `prompt` برای مدل زبانی می‌سازد و از مدل می‌خواهد دقیقاً همان تعداد مفهوم اصلی و روابط پیش‌نیازی میان آن‌ها را تولید کند. در پرامپت تأکید شده است که خروجی باید فقط JSON معتبر باشد و هیچ توضیح اضافی خارج از JSON وجود نداشته باشد. پس از دریافت پاسخ، تابع `extract_json` آن را پردازش می‌کند. سپس مفاهیم پاک‌سازی می‌شوند، تعداد آن‌ها کنترل می‌شود و وابستگی‌ها فقط در صورتی نگه داشته می‌شوند که معتبر باشند. در نهایت این تابع دو مقدار `concepts` و `cleaned_dependencies` را برمی‌گرداند.

کلاس `Student` برای مدل‌سازی دانشجویان استفاده شده است. هر دانشجو دارای نام، سطح، حافظه و گراف دانش است. در سازنده کلاس، برای هر مفهوم یک گره در گراف دانش ایجاد می‌شود و مقدار تسلط آن صفر قرار می‌گیرد. سپس روابط وابستگی به گراف اضافه می‌شوند. متد `learn` توضیح مربوط به یک مفهوم را در حافظه دانشجو ذخیره می‌کند. متد `get_style` بر اساس سطح دانشجو، پیام سیستمی و مقدار دمای مدل را مشخص می‌کند. متد `answer` نیز سؤال را همراه با حافظه دانشجو به مدل زبانی می‌دهد و پاسخ یک‌خطی تولید می‌کند. به این ترتیب، دانشجو در برنامه فقط یک ساختار داده ساده نیست، بلکه دارای حافظه، سطح رفتاری و گراف دانش اختصاصی است.

تابع `evaluate` برای ارزیابی پاسخ دانشجویان پیاده‌سازی شده است. این تابع سؤال و پاسخ دانشجو را دریافت می‌کند و با استفاده از مدل زبانی، نمره‌ای بین صفر تا یک تولید می‌کند. سپس با استفاده از عبارت منظم، عدد معتبر از متن خروجی استخراج می‌شود. اگر عدد استخراج نشود، مقدار صفر برگردانده می‌شود. این تابع نقش معیار اصلی ارزیابی در سامانه را دارد و خروجی آن برای به‌روزرسانی تسلط دانشجویان استفاده می‌شود.

کلاس `GeneralKG` برای نگهداری گراف دانش عمومی طراحی شده است. این کلاس نیز مانند کلاس `Student` یک گراف جهت‌دار می‌سازد، اما هدف آن نمایش وضعیت کلی گروه دانشجویان است. متد `update` در این کلاس برای هر مفهوم، مقدار تسلط همان مفهوم را از گراف تمام دانشجویان استخراج می‌کند و میانگین آن‌ها را با استفاده از `NumPy` محاسبه می‌کند. نتیجه این میانگین به‌عنوان مقدار تسلط گره مربوط به همان مفهوم در گراف عمومی ذخیره می‌شود.

تابع `build_curriculum` برای ایجاد محتوای آموزشی ساده برای هر مفهوم استفاده شده است. این تابع برای هر مفهوم، یک توضیح و یک سؤال تولید می‌کند. توضیح بیان می‌کند که مفهوم مربوط به موضوع وارد شده است و باید تعریف، نقش، مثال‌ها و ارتباط آن با دیگر مفاهیم مطالعه شود. سؤال نیز از دانشجو می‌پرسد که آن مفهوم در موضوع موردنظر چیست. این ساختار برای همه مفاهیم ساخته می‌شود و در فرآیند شبیه‌سازی مورد استفاده قرار می‌گیرد.

تابع `draw_knowledge_graph` وظیفه رسم گراف دانش را برعهده دارد. این تابع ابتدا یک شکل با اندازه مشخص می‌سازد. اگر گراف خالی باشد، عنوان `Empty` گراف دانش نمایش داده و تصویر ذخیره می‌شود. اگر گراف دارای گره باشد، موقعیت گره‌ها با `nx.spring_layout` محاسبه می‌شود. سپس برچسب هر گره شامل نام مفهوم و مقدار تسلط آن ساخته می‌شود. مقدار تسلط گره‌ها نیز برای تعیین رنگ آن‌ها استفاده می‌شود. گره‌ها، یال‌ها و برچسب‌ها با توابع `NetworkX` رسم می‌شوند و تصویر نهایی در مسیر خروجی ذخیره می‌گردد.

تابع `plot_learning_history` برای رسم روند یادگیری استفاده شده است. این تابع برای هر مفهوم، مقادیر ذخیره‌شده در تاریخچه یادگیری را به‌صورت خطی رسم می‌کند. محور افقی شماره تکرار و محور عمودی میانگین تسلط است. مقدار محور عمودی بین صفر تا یک تنظیم شده است. اگر تاریخچه وجود داشته باشد، راهنمای نمودار نیز نمایش داده می‌شود. تصویر نهایی در مسیر مشخص‌شده ذخیره می‌گردد.

تابع `generate_teacher_analysis` تحلیل مخصوص معلم را تولید می‌کند. در این تابع، ابتدا آستانه یادگیری برابر ۰.۶ قرار داده می‌شود. سپس برای هر مفهوم، تمام نمره‌های مربوط به آن مفهوم از `answer_logs` استخراج می‌شوند. تعداد نمره‌هایی که برابر یا بیشتر از آستانه هستند شمارش می‌شود و درصد یادگیری آن مفهوم محاسبه می‌گردد. سپس نمودار ستونی درصد یادگیری مفاهیم رسم و ذخیره می‌شود. در ادامه، همه نمره‌ها بررسی می‌شوند و تعداد پاسخ‌های یادگرفته‌شده و یادگرفته‌نشده محاسبه می‌گردد. این دو مقدار در نمودار دایره‌ای نمایش داده می‌شوند. در نهایت، درصد یادگیری مفاهیم و نام فایل‌های نمودارها برگردانده می‌شود.

تابع `run_simulation` هسته اصلی اجرای برنامه است. این تابع کل فرآیند شبیه‌سازی را مدیریت می‌کند. ابتدا پیشرفت کار روی ۳ درصد قرار می‌گیرد و مفاهیم تولید می‌شوند. سپس دانشجویان و گراف‌ها مقداردهی اولیه می‌شوند. سه دانشجو با سطوح ضعیف، متوسط و قوی ساخته می‌شوند. سپس گراف عمومی و محتوای آموزشی ساخته می‌شود. بعد از آن، ساختارهای لازم

برای ذخیره تاریخچه، پاسخ‌ها و تصاویر تکرارها آماده می‌شوند. برای هر اجرا نیز یک شناسه یکتا ساخته می‌شود تا فایل‌های تصویری خروجی با اجراهای دیگر تداخل نداشته باشند.

در حلقه اصلی تابع `run_simulation`، به تعداد تکرارهای مشخص شده توسط کاربر، فرآیند آموزش و ارزیابی انجام می‌شود. در هر تکرار، ابتدا دانشجویان توضیح همه مفاهیم را دریافت می‌کنند. سپس برای هر مفهوم از هر دانشجو سؤال پرسیده می‌شود. پاسخ تولید می‌شود، ارزیابی انجام می‌شود، مقدار تسلط دانشجو به‌روزرسانی می‌شود و اطلاعات پاسخ در `answer_logs` ذخیره می‌شود. پس از پایان پرسش‌ها، گراف عمومی به‌روزرسانی می‌شود، تاریخچه یادگیری ثبت می‌شود و تصویر گراف آن تکرار ذخیره می‌گردد.

بعد از پایان تکرارها، تصاویر و نمودارهای نهایی تولید می‌شوند. گراف نهایی عمومی، نمودار روند یادگیری، گراف دانشجوی ضعیف، نمودار ستونی درصد یادگیری و نمودار دایره‌ای توزیع یادگیری ساخته می‌شوند. سپس تابع `run_simulation` یک دیکشنری شامل تمام نتایج برمی‌گرداند. این دیکشنری شامل موضوع، مفاهیم، وابستگی‌ها، تاریخچه، پاسخ‌ها، مسیر تصاویر و داده‌های تحلیل معلم است.

تابع `background_worker` برای اجرای ایمن شبیه‌سازی در نخ پس‌زمینه استفاده می‌شود. این تابع تلاش می‌کند `run_simulation` را اجرا کند. اگر اجرا موفق باشد، نتیجه با `set_job_done` ذخیره می‌شود. اگر خطایی رخ دهد، خطا در خروجی چاپ می‌شود و وضعیت کار با `set_job_error` به حالت خطا تغییر می‌کند. استفاده از `traceback.print_exc()` نیز باعث می‌شود جزئیات خطا در ترمینال قابل مشاهده باشد و عیب‌یابی برنامه آسان‌تر شود.

در بخش مسیرهای `Flask`، مسیر صفحه اصلی را نمایش می‌دهد و مقدار `result` و `error` را برابر `None` قرار می‌دهد. مسیر `start/` داده‌های فرم را دریافت می‌کند، ورودی‌ها را کنترل می‌کند، یک شناسه شبیه‌سازی می‌سازد، وضعیت اولیه را در `jobs` ذخیره می‌کند و سپس یک نخ پس‌زمینه برای اجرای شبیه‌سازی ایجاد می‌کند. این مسیر در نهایت یک پاسخ `JSON` شامل وضعیت `started` و شناسه کار برمی‌گرداند. مسیر `/job_id/status/` وضعیت فعلی شبیه‌سازی را از دیکشنری `jobs` می‌خواند و به‌صورت `JSON` بازمی‌گرداند. مسیر `/job_id/result/` نیز اگر شبیه‌سازی تمام شده باشد، نتیجه را به قالب `index.html` می‌فرستد تا در صفحه نمایش داده شود.

فایل `index.html` رابط کاربری پروژه را تشکیل می‌دهد. در این فایل ابتدا ساختار صفحه و سبک‌های `CSS` تعریف شده‌اند. صفحه دارای یک `container` مرکزی، عنوان، زیرعنوان، فرم ورودی، بخش خطا، بخش نوار پیشرفت و بخش‌های نمایش نتیجه است. فرم شامل سه ورودی است: موضوع، تعداد مفاهیم و تعداد تکرارها. دکمه `Run Simulation` اجرای شبیه‌سازی را آغاز می‌کند. طراحی صفحه با استفاده از `CSS` ساده اما مرتب انجام شده است. رنگ پس‌زمینه، حاشیه‌ها، فاصله‌ها، جدول‌ها، تصاویر، `badge`ها، پیام‌های خطا و موفقیت همگی در همین فایل تعریف شده‌اند.

در بخش `JavaScript` فایل `index.html`، ابتدا عناصر اصلی صفحه مانند فرم، دکمه اجرا، نوار پیشرفت و پیام پیشرفت انتخاب می‌شوند. زمانی که فرم ارسال می‌شود، از ارسال عادی صفحه جلوگیری می‌شود و به‌جای آن با استفاده از `fetch` یک درخواست `POST` به مسیر `start/` ارسال می‌شود. اگر شبیه‌سازی با موفقیت شروع شود، تابع `checkStatus` فراخوانی می‌شود. این تابع هر یک ثانیه وضعیت شبیه‌سازی را از مسیر `/job_id/status/` می‌گیرد و درصد پیشرفت، عرض نوار پیشرفت و

پیام وضعیت را به روزرسانی می کند. اگر وضعیت `done` باشد، صفحه به مسیر نتیجه هدایت می شود. اگر وضعیت `error` باشد، پیام خطا نمایش داده شده و دکمه اجرا دوباره فعال می شود.

در بخش نمایش نتایج، از قابلیت `template` در `Flask/Jinja` استفاده شده است. اگر متغیر `result` وجود داشته باشد، بخش های مختلف خروجی در صفحه نمایش داده می شوند. ابتدا پیام موفقیت نمایش داده می شود. سپس مفاهیم، وابستگی ها، تحلیل معلم، گراف های تکرارها، گراف نهایی، گراف دانشجوی ضعیف، نمودار روند یادگیری و جدول پاسخ های دانشجویان به ترتیب نمایش داده می شوند. تصاویر با استفاده از `url_for('static', filename` ...) از پوشه `static` خوانده می شوند. این طراحی باعث می شود تصاویر تولیدشده در سمت سرور به درستی در مرورگر نمایش داده شوند.

در انتهای فایل `app.py`، اگر برنامه به صورت مستقیم اجرا شود، ابتدا پوشه `static/generated` ساخته می شود و سپس `Flask` با حالت `debug` اجرا می گردد. پارامتر `use_reloader=False` قرار داده شده است تا در حالت `debug`، نخ های پس زمینه به صورت تکراری ایجاد نشوند. این نکته برای برنامه هایی که از `threading` استفاده می کنند مهم است، زیرا فعال بودن `reloader` می تواند باعث اجرای دوباره کد و ایجاد رفتارهای ناخواسته شود.

بنابراین پیاده سازی این پروژه از چند بخش هماهنگ تشکیل شده است. سمت سرور با `Flask` منطق اصلی را اجرا می کند. مدل زبانی برای تولید مفاهیم، پاسخ دهی و ارزیابی استفاده می شود. گراف ها و نمودارها با `NetworkX` و `Matplotlib` ساخته می شوند. وضعیت اجرا با `uuid.threading` و دیکشنری `jobs` مدیریت می شود. سمت کاربر نیز با `CSS.HTML` و `JavaScript` امکان ورود اطلاعات، مشاهده نوار پیشرفت و بررسی نتایج را فراهم می کند. این ساختار باعث شده است که پروژه از حالت یک اسکریپت ساده خارج شده و به یک سامانه وب تعاملی برای شبیه سازی و تحلیل یادگیری تبدیل شود.

فصل ۵: نتایج

در بخش نتایج، سه سناریوی شبیه‌سازی شده برای موضوع **neurosis** نشان می‌دهد که ترکیب سطح توانایی دانشجویان و تعداد مفاهیم آموزشی چگونه می‌تواند بر خروجی نهایی گراف دانش اثر بگذارد. در این بخش سه سناریو را بررسی کردیم:

سناریوی اول حالت ترکیبی‌تری دارد؛ در این حالت گروه شامل یک دانشجوی ضعیف، یک دانشجوی متوسط و یک دانشجوی قوی است. این سناریو از نظر آموزشی به شرایط واقعی کلاس نزدیک‌تر است، زیرا معمولاً کلاس‌ها از دانشجویانی با سطح یکسان تشکیل نمی‌شوند. کاهش تعداد مفاهیم در سناریوی سوم می‌تواند باعث ساده‌تر شدن بار شناختی شود، اما وجود دانشجوی ضعیف و متوسط باعث می‌شود میانگین نهایی الزاماً به سطح گروه قوی نرسد.

سناریوی دوم، با ۱۰ مفهوم و همان تعداد **iteration** یعنی ۲ مرحله اجرا شده، اما این بار جامعه فقط شامل ۳ دانشجوی ضعیف است؛ در نتیجه، با وجود یکسان بودن موضوع، تعداد مفاهیم و تعداد مراحل یادگیری، عملکرد مورد انتظار پایین‌تر خواهد بود، چون دانشجویان ضعیف برای فهم مفاهیم پایه، تثبیت روابط بین مفاهیم و عبور از گره‌های دشوار گراف به زمان، تکرار و پشتیبانی بیشتری نیاز دارند. مقایسه این دو حالت اهمیت زیادی دارد، زیرا نشان می‌دهد وقتی ساختار محتوایی ثابت است، تفاوت در سطح اولیه دانشجویان می‌تواند عامل اصلی تفاوت در میزان **mastery** و **remaining gap** باشد.

سناریوی سوم، تعداد مفاهیم را ۱۰ در نظر گرفته است و جامعه یادگیرندگان فقط شامل ۳ دانشجوی قوی است؛ بنابراین انتظار می‌رود میزان تسلط نهایی بالاتر باشد، زیرا دانشجویان قوی معمولاً با سرعت بیشتری روابط بین مفاهیم را درک می‌کنند، وابستگی‌های دانشی را بهتر تشخیص می‌دهند و در دو مرحله یادگیری می‌توانند بخش بیشتری از گراف را پوشش دهند.

بنابراین، این سه شبیه‌سازی در کنار هم نشان می‌دهند که نتایج یادگیری فقط به موضوع درس یا تعداد **iteration** وابسته نیست، بلکه به ترکیب توانایی دانشجویان، تعداد مفاهیم موجود در گراف و میزان پیچیدگی ارتباط میان آن‌ها نیز بستگی دارد. به طور کلی، سناریوی دانشجویان قوی می‌تواند نقش حالت ایده‌آل یا سقف عملکرد را نشان دهد، سناریوی دانشجویان ضعیف وضعیت پرچالش و نیازمند مداخله آموزشی را برجسته می‌کند، و سناریوی ترکیبی تصویری واقع‌بینانه‌تر از کلاس‌های ناهمگن ارائه می‌دهد؛ به همین دلیل، تحلیل این سه حالت می‌تواند مقدمه‌ای مناسب برای نتیجه‌گیری باشد که در آن بر ضرورت طراحی آموزش تطبیقی، افزایش مراحل یادگیری برای گروه‌های ضعیف‌تر، و شخصی‌سازی مسیر آموزشی بر اساس سطح واقعی دانشجویان تأکید می‌شود.

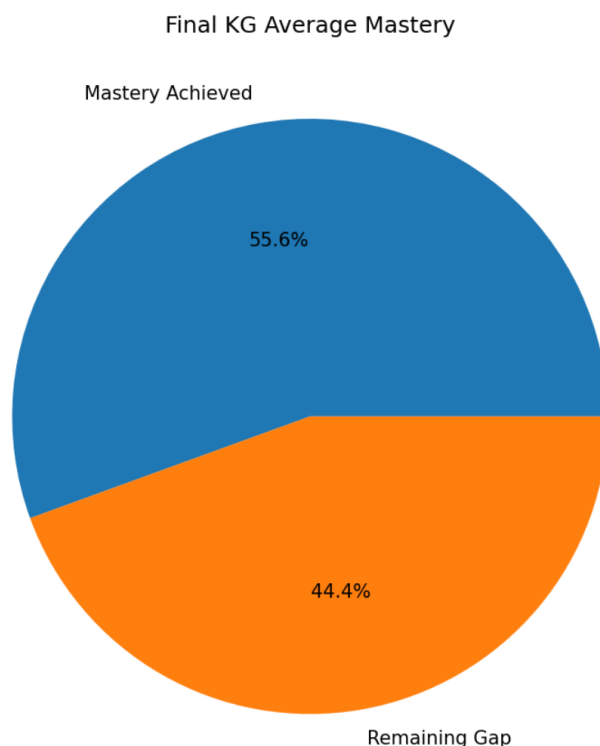
Curriculum Knowledge Graph Simulator

Generate concepts, simulate learning, show static knowledge graphs each iteration, then explore the interactive final graph.

Subject	Number of Concepts	Iterations
neurosis	7	2
Weak Students	Medium Students	Strong Students
1	1	1
Run Simulation		

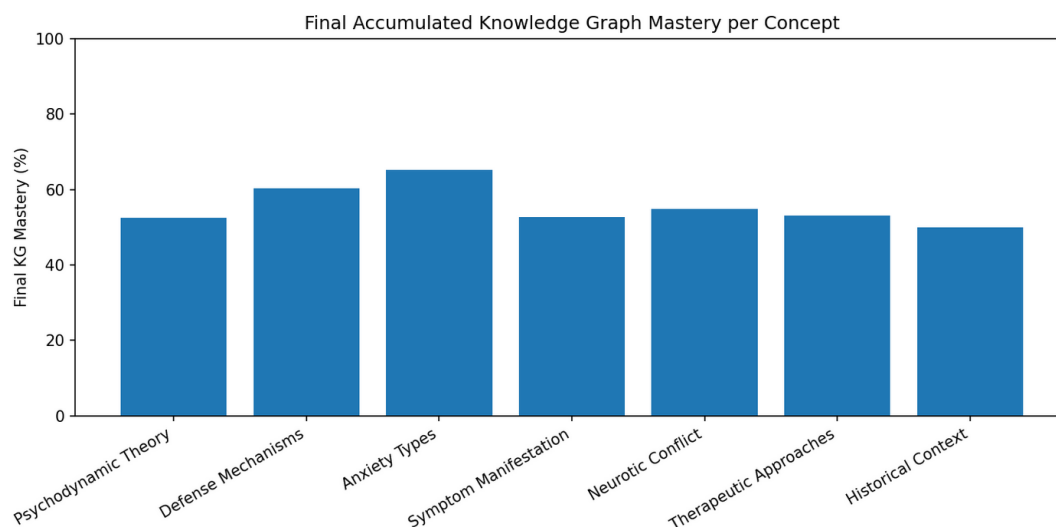
شکل ۱. مقادیر ورودی اپلیکیشن Curriculum Knowledge Graph Simulator

همانطور که در شکل ۱ دیده می شود، در این شبیه سازی، موضوع آموزشی *neurosis* با ۷ مفهوم در قالب یک گراف دانشی بررسی شده و جامعه یادگیرندگان شامل سه سطح متفاوت بوده است: یک دانشجوی ضعیف، یک دانشجوی متوسط و یک دانشجوی قوی؛ بنابراین نتیجه نهایی را نباید به عنوان عملکرد یک فرد خاص تفسیر کرد، بلکه باید آن را میانگین عملکرد یک گروه ناهمگن دانست.



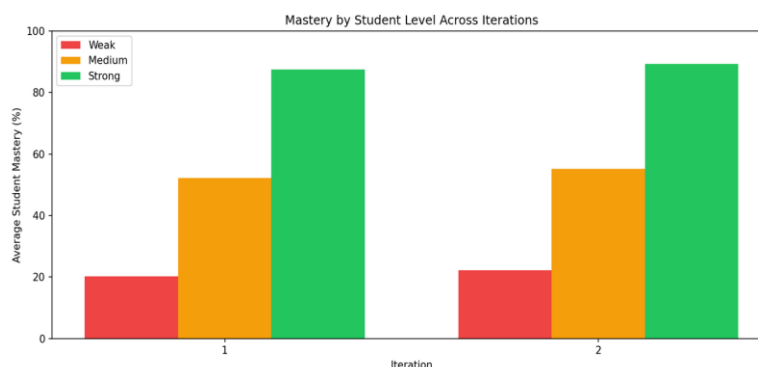
شکل ۲. نمودار دایره ای تسلط دانشجویان

طبق شکل ۲، نمودار دایره ای نشان می دهد که پس از ۲ مرحله یادگیری، میزان *Mastery Achieved* برابر با ۵۵.۶٪ شده است؛ یعنی گروه توانسته کمی بیش از نیمی از دانش و مهارت های مورد انتظار را کسب کند. در مقابل، مقدار *Remaining Gap* برابر با ۴۴.۴٪ است که عدد نسبتاً بزرگی محسوب می شود و نشان می دهد تقریباً نیمی از مسیر یادگیری هنوز کامل نشده است. این وضعیت از نظر آموزشی یک حالت میانی و قابل بهبود است: از یک طرف، یادگیری کاملاً ناموفق نبوده، چون *mastery* از ۵۰ درصد عبور کرده است؛ اما از طرف دیگر، هنوز نمی توان گفت گروه به تسلط قابل قبول یا پایدار رسیده است. دلیل اصلی این نتیجه احتمالاً ترکیب متفاوت دانشجویان است؛ دانشجوی قوی باعث بالا رفتن میانگین شده، دانشجوی متوسط بخشی از یادگیری را تثبیت کرده، اما دانشجوی ضعیف همچنان بخش زیادی از *gap* را در میانگین نهایی نگه داشته است. همچنین چون تعداد *iteration* فقط ۲ بوده، زمان و فرصت یادگیری محدود بوده و طبیعی است که مخصوصاً دانشجوی ضعیف و حتی دانشجوی متوسط نتوانند در همین دو مرحله به *mastery* بالا برسند. بنابراین این خروجی نشان می دهد که برای چنین گروهی، آموزش یکسان برای همه مناسب نیست؛ دانشجوی قوی نیاز به تمرین های چالشی و تکمیلی دارد، دانشجوی متوسط باید با تمرین های تثبیتی و مثال های بیشتر *gap* خود را کاهش دهد، و دانشجوی ضعیف به بازآموزی مرحله به مرحله، تمرکز بر مفاهیم پایه و پشتیبانی بیشتر نیاز دارد. به طور کلی، عدد ۵۵.۶٪ *mastery* نشان دهنده شروع خوب و عبور از سطح اولیه یادگیری است، اما ۴۴.۴٪ *gap* هشدار می دهد که هنوز بخش قابل توجهی از مفاهیم *neurosis* به درستی درک یا تثبیت نشده و برای رسیدن به سطح مطلوب، باید تعداد *iteration* ها افزایش یابد و مسیر آموزشی بر اساس سطح واقعی هر دانشجو شخصی سازی شود.



شکل ۳. میزان تسلط نهایی به تفکیک مفاهیم در تجميع دانشجویان

طبق نمودار شکل ۳، میزان تسلط نهایی گراف دانش برای هر مفهوم نشان داده شده است و این نمودار مشخص می‌کند که در گروهی شامل یک دانشجوی ضعیف، یک دانشجوی متوسط و یک دانشجوی قوی، همه مفاهیم به یک اندازه یاد گرفته نشده‌اند. بالاترین میزان تسلط مربوط به Anxiety Types با حدود ۶۵٪ است و پس از آن Defense Mechanisms با حدود ۶۰٪ قرار دارد؛ این موضوع نشان می‌دهد که این دو مفهوم برای گروه قابل‌فهم‌تر بوده‌اند یا ارتباط آن‌ها در گراف دانش بهتر تثبیت شده است. در سطح میانی، مفاهیمی مانند Neurotic Conflict، Therapeutic Approaches، Symptom Manifestation و Psychodynamic Theory قرار دارند که میزان تسلط آن‌ها حدود ۵۲ تا ۵۵٪ است؛ یعنی گروه در این بخش‌ها به فهم نسبی رسیده، اما هنوز تسلط کامل یا عمیق شکل نگرفته است. پایین‌ترین مقدار مربوط به Historical Context با حدود ۵۰٪ است که نشان می‌دهد این مفهوم بیشترین فاصله را تا mastery دارد و احتمالاً برای بخشی از گروه، به‌ویژه دانشجوی ضعیف، کمتر قابل درک یا کمتر جذاب بوده است. به طور کلی، این نمودار نشان می‌دهد که میانگین گروه در بیشتر مفاهیم نزدیک به سطح متوسط است و هیچ مفهومی به mastery بسیار بالا نرسیده؛ بنابراین نتیجه اصلی این است که حضور دانشجوی قوی توانسته سطح کلی را بالا نگه دارد، اما وجود دانشجوی ضعیف و متوسط باعث شده تسلط نهایی در همه مفاهیم محدود بماند و هنوز نیاز به آموزش تکمیلی وجود داشته باشد.



شکل ۴. روند تغییر میانگین تسلط دانشجویان بر اساس سطح توانایی در دو مرحله یادگیری

نمودار شکل ۴، روند میانگین تسلط دانشجویان در دو iteration را بر اساس سطح دانشجو نشان می‌دهد و تفاوت آشکاری میان سه گروه توانایی را مشخص می‌کند. دانشجوی ضعیف از حدود 20٪ در iteration اول به حدود 22٪ در iteration دوم رسیده است؛ این افزایش کم نشان می‌دهد که یادگیری برای او کند بوده و دو مرحله آموزشی برای ایجاد تغییر جدی کافی نبوده است. دانشجوی متوسط از حدود 52٪ به حدود 55٪ رسیده که رشد کمی بهتر از دانشجوی ضعیف دارد، اما همچنان در سطح میانی باقی مانده و برای رسیدن به mastery بالا نیازمند تمرین و تکرار بیشتر است. در مقابل، دانشجوی قوی از حدود 87٪ به حدود 89٪ رسیده و تقریباً از ابتدا در سطح بالایی از mastery قرار داشته است؛ افزایش اندک او طبیعی است، چون از قبل به سقف عملکرد نزدیک بوده و فضای رشد کمتری نسبت به دیگران داشته است. نکته مهم این نمودار این است که با وجود افزایش mastery در هر سه سطح، فاصله بین دانشجوی ضعیف و قوی همچنان بسیار زیاد باقی مانده است؛ یعنی iteration دوم باعث بهبود کلی شده، اما شکاف یادگیری را به‌طور جدی کاهش نداده است. بنابراین، این نمودار نشان می‌دهد که برای چنین گروه ناهمگنی، تنها افزایش محدود مراحل یادگیری کافی نیست و باید برای دانشجوی ضعیف مسیر حمایتی و پایه‌ای، برای دانشجوی متوسط مسیر تثبیتی، و برای دانشجوی قوی فعالیت‌های پیشرفته‌تر طراحی شود.

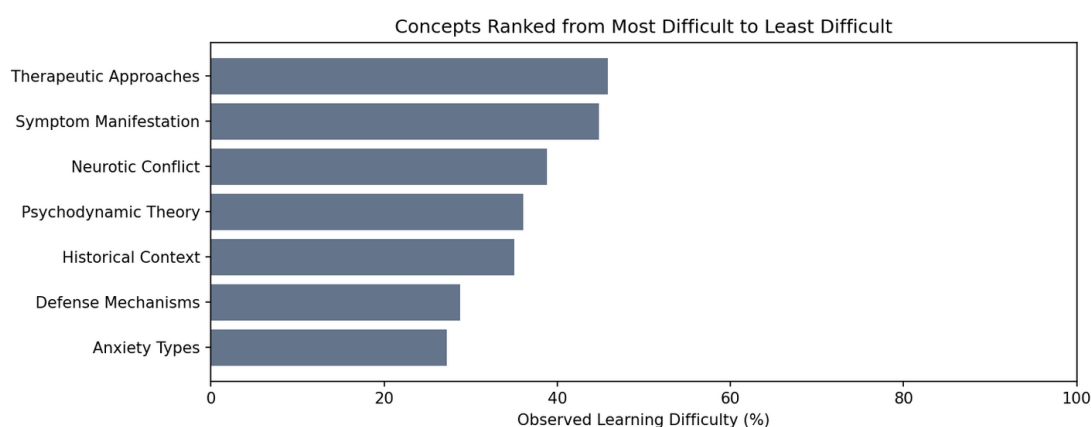
Observed Concept Learning Difficulty

Difficulty is estimated from final mastery gap, average answer-score gap, low-score rate, non-answer rate, and prerequisite structure.

Concept	Learning Difficulty
Anxiety Types	27.25%
Defense Mechanisms	28.74%
Historical Context	35.0%
Psychodynamic Theory	36.05%
Neurotic Conflict	38.79%
Symptom Manifestation	44.83%
Therapeutic Approaches	45.86%

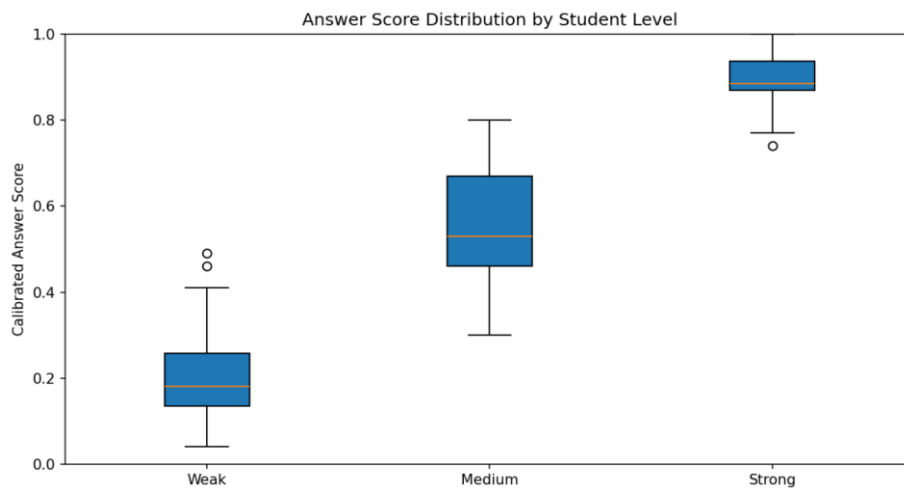
شکل ۵ جدول میزان سختی مشاهده‌شده مفاهیم

جدول شکل ۵، مقدار دقیق سختی یادگیری هر مفهوم را نشان می‌دهد و مکمل نمودار ۶ (نمودار سختی مفاهیم) است. طبق جدول، سختی از 27.25٪ برای Anxiety Types شروع می‌شود و تا 45.86٪ برای Therapeutic Approaches افزایش می‌یابد. این اعداد نشان می‌دهند که اختلاف بین آسان‌ترین و سخت‌ترین مفهوم قابل توجه است. بنابراین، جدول کمک می‌کند اولویت‌بندی آموزشی دقیق‌تر انجام شود و مفاهیم دشوارتر زودتر مورد بازآموزی قرار گیرند.



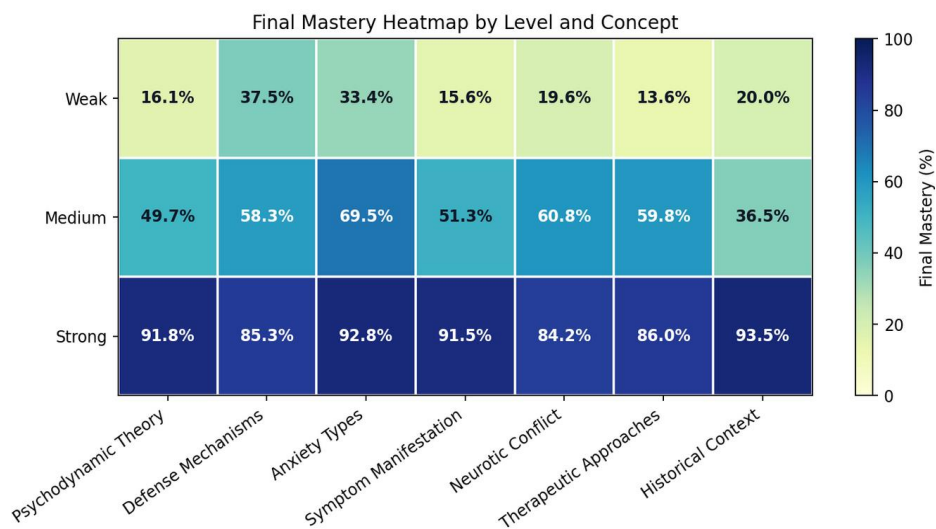
شکل ۶. نمودار سختی مفاهیم

نمودار شکل ۶، مفاهیم مربوط به neurosis را از سخت‌ترین تا آسان‌ترین رتبه‌بندی می‌کند. بیشترین سختی مربوط به Symptom Manifestation و Therapeutic Approaches است که نشان می‌دهد دانشجویان در کاربرد مفاهیم درمانی و تشخیص نشانه‌ها بیشتر دچار چالش شده‌اند. در مقابل، Anxiety Types و Defense Mechanisms سختی کمتری دارند و احتمالاً برای دانشجویان قابل‌فهم‌تر بوده‌اند. بنابراین، نتایج نشان می‌دهد آموزش‌های تکمیلی باید بیشتر روی مفاهیم تحلیلی و کاربردی متمرکز شود..



شکل ۷. نمودار توزیع نمره پاسخ بر اساس سطح دانشجو

نمودار شکل ۷، توزیع نمره پاسخ بر اساس سطح دانشجو را نشان می‌دهد. این نمودار نشان می‌دهد که نمرات پاسخ‌ها میان سه سطح دانشجو تفاوت واضحی دارد. دانشجویان قوی بالاترین و پایدارترین نمرات را دارند، در حالی که دانشجویان ضعیف پایین‌ترین نمرات و پراکندگی بیشتری نشان می‌دهند. دانشجویان متوسط در جایگاه میانی قرار گرفته‌اند و عملکرد آن‌ها نسبتاً متغیر است. این نتیجه نشان می‌دهد سطح اولیه دانشجو نقش مهمی در کیفیت پاسخ‌ها و میزان یادگیری نهایی دارد.



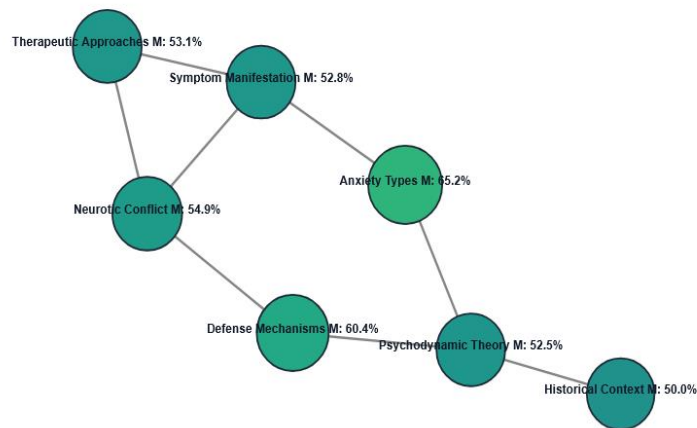
شکل ۸. توزیع نمره پاسخ بر اساس سطح دانشجو

این نقشه حرارتی^۷ نشان می‌دهد که میزان تسلط نهایی در سه سطح دانشجو، یعنی ضعیف، متوسط و قوی، تفاوت قابل توجهی دارد. دانشجویان قوی تقریباً در همه مفاهیم به تسلط بالای ۸۴ تا ۹۳ درصد رسیده‌اند، در حالی که دانشجویان ضعیف عمدتاً در بازه پایین ۱۳ تا ۳۷ درصد قرار دارند. دانشجویان متوسط در سطح میانی قرار گرفته‌اند و بیشترین تسلط آن‌ها مربوط به Anxiety Types با 69.5٪ است. این نمودار به خوبی نشان می‌دهد که سطح اولیه دانشجو نقش مهمی در میزان یادگیری نهایی دارد و برای کاهش شکاف یادگیری، آموزش باید متناسب با سطح هر گروه طراحی شود.



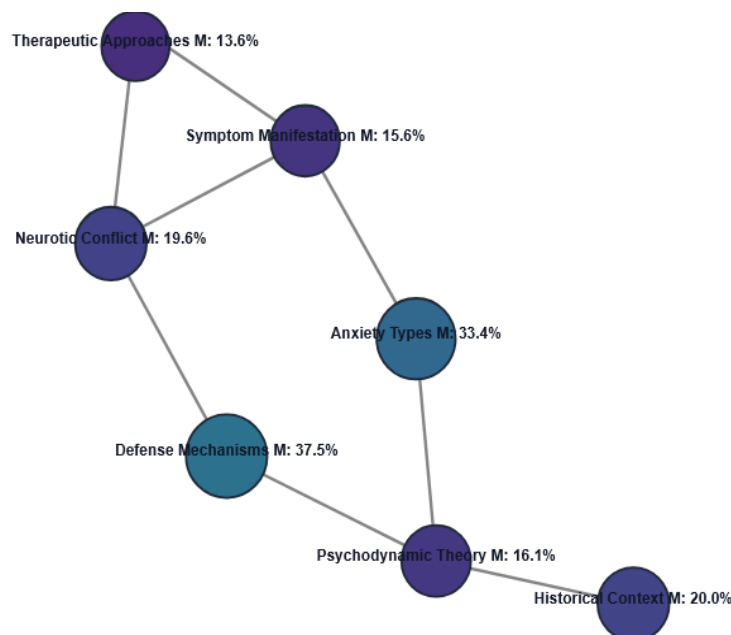
شکل ۹. گراف دانش برای تکرار ۱ یادگیری

شکل ۹، وضعیت mastery مفاهیم را در iteration اول نشان می‌دهد و مشخص می‌کند که یادگیری در این مرحله هنوز در سطح میانی قرار دارد. مفاهیمی مانند Anxiety Types و Defense Mechanisms مقدار بالاتری دارند و نشان می‌دهند زودتر از سایر مفاهیم تثبیت شده‌اند. در مقابل، Therapeutic Approaches mastery و Historical Context پایین‌تری دارند و بیشتر نیازمند تقویت هستند. این شکل نشان می‌دهد که در آغاز فرایند یادگیری، تسلط مفاهیم یکدست نیست و برخی گره‌ها به عنوان نقاط ضعف اولیه ظاهر می‌شوند.



شکل ۱۰. گراف دانش برای تکرار ۲ یادگیری

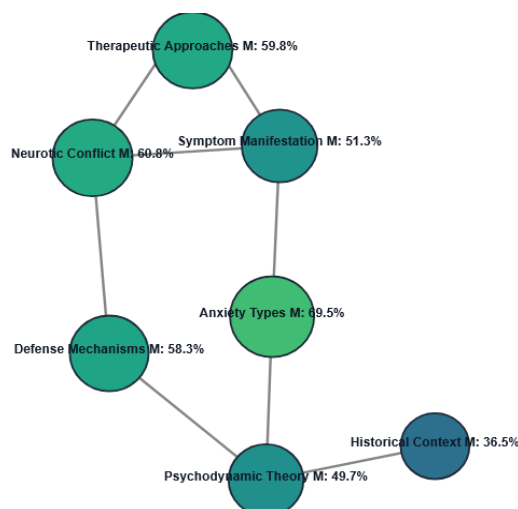
شکل ۱۰ نشان می‌دهد که در تکرار دوم مقدار mastery در بیشتر مفاهیم نسبت به مرحله قبل افزایش یافته و گراف وضعیت متعادل‌تری پیدا کرده است. Anxiety Types، همچنان یکی از قوی‌ترین گره‌هاست و مفاهیمی مانند Neurotic Conflict و Defense Mechanisms نیز در سطح مناسبی قرار گرفته‌اند. با این حال، Historical Context هنوز پایین‌ترین مقدار را دارد و می‌تواند به عنوان یکی از گلوگاه‌های یادگیری باقی بماند. این شکل نشان می‌دهد که تکرار یادگیری باعث بهبود کلی شده، اما همه مفاهیم به یک اندازه رشد نکرده‌اند.



شکل ۱۱. گراف دانش تجمیع شده ی نهایی برای دانشجویان ضعیف

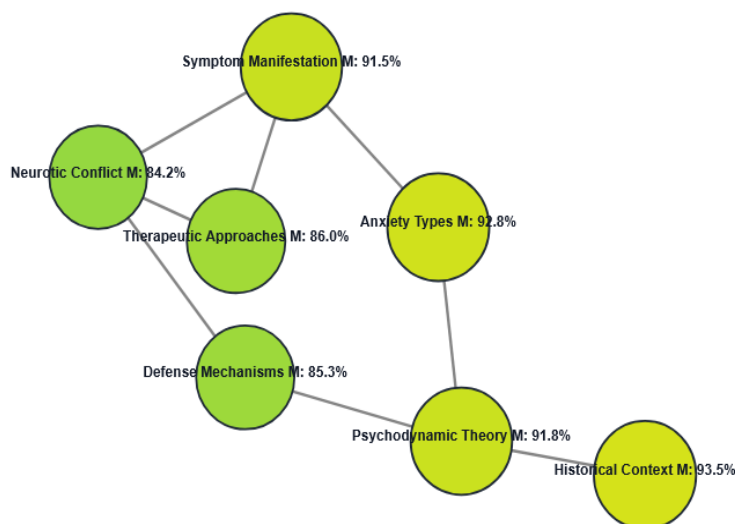
در شکل ۱۱، این گراف نشان می‌دهد که دانشجویان ضعیف در بیشتر مفاهیم mastery پایینی دارند و مقادیر آن‌ها عمدتاً زیر ۴۰ درصد باقی مانده است. بیشترین تسلط مربوط به Defense Mechanisms و Anxiety Types است، اما حتی این مفاهیم نیز به سطح قابل قبول mastery نرسیده‌اند. مفاهیمی مانند Therapeutic Approaches، Symptom

Manifestation و Psychodynamic Theory بسیار پایین تر هستند و نیاز جدی به بازآموزی دارند. بنابراین، این گراف نشان‌دهنده شکاف یادگیری گسترده در گروه ضعیف است.



شکل ۱۲. گراف دانش تجمیع شده ی نهایی برای دانشجویان متوسط

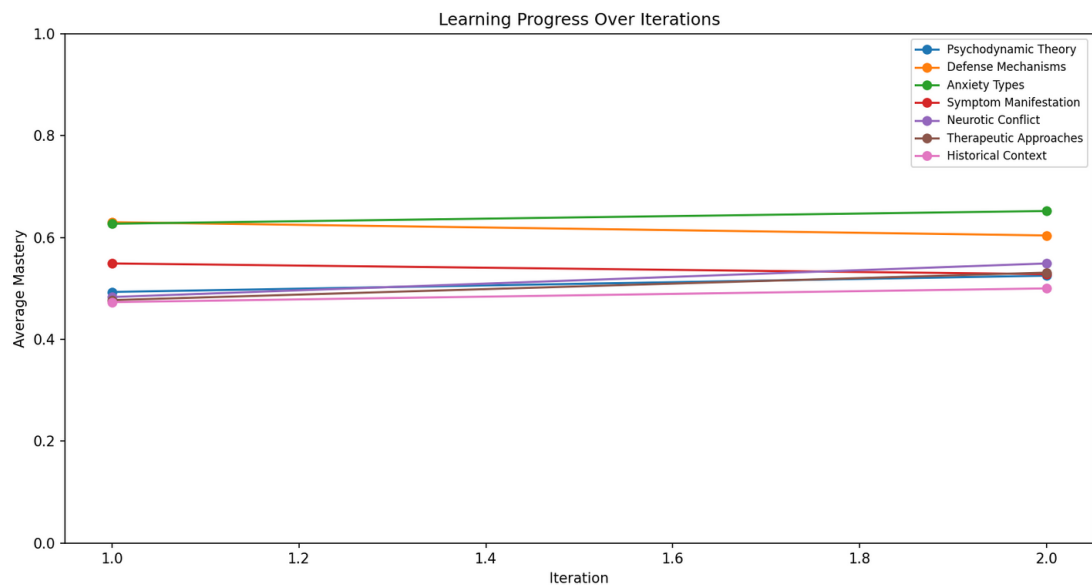
در شکل ۱۲، در گراف دانشجویان متوسط، mastery بیشتر مفاهیم در سطح میانی قرار دارد و نسبت به دانشجویان ضعیف پیشرفت قابل توجهی دیده می‌شود. Anxiety Types با حدود ۶۹٫۵٪ بالاترین مقدار را دارد و نشان می‌دهد این مفهوم برای دانشجویان متوسط بهتر تثبیت شده است. در مقابل، Historical Context پایین‌ترین مقدار را دارد و همچنان یکی از گلوگاه‌های یادگیری محسوب می‌شود. این گراف نشان می‌دهد دانشجویان متوسط به درک نسبی رسیده‌اند، اما برای رسیدن به mastery کامل نیاز به تمرین و تعمیق بیشتر دارند.



شکل ۱۳. گراف دانش تجمیع شده ی نهایی برای دانشجویان قوی

در شکل ۱۳، این گراف نشان می‌دهد که دانشجویان قوی تقریباً در همه مفاهیم به mastery بسیار بالا رسیده‌اند. مقادیر بیشتر گره‌ها بالای ۸۴ درصد است و مفاهیمی مانند Historical Context، Anxiety Types و Symptom Manifestation

از بالاترین سطح تسلط برخوردارند. پایین‌ترین مقدار نیز مربوط به Neurotic Conflict است، اما همچنان در سطح بالایی قرار دارد. بنابراین، این گراف بیانگر یادگیری پایدار و کامل‌تر در گروه قوی است و نشان می‌دهد این دانشجویان بیشتر به تمرین‌های پیشرفته و تحلیلی نیاز دارند تا آموزش پایه‌ای.



شکل ۱۴. نمایش پروسه یادگیری در ۲ تکرار

شکل ۱۴، روند پیشرفت یادگیری مفاهیم را در دو تکرار نشان می‌دهد و مشخص می‌کند که بیشتر مفاهیم از تکرار اول به تکرار دوم رشد ملایمی داشته‌اند. مفهوم Anxiety Types در هر دو تکرار بالاترین سطح mastery را دارد و نشان می‌دهد برای دانشجویان قابل‌فهم‌تر بوده است. در مقابل، مفاهیمی مانند Historical Context و Therapeutic Approaches در Psychodynamic Theory سطح پایین‌تری قرار دارند و می‌توانند به‌عنوان بخش‌های نیازمند تقویت شناخته شوند. نکته مهم این است که شیب خطوط زیاد نیست؛ یعنی دو تکرار باعث بهبود شده، اما برای رسیدن به تسلط کامل کافی نبوده است.

Student Answers

Iteration	Student	Level	Concept	Answer	Score
1	S1	weak	Psychodynamic Theory	It is about unconscious conflicts maybe.	0.13
1	S1	weak	Defense Mechanisms	They protect the ego from anxiety.	0.41
1	S1	weak	Anxiety Types	It is fear of something unknown.	0.26
1	S1	weak	Symptom Manifestation	Symptoms are physical signs of stress.	0.2
1	S1	weak	Neurotic Conflict	It is a fight inside the mind.	0.11
1	S1	weak	Therapeutic Approaches	Talking helps resolve inner issues.	0.04
1	S1	weak	Historical Context	It started with Freud long ago.	0.25
1	S2	medium	Psychodynamic Theory	It involves unconscious processes influencing behavior.	0.43
1	S2	medium	Defense Mechanisms	They are ways to handle stress unconsciously.	0.78
1	S2	medium	Anxiety Types	It includes moral anxiety and realistic fear.	0.69
1	S2	medium	Symptom Manifestation	They are expressions of underlying unconscious conflicts.	0.46
1	S2	medium	Neurotic Conflict	It is tension between desires and rules.	0.51
1	S2	medium	Therapeutic Approaches	Therapy aims to make the unconscious conscious.	0.55
1	S2	medium	Historical Context	It emerged from early psychoanalytic studies.	0.3
1	S3	strong	Psychodynamic Theory	It focuses on unconscious drives and early experiences shaping personality.	0.93
1	S3	strong	Defense Mechanisms	They are unconscious strategies to reduce anxiety and protect self-esteem.	0.77
1	S3	strong	Anxiety Types	It distinguishes between realistic, neurotic, and moral anxiety sources.	1.0
1	S3	strong	Symptom Manifestation	Symptoms are compromise formations representing unresolved internal conflicts.	1.0
1	S3	strong	Neurotic Conflict	It is an unconscious struggle between opposing impulses and defenses.	0.74
1	S3	strong	Therapeutic Approaches	Approaches like psychoanalysis explore unconscious material to resolve conflicts.	0.83
1	S3	strong	Historical Context	It originated in late 19th-century Vienna with Freud's clinical work.	0.87

شکل ۱۵. نمایش جزئیات پاسخ دانشجویان و نمره دهی آن در تکرار ۱

2	S1	weak	Psychodynamic Theory	I think it is something about the mind maybe.	0.16
2	S1	weak	Defense Mechanisms	Maybe it is about blocking thoughts vaguely.	0.46
2	S1	weak	Anxiety Types	I think it is something about fear maybe.	0.49
2	S1	weak	Symptom Manifestation	Maybe it is about being sick vaguely.	0.09
2	S1	weak	Neurotic Conflict	I think it is something about fighting inside maybe.	0.16
2	S1	weak	Therapeutic Approaches	Maybe it is about talking to someone vaguely.	0.21
2	S1	weak	Historical Context	I think it is something about old times maybe.	0.15
2	S2	medium	Psychodynamic Theory	It involves unconscious conflicts driving behavior.	0.63
2	S2	medium	Defense Mechanisms	They are ways the ego protects itself from anxiety.	0.46
2	S2	medium	Anxiety Types	There are realistic, neurotic, and moral anxiety types.	0.8
2	S2	medium	Symptom Manifestation	Symptoms are signs of underlying psychological distress.	0.5
2	S2	medium	Neurotic Conflict	It is a struggle between opposing desires or fears.	0.67
2	S2	medium	Therapeutic Approaches	Therapy aims to make the unconscious conscious.	0.67
2	S2	medium	Historical Context	It started with Freud's early work in Vienna.	0.43
2	S3	strong	Psychodynamic Theory	It focuses on how unconscious drives and early experiences shape personality and neurosis.	0.91
2	S3	strong	Defense Mechanisms	These are unconscious strategies like repression or projection that reduce anxiety by distorting reality.	0.89
2	S3	strong	Anxiety Types	Freud identified realistic, neurotic, and moral anxiety based on the source of the threat.	0.87
2	S3	strong	Symptom Manifestation	Neurotic symptoms are compromise formations that express repressed conflicts in disguised forms.	0.87
2	S3	strong	Neurotic Conflict	It is an unconscious struggle between id impulses and superego demands that the ego cannot resolve.	0.94
2	S3	strong	Therapeutic Approaches	Psychoanalysis uses techniques like free association and dream analysis to resolve neurotic conflicts.	0.88
2	S3	strong	Historical Context	Psychodynamic theory emerged in the late 19th century, shifting focus from biology to psychology.	1.0

شکل ۱۶. نمایش جزئیات پاسخ دانشجویان و نمره دهی آن در تکرار ۲

شکل ۱۵ و ۱۶، جزئیات پاسخ‌های دانشجویان و نمره‌دهی آن‌ها را در تکرار اول نشان می‌دهد و مکمل شکل قبل است، چون مشخص می‌کند تفاوت mastery از کجا ایجاد شده است. در جدول، دانشجوی ضعیف پاسخ‌های کوتاه‌تر و ساده‌تری داده و نمرات پایین‌تری گرفته، در حالی که دانشجوی قوی پاسخ‌های دقیق‌تر، کامل‌تر و مفهومی‌تر ارائه کرده است. دانشجوی متوسط

بین این دو قرار دارد و در برخی مفاهیم عملکرد قابل قبول دارد، اما هنوز پاسخ‌های او به عمق پاسخ‌های دانشجوی قوی نمی‌رسد. بنابراین، این شکل نشان می‌دهد که اختلاف سطح دانشجویان مستقیماً در کیفیت پاسخ‌ها و در نهایت در میزان mastery گراف دانش اثر گذاشته است.

حال همه ی دانشجویان را ضعیف انتخاب می‌کنیم:

Curriculum Knowledge Graph Simulator

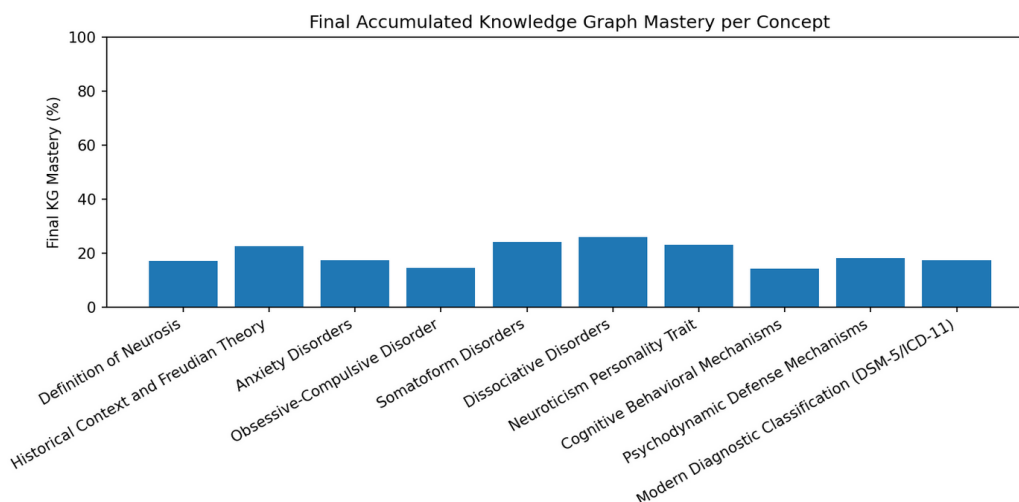
Generate concepts, simulate learning, show static knowledge graphs each iteration, then explore the interactive final graph.

Subject neurosis	Number of Concepts 10	Iterations 2
Weak Students 3	Medium Students 0	Strong Students 0

[Run Simulation](#)

شکل ۱۷. تنظیمات شبیه سازی

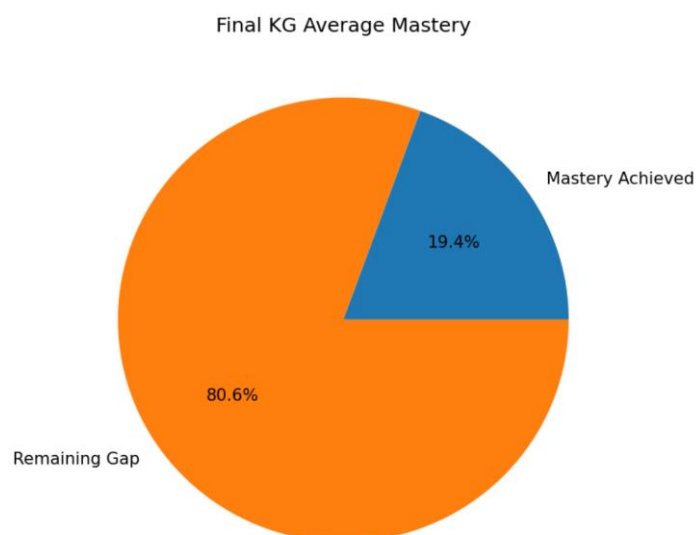
طبق شکل ۱۷، در این سناریو، با توجه به شکل بالا که نشان دهنده ی تنظیمات شبیه‌سازی است، موضوع آموزشی neurosis با ۱۰ مفهوم و در ۲ ترکیب دانشجویان فقط شامل ۳ دانشجوی ضعیف است و هیچ دانشجوی متوسط یا قوی در گروه وجود ندارد. این تنظیمات نشان می‌دهد که هدف شبیه‌سازی، بررسی عملکرد گروهی با سطح پایه پایین در برابر تعداد نسبتاً زیادی از مفاهیم بوده است. بنابراین انتظار می‌رود میزان تسلط نهایی پایین باشد، چون دانشجویان ضعیف برای یادگیری مفاهیم متعدد به زمان، تکرار و پشتیبانی بیشتری نیاز دارند



شکل ۱۸. نمودار تسلط نهایی به تفکیک مفهوم

نمودار تسلط نهایی به تفکیک مفهوم نشان می‌دهد این نمودار نشان می‌دهد که میزان Final KG Mastery برای همه مفاهیم نسبتاً پایین است و بیشتر مقادیر در حدود ۱۴ تا ۲۶ درصد قرار دارند. بالاترین تسلط مربوط به مفاهیمی مانند Dissociative Disorders و Somatoform Disorders است، اما حتی این موارد هم به سطح mastery قابل قبول نرسیده‌اند. مفاهیمی مانند Obsessive-Compulsive Disorder و Cognitive Behavioral Mechanisms

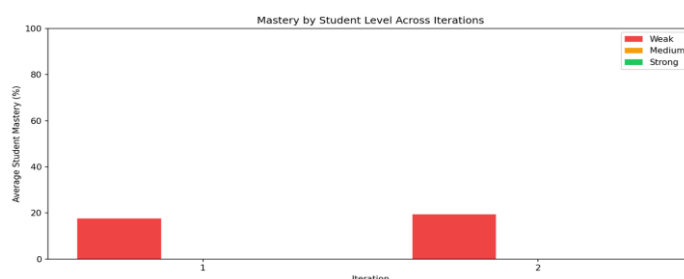
پایین‌ترین میزان تسلط را نشان می‌دهند و احتمالاً برای این گروه دشوارتر بوده‌اند. در مجموع، نمودار نشان می‌دهد که دو iteration برای گروهی شامل فقط دانشجویان ضعیف کافی نبوده و تقریباً همه مفاهیم به بازآموزی و تمرین مرحله‌ای نیاز دارند.



شکل ۱۹. نمودار دایره ای تسلط دانشجویان

این نمودار نشان می‌دهد که در پایان شبیه‌سازی، فقط ۱۹,۴٪ از گراف دانش به مرحله تسلط رسیده است. در مقابل، ۸۰,۶٪ به‌عنوان Remaining Gap باقی مانده که نشان‌دهنده فاصله بسیار زیاد تا یادگیری کامل است. چنین نتیجه‌ای معمولاً برای گروهی از دانشجویان ضعیف قابل انتظار است، چون آن‌ها برای فهم و تثبیت مفاهیم به زمان و تکرار بیشتری نیاز دارند.

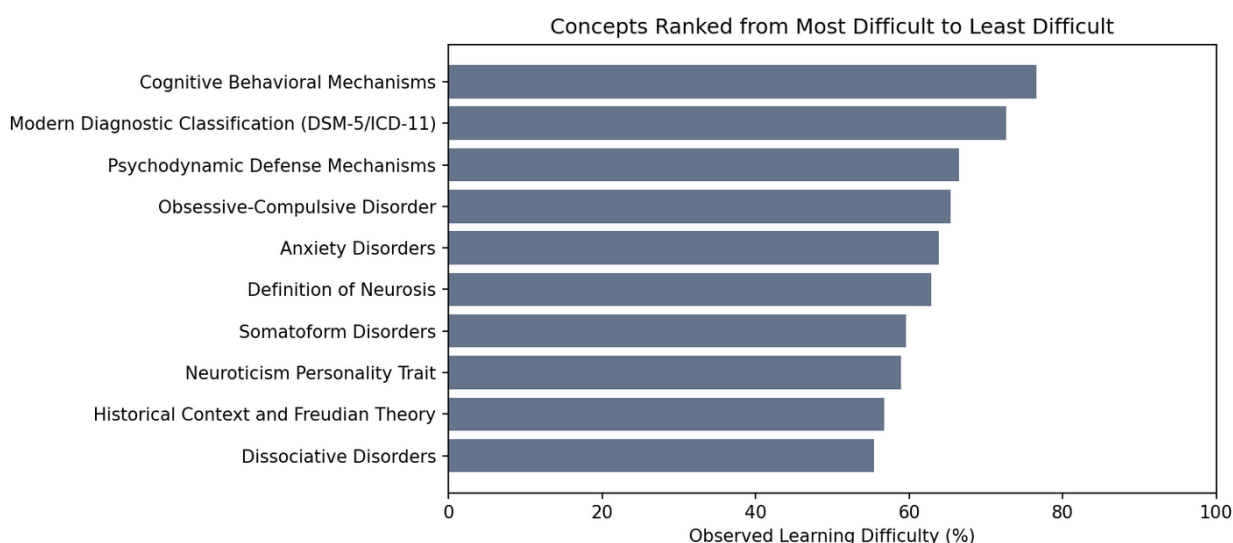
بنابراین، این خروجی نشان می‌دهد که دو مرحله یادگیری کافی نبوده و باید آموزش پایه‌ای، تمرین بیشتر و بازخورد هدفمند برای کاهش شکاف یادگیری طراحی شود.



شکل ۲۰. تغییرات میانگین تسلط دانشجویان ضعیف در دو مرحله یادگیری

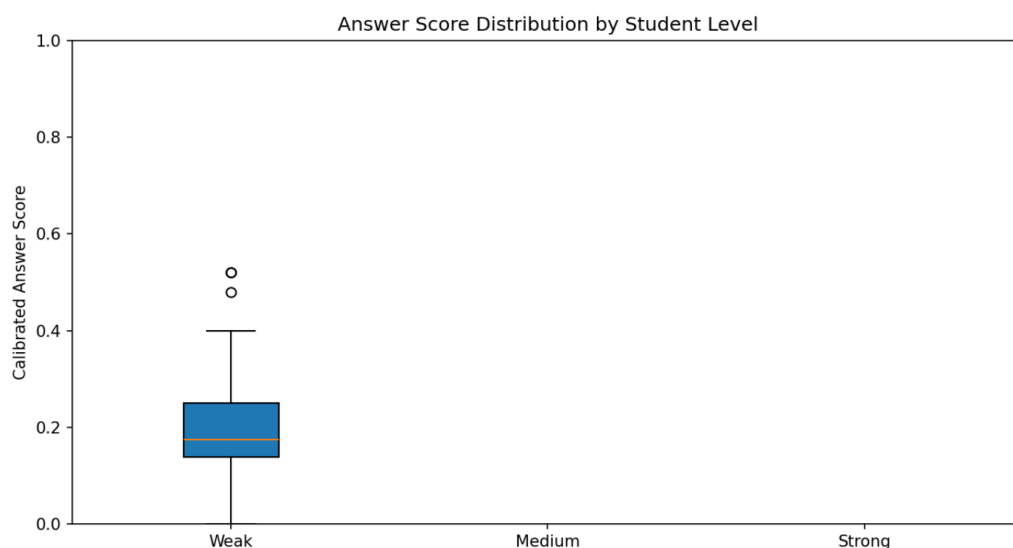
این نمودار روند mastery دانشجویان ضعیف را در دو iteration نمایش می‌دهد. میزان تسلط از حدود ۱۷٪ در مرحله اول به حدود ۱۹٪ در مرحله دوم رسیده است. این افزایش بسیار محدود نشان می‌دهد که دو مرحله یادگیری برای ایجاد

پیشرفت معنادار کافی نبوده است. بنابراین، برای این گروه باید تعداد مراحل یادگیری، تمرین‌ها و بازخوردهای آموزشی افزایش یابد.



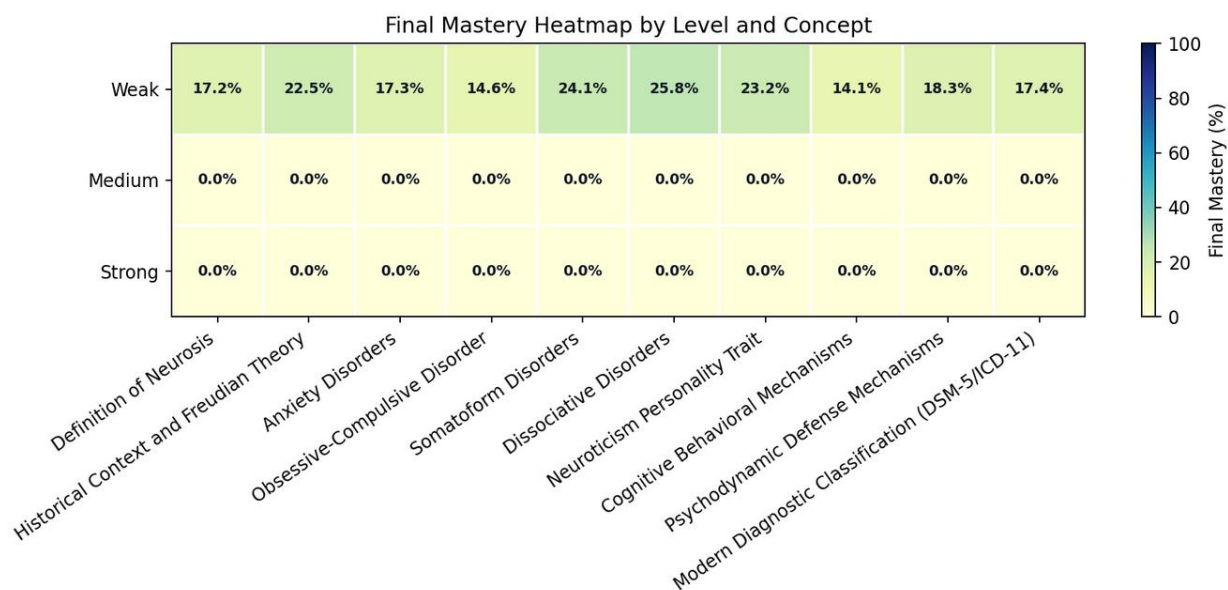
شکل ۲۱. درصد سختی یادگیری هر مفهوم به تفکیک

این جدول درصد سختی یادگیری هر مفهوم را به صورت دقیق نشان می‌دهد. کمترین سختی مربوط به **Cognitive Behavioral Disorders** با حدود ۵۵,۴۴٪ و بیشترین سختی مربوط به **Dissociative Mechanisms** با حدود ۷۶,۵۴٪ است. بالا بودن همه مقادیر نشان می‌دهد که تقریباً تمام مفاهیم برای این گروه چالش برانگیز بوده‌اند. این جدول کمک می‌کند مفاهیم دشوارتر برای بازآموزی و مداخله آموزشی در اولویت قرار گیرند.



شکل ۲۲. توزیع نمرات پاسخ دانشجویان ضعیف

این نمودار نشان می‌دهد که نمرات پاسخ فقط برای گروه دانشجویان ضعیف ثبت شده و گروه‌های متوسط و قوی در این سناریو حضور ندارند. میانه نمرات پایین است و بیشتر پاسخ‌ها در محدوده حدود ۰,۱۵ تا ۰,۲۵ قرار گرفته‌اند. وجود چند مقدار بالاتر نشان می‌دهد برخی پاسخ‌ها بهتر بوده‌اند، اما این موارد استثنایی هستند. در مجموع، نمودار ضعف کلی در کیفیت پاسخ‌ها و نیاز به آموزش پایه‌ای بیشتر را نشان می‌دهد.

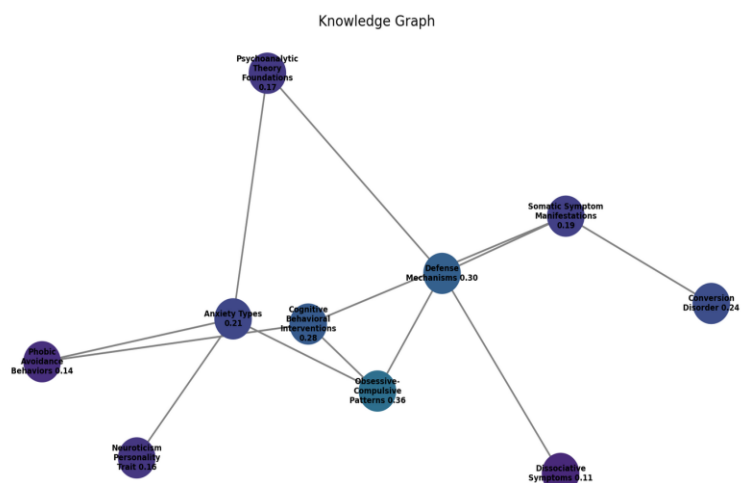


شکل ۲۳. نقشه حرارتی تسلط نهایی به تفکیک سطح دانشجو و مفهوم در سناریوی دانشجویان ضعیف

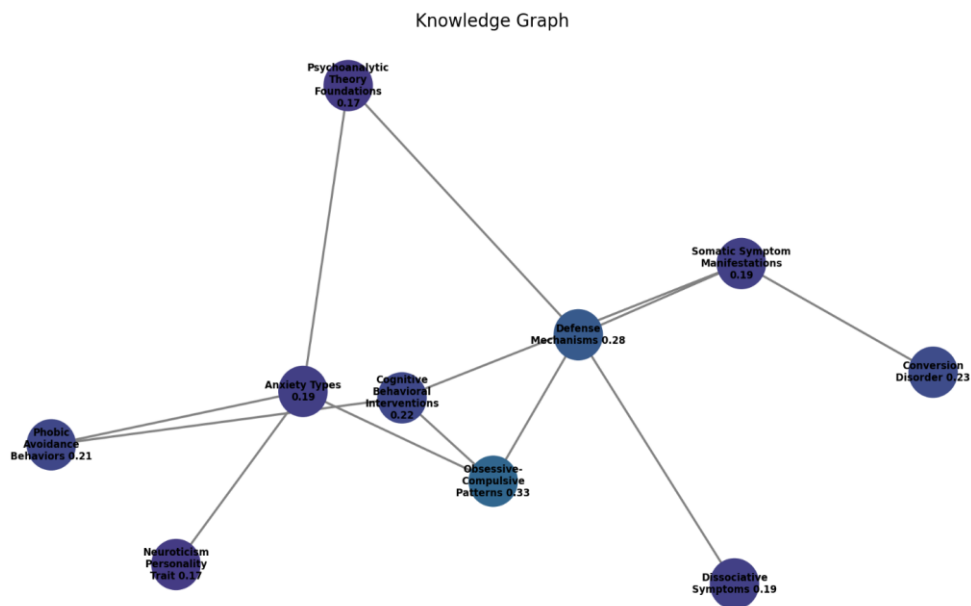
این نقشه حرارتی میزان Final Mastery را برای هر سطح دانشجو و هر مفهوم نشان می‌دهد. از آنجا که در این سناریو فقط دانشجویان ضعیف حضور دارند، مقدارهای مربوط به سطح متوسط و قوی همگی ۰٪ ثبت شده‌اند. در ردیف دانشجویان ضعیف، بیشترین تسلط مربوط به Dissociative Disorders با ۲۵٫۸٪ و Somatoform Disorders با ۲۴٫۱٪ است.

در مقابل، مفاهیمی مانند Cognitive Behavioral Mechanisms و Obsessive-Compulsive Disorder

پایین‌ترین تسلط را دارند و نشان می‌دهند این بخش‌ها نیازمند آموزش و تمرین بیشتری هستند.



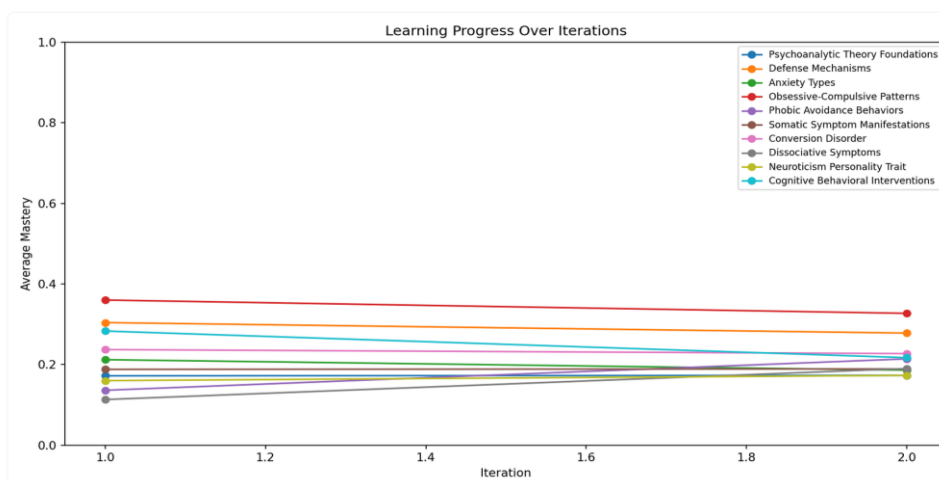
شکل ۲۴. گراف دانش در تکرار ۱



شکل ۲۵. گراف دانش در تکرار ۲

این دو شکل گراف دانش در شکل ۲۵ و ۲۶ به ترتیب گراف دانش را در تکرار اول و دوم نشان می‌دهند و تغییر مقدار تسلط مفاهیم را در طول یادگیری مقایسه می‌کنند. در تکرار اول، برخی گره‌ها مانند Obsessive-Compulsive Patterns و Defense Mechanisms مقدار بالاتری دارند، اما بسیاری از مفاهیم هنوز در سطح پایین قرار گرفته‌اند. در تکرار دوم، تغییرات کلی محدود است و بعضی مفاهیم کمی رشد یا افت دارند، که نشان می‌دهد یادگیری هنوز پایدار و کامل نشده است. به‌طور کلی، این دو گراف نشان می‌دهند که تنها دو تکرار برای افزایش چشمگیر mastery کافی نیست و برخی مفاهیم همچنان به تمرین و بازخورد بیشتری نیاز دارند.

Learning Progress



شکل ۲۶. روند پیشرفت یادگیری مفاهیم مختلف در دو مرحله شبیه‌سازی برای گروه دانشجویان ضعیف

این نمودار روند تغییر میانگین تسلط مفاهیم مختلف را در دو iteration نشان می‌دهد. به‌طور کلی، بیشتر مفاهیم در سطح پایینی از mastery قرار دارند و مقدار آن‌ها عمدتاً کمتر از ۰,۳۵ است. برخی مفاهیم مانند Obsessive-Compulsive Patterns و Defense Mechanisms از ابتدا سطح بالاتری دارند، اما رشد آن‌ها در مرحله دوم محدود است.

در مجموع، نمودار نشان می‌دهد که دو مرحله یادگیری برای ایجاد پیشرفت قوی کافی نبوده و بیشتر مفاهیم نیاز به تکرار، تمرین و بازخورد بیشتری دارند.

حال همه ی دانشجویان را قوی انتخاب می‌کنیم:

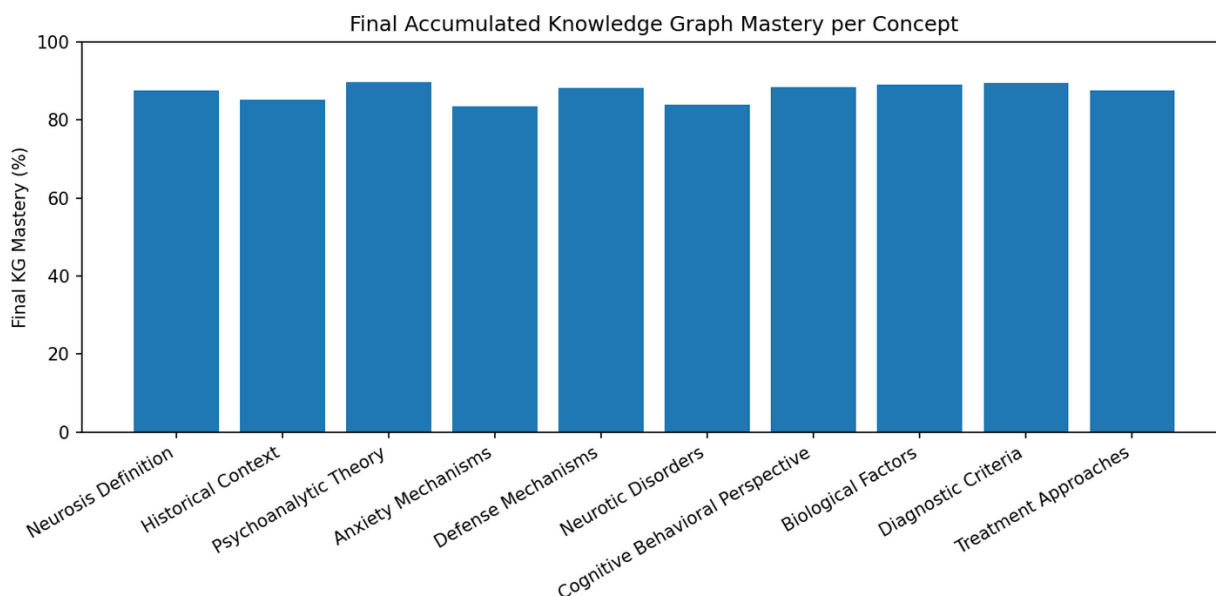
Curriculum Knowledge Graph Simulator
Generate concepts, simulate learning, show static knowledge graphs each iteration, then explore the interactive final graph.

Subject neurosis	Number of Concepts 10	Iterations 2
Weak Students 0	Medium Students 0	Strong Students 3

Run Simulation

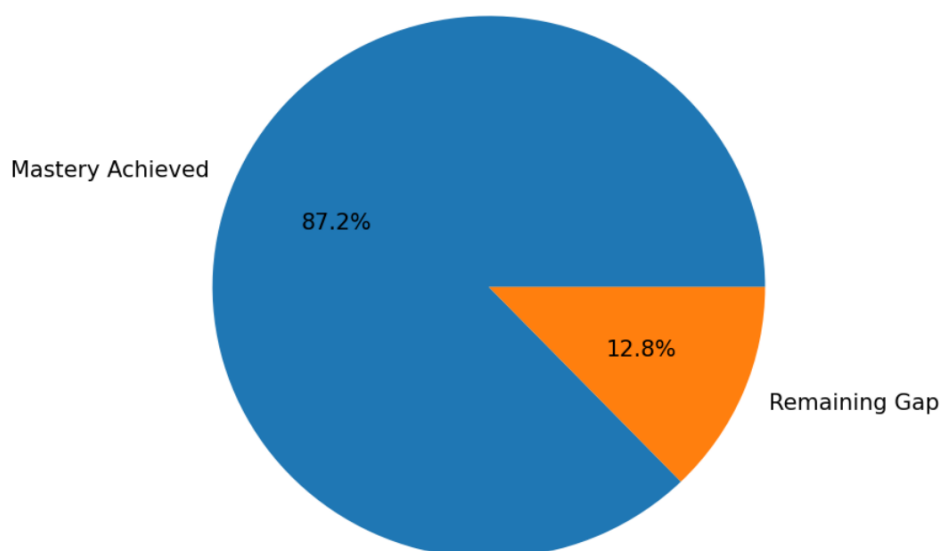
شکل ۲۷ تنظیمات شبیه‌سازی دانشجویان قوی

در این سناریو، موضوع **neurosis** با ۱۰ مفهوم و در ۲ iteration شبیه‌سازی شده است و گروه فقط شامل ۳ دانشجوی قوی است. نمودار تسلط نهایی نشان می‌دهد که همه مفاهیم در سطح بالایی، حدود ۸۳ تا ۹۰ درصد، یاد گرفته شده‌اند. این نتیجه نشان می‌دهد که دانشجویان قوی توانسته‌اند در مدت کوتاه، ارتباط میان مفاهیم را به‌خوبی درک و تثبیت کنند. با این حال، بعضی مفاهیم مانند **Treatment Approaches** هنوز نسبت به بقیه gap بیشتری دارند و نیازمند تمرین تکمیلی هستند.



شکل ۲۸ نمودار تسلط نهایی گراف دانش برای گروه دانشجویان قوی

Final KG Average Mastery

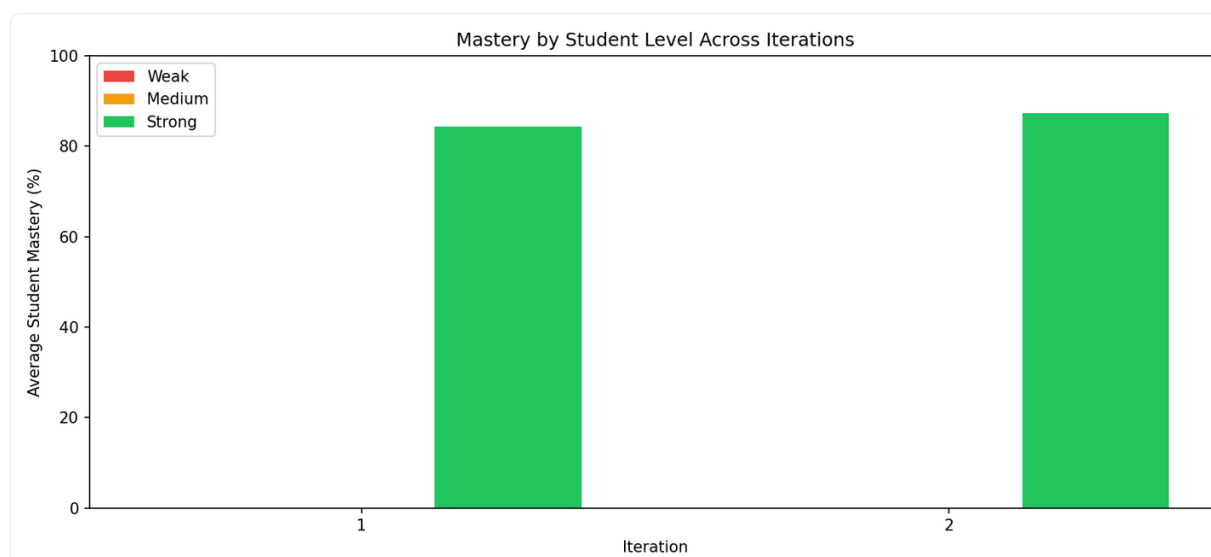


شکل ۲۹. نمودار دایره ای تسلط دانشجویان

در این نمودار شکل ۲۹، ۸۷٫۲٪ تسلط کسب شده و فقط ۱۲٫۸٪ فاصله باقی مانده است.

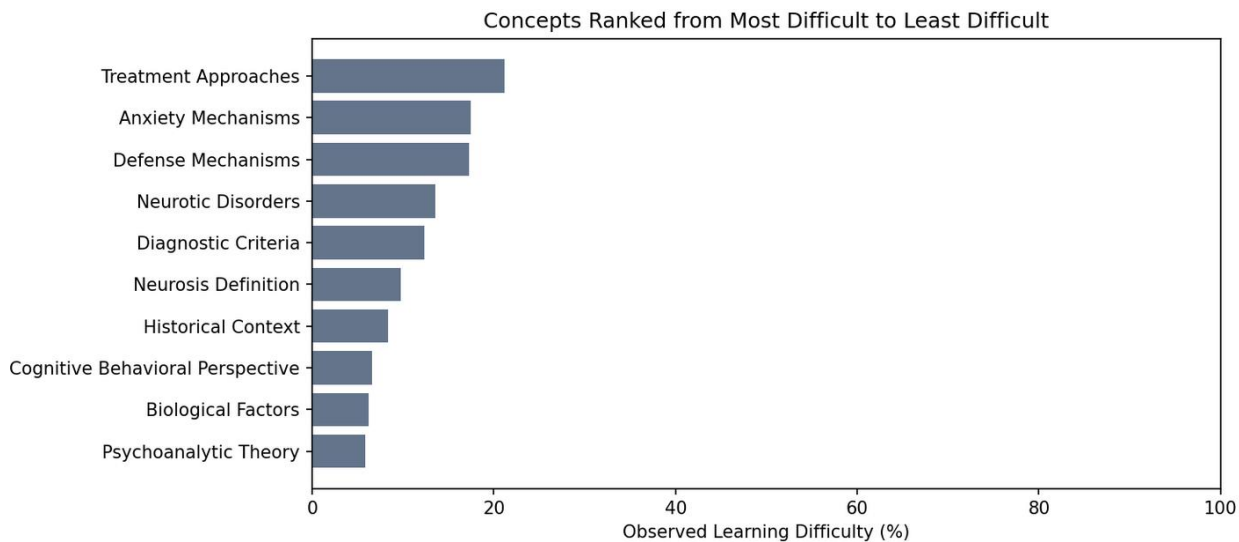
این دانشجویان قوی تقریباً به سطح mastery رسیده اند و فقط بخش کوچکی از یادگیری باقی مانده است. نسبت mastery به gap بسیار بالاست؛ یعنی عملکرد کلی خیلی خوب است.

Mastery by Student Level per Iteration



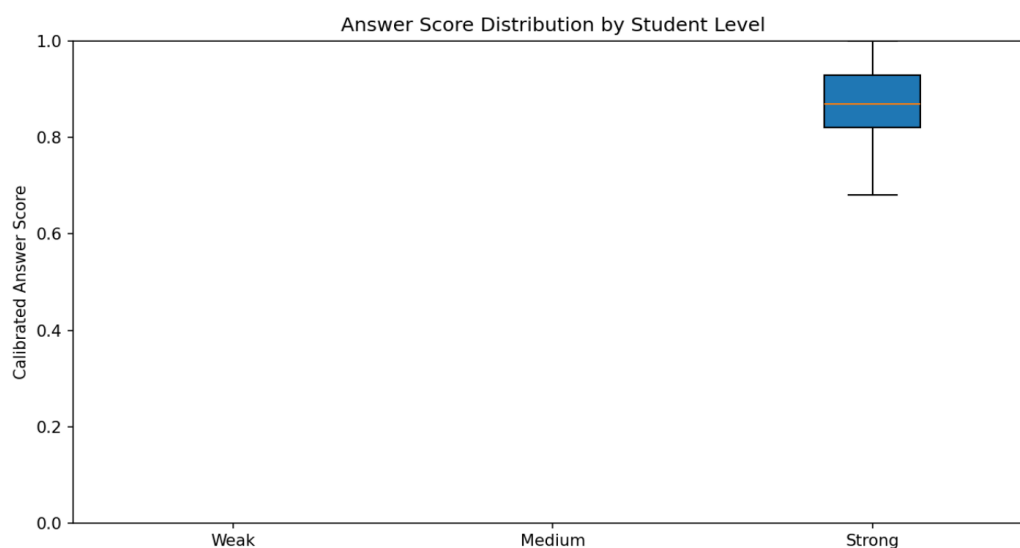
شکل ۳۰. تغییرات میانگین تسلط دانشجویان قوی در دو مرحله یادگیری

نمودار شکل ۳۰ نشان می‌دهد که چون فقط دانشجویان قوی در شبیه‌سازی حضور دارند، مقدار مربوط به گروه‌های ضعیف و متوسط صفر است. میانگین تسلط دانشجویان قوی از حدود ۸۴٪ در iteration اول به حدود ۸۷٪ در iteration دوم افزایش یافته است. این رشد هرچند زیاد نیست، اما نشان می‌دهد یادگیری در مسیر مثبت ادامه داشته است. کم بودن میزان افزایش نیز طبیعی است، چون دانشجویان قوی از همان ابتدا در سطح بالایی قرار داشته‌اند و به سقف mastery نزدیک بوده‌اند.



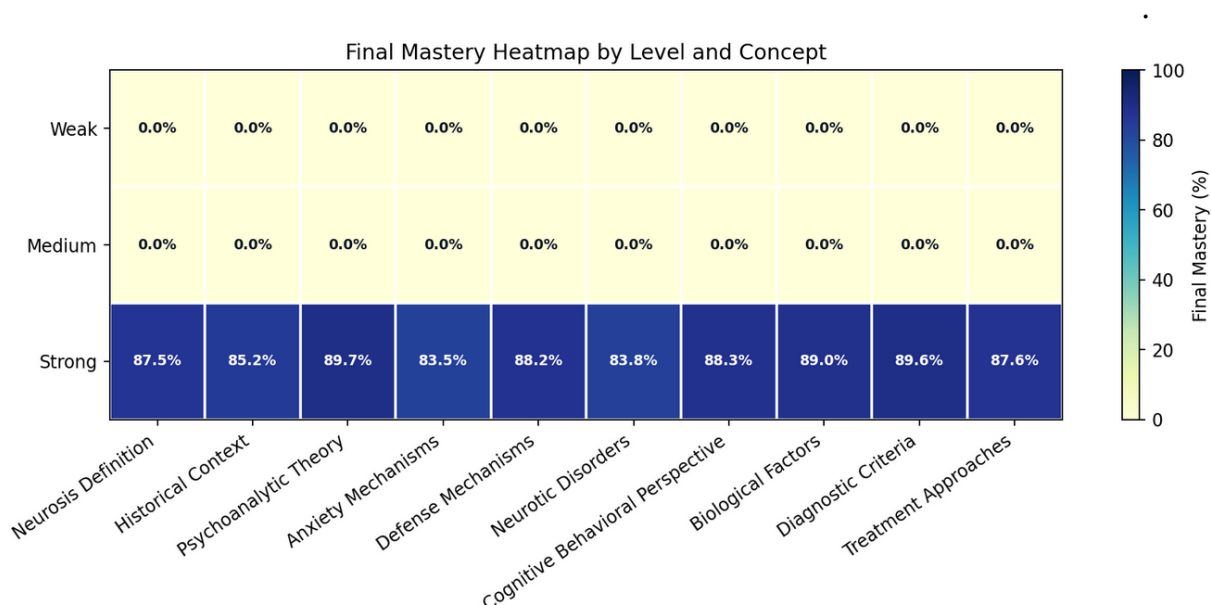
شکل ۳۱. سختی یادگیری مفاهیم در گروه دانشجویان قوی

جدول ۳۰ نشان می‌دهد که سختی یادگیری مفاهیم در گروه دانشجویان قوی بسیار پایین است و بیشتر مقادیر زیر ۲۰٪ قرار دارند. آسان‌ترین مفهوم Psychoanalytic Theory با حدود ۶٪ سختی است و سخت‌ترین مفهوم Treatment Approaches با حدود ۲۲٪ گزارش شده است. این اختلاف نشان می‌دهد که حتی در گروه قوی نیز مفاهیم کاربردی و درمانی کمی چالش‌برانگیزتر هستند. در مجموع، جدول تأیید می‌کند که گروه قوی به تسلط مناسبی رسیده و فقط به رفع gapهای محدود نیاز دارد.



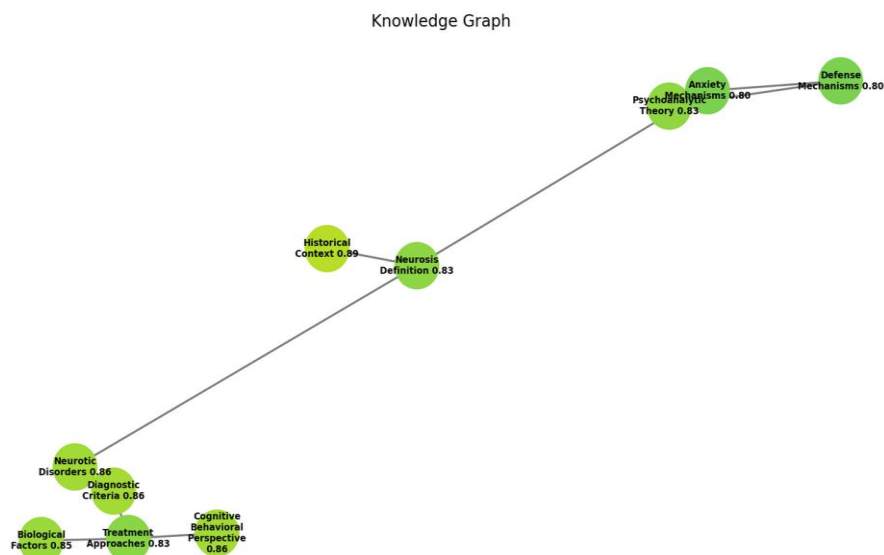
شکل ۳۲. توزیع نمرات پاسخ در گروه دانشجویان قوی

نمودار شکل ۳۲ نشان می‌دهد که در این سناریو فقط گروه Strong حضور دارد و برای گروه‌های Weak و Medium داده‌ای ثبت نشده است. نمرات پاسخ دانشجویان قوی عمدتاً در بازه حدود ۰,۸۲ تا ۰,۹۴ قرار دارند و میانه نمرات نزدیک به ۰,۸۷ است. این موضوع نشان می‌دهد پاسخ‌ها از کیفیت بالایی برخوردار بوده‌اند و پراکندگی نمرات نسبتاً محدود است. در نتیجه، عملکرد دانشجویان قوی هم از نظر میانگین و هم از نظر ثبات پاسخ‌دهی مطلوب ارزیابی می‌شود.

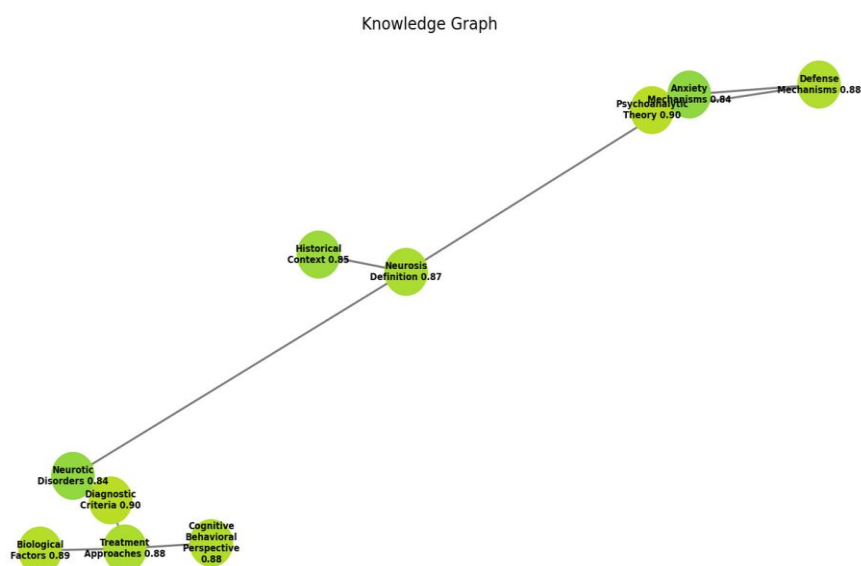


شکل ۳۳. نقشه حرارتی تسلط نهایی بر اساس سطح دانشجو و مفهوم

این نقشه حرارتی در شکل ۳۳ نشان می‌دهد که مقدارهای مربوط به Weak و Medium همگی ۰٪ هستند، چون در این سناریو فقط دانشجویان قوی شرکت داشته‌اند. در ردیف Strong، میزان تسلط برای همه مفاهیم بسیار بالا و در محدوده حدود ۸۳,۵٪ تا ۸۹,۷٪ قرار دارد. بیشترین تسلط مربوط به Psychoanalytic Theory و Diagnostic Criteria است، در حالی که Anxiety Mechanisms و Neurotic Disorders کمی پایین‌تر هستند. این نتیجه نشان می‌دهد دانشجویان قوی تقریباً در همه بخش‌های گراف دانش به سطح بالایی از mastery رسیده‌اند.



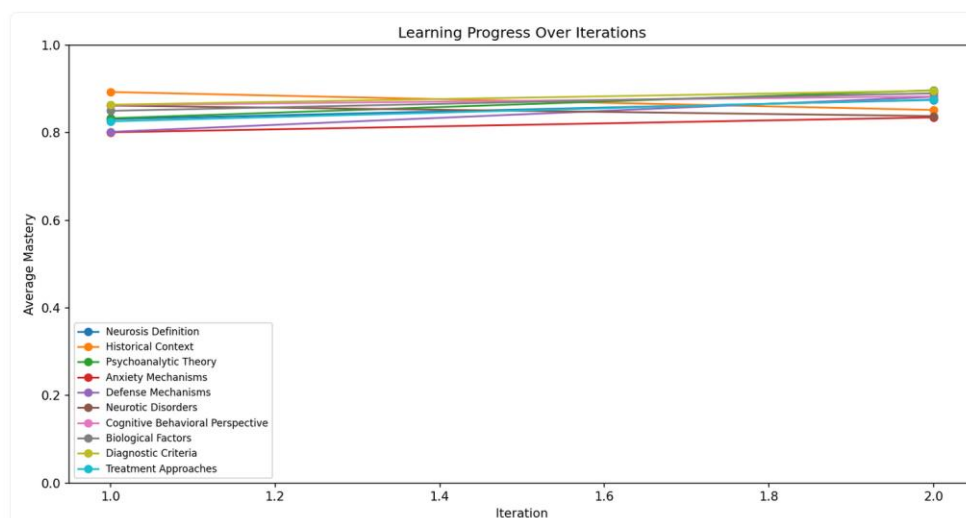
شکل ۳۴. گراف دانش دانشجویان در تکرار ۱



شکل ۳۵. گراف دانش دانشجویان در تکرار ۲

این دو شکل ۳۴ و ۳۵، گراف دانش دانشجویان قوی را در تکرار اول و دوم نشان می‌دهند. در تکرار اول، بیشتر مفاهیم از همان ابتدا در سطح بالای mastery قرار دارند و مقادیر آن‌ها عمدتاً بالاتر از ۰,۸۰ است. در تکرار دوم، تسلط در بسیاری از گره‌ها کمی افزایش یافته و گراف به وضعیت پایدارتر و کامل‌تری رسیده است. بنابراین، این دو شکل نشان می‌دهند که دانشجویان قوی حتی با تعداد تکرار کم نیز عملکرد بالایی دارند و تکرار دوم بیشتر نقش تکمیل و تقویت یادگیری را داشته است.

Learning Progress



شکل ۳۶. روند پیشرفت یادگیری مفاهیم در دو مرحله برای دانشجویان قوی

نمودار شکل ۳۶، روند mastery مفاهیم مختلف را از iteration اول تا دوم نشان می‌دهد. بیشتر مفاهیم از همان مرحله اول در سطح بالای ۰٫۸ قرار دارند و در مرحله دوم نیز مقدار آن‌ها اندکی افزایش یافته است. رشد کم اما مثبت نشان می‌دهد که دانشجویان قوی از ابتدا آمادگی بالایی داشته‌اند و در ادامه فقط gap های کوچک را کاهش داده‌اند. بنابراین، برای این گروه تمرکز آموزشی بهتر است بر فعالیت‌های تکمیلی، تحلیل عمیق‌تر و تمرین‌های چالشی باشد، نه آموزش پایه‌ای.

فصل ۶: جمع‌بندی، نتیجه‌گیری، پیشنهادها

در این پروژه، یک سامانه تحت وب برای شبیه‌سازی یادگیری مبتنی بر گراف دانش طراحی و پیاده‌سازی شد. هدف اصلی سامانه این بود که کاربر بتواند یک موضوع آموزشی را وارد کند و برنامه به‌صورت خودکار مفاهیم اصلی آن موضوع، روابط وابستگی میان مفاهیم، وضعیت یادگیری چند دانشجوی شبیه‌سازی‌شده و تحلیل‌های مربوط به یادگیری را تولید و نمایش دهد. پروژه از ترکیب مدل زبانی، گراف دانش، شبیه‌سازی دانشجو، ارزیابی پاسخ و تصویرسازی نتایج استفاده کرده است.

در مرحله نخست، برنامه موضوع واردشده توسط کاربر را دریافت می‌کند. علاوه بر موضوع، کاربر تعداد مفاهیم و تعداد تکرارهای شبیه‌سازی را نیز مشخص می‌کند. برای جلوگیری از ورودی‌های نامعتبر، تعداد مفاهیم بین ۲ تا ۱۵ و تعداد تکرارها بین ۱ تا ۱۰ محدود شده است. این کنترل ورودی باعث می‌شود اجرای برنامه از نظر محاسباتی قابل مدیریت باشد و خطاهای ناشی از ورودی‌های نامناسب کاهش یابد.

پس از دریافت ورودی، سامانه با استفاده از مدل زبانی، مفاهیم اصلی مرتبط با موضوع و روابط پیش‌نیازی میان آن‌ها را تولید می‌کند. خروجی مدل باید در قالب JSON باشد و شامل دو بخش `concepts` و `dependencies` است. برای افزایش پایداری برنامه، تابع `extract_json` طراحی شده است تا حتی اگر مدل خروجی را در قالب Markdown یا همراه با متن اضافی تولید کند، بتوان JSON اصلی را استخراج کرد. سپس مفاهیم و روابط وابستگی پاک‌سازی می‌شوند و فقط روابط معتبر در گراف دانش وارد می‌گردند.

در ادامه، سه دانشجوی شبیه‌سازی‌شده با سطح‌های متفاوت ساخته می‌شوند: ضعیف، متوسط و قوی. هر دانشجو دارای گراف دانش اختصاصی است. در این گراف، هر مفهوم یک گره است و روابط وابستگی به‌صورت یال‌های جهت‌دار نمایش داده می‌شوند. مقدار تسلط هر دانشجو بر هر مفهوم ابتدا صفر است و در طول شبیه‌سازی با توجه به پاسخ‌های او تغییر می‌کند. تفاوت میان دانشجویان با استفاده از پیام سیستمی مدل زبانی و مقدار دمای تولید پاسخ ایجاد شده است. دانشجوی ضعیف پاسخی کوتاه‌تر و ناقص‌تر تولید می‌کند، دانشجوی متوسط عملکردی بینابینی دارد و دانشجوی قوی پاسخی روشن‌تر و صحیح‌تر ارائه می‌دهد.

در هر تکرار آموزشی، دانشجویان توضیح مربوط به هر مفهوم را دریافت می‌کنند. سپس از هر دانشجو برای هر مفهوم یک سؤال پرسیده می‌شود. پاسخ دانشجو توسط مدل زبانی تولید می‌شود و سپس همان پاسخ با استفاده از تابع `evaluate` ارزیابی می‌گردد. خروجی ارزیابی یک عدد بین صفر تا یک است. این عدد نشان‌دهنده کیفیت پاسخ دانشجو است و برای به‌روزرسانی مقدار تسلط او روی مفهوم استفاده می‌شود. مقدار جدید تسلط از میانگین مقدار قبلی و نمره جدید به‌دست می‌آید.

برای نمایش وضعیت کلی یادگیری، یک گراف دانش عمومی نیز ایجاد شده است. این گراف میانگین میزان تسلط سه دانشجو را برای هر مفهوم نشان می‌دهد. در پایان هر تکرار، مقدار تسلط عمومی هر مفهوم ذخیره می‌شود و از آن برای رسم نمودار روند یادگیری استفاده می‌شود. بنابراین سامانه فقط وضعیت نهایی را نمایش نمی‌دهد، بلکه روند تغییرات یادگیری را نیز در طول تکرارها ثبت می‌کند.

در پایان شبیه‌سازی، سامانه چند خروجی تولید می‌کند. فهرست مفاهیم و جدول وابستگی‌ها ساختار آموزشی تولیدشده را نشان می‌دهند. گراف‌های هر تکرار تغییرات گراف دانش را در طول زمان نمایش می‌دهند. گراف نهایی وضعیت کلی یادگیری در پایان اجرا را نشان می‌دهد. گراف دانشجوی ضعیف به‌صورت جداگانه نمایش داده می‌شود تا وضعیت یادگیری دانشجویی که نیاز

بیشتری به بررسی دارد قابل مشاهده باشد. نمودار روند یادگیری تغییرات میزان تسلط مفاهیم را در طول تکرارها نشان می‌دهد. جدول پاسخ‌های دانشجویان نیز تمام پاسخ‌ها و نمره‌های ثبت‌شده را به‌صورت جزئی نمایش می‌دهد.

همچنین سامانه دارای بخش تحلیل معلم است. در این بخش، برای تشخیص یادگرفته‌شدن یا نشدن پاسخ‌ها، آستانه ۰.۶ در نظر گرفته شده است. پاسخ‌هایی که نمره آن‌ها برابر یا بیشتر از این مقدار باشد، در دسته یادگرفته‌شده قرار می‌گیرند. سپس درصد یادگیری هر مفهوم محاسبه شده و در قالب جدول و نمودار ستونی نمایش داده می‌شود. نمودار دایره‌ای نیز نسبت کلی پاسخ‌های یادگرفته‌شده و یادگرفته‌نشده را نشان می‌دهد.

از نظر پیاده‌سازی، برنامه با Flask ساخته شده است. سمت سرور در فایل app.py قرار دارد و شامل منطق اصلی برنامه است. سمت کاربر در فایل index.html قرار دارد و شامل فرم ورودی، نوار پیشرفت، نمایش خطاها و نمایش نتایج است. برای جلوگیری از متوقف شدن صفحه در زمان اجرای شبیه‌سازی، برنامه از نخ پس‌زمینه استفاده می‌کند. هر اجرای شبیه‌سازی یک شناسه یکتا دارد و وضعیت آن در دیکشنری jobs ذخیره می‌شود. صفحه وب با استفاده از JavaScript وضعیت اجرا را به‌صورت دوره‌ای بررسی می‌کند و نوار پیشرفت را به‌روزرسانی می‌نماید.

در مجموع، پروژه توانسته است یک چرخه کامل از تولید محتوای آموزشی، ساخت گراف دانش، شبیه‌سازی یادگیری، ارزیابی پاسخ‌ها و نمایش تحلیلی نتایج را پیاده‌سازی کند. این سامانه می‌تواند برای بررسی مفاهیم آموزشی، مشاهده روابط پیش‌نیازی، تحلیل یادگیری دانشجویان و نمایش روند پیشرفت مورد استفاده قرار گیرد.

۶-۱ نتیجه‌گیری

بر اساس پیاده‌سازی انجام‌شده، می‌توان نتیجه گرفت که استفاده از گراف دانش در کنار مدل زبانی می‌تواند روشی مناسب برای نمایش ساختار مفاهیم آموزشی و شبیه‌سازی فرآیند یادگیری باشد. در این پروژه، گراف دانش این امکان را فراهم کرده است که مفاهیم به‌صورت گره و روابط پیش‌نیازی به‌صورت یال‌های جهت‌دار نمایش داده شوند. این نمایش گرافی، درک ساختار درس و ارتباط میان مفاهیم را ساده‌تر می‌کند.

همچنین استفاده از مدل زبانی باعث شده است سامانه بتواند بر اساس یک موضوع ورودی، مفاهیم و روابط آموزشی را به‌صورت خودکار تولید کند. علاوه بر آن، مدل زبانی در نقش دانشجوی شبیه‌سازی‌شده و ارزیاب پاسخ نیز استفاده شده است. این موضوع نشان می‌دهد که مدل زبانی در این پروژه فقط برای تولید متن ساده به کار نرفته، بلکه در چند مرحله از چرخه آموزشی نقش داشته است: تولید برنامه مفهومی، تولید پاسخ دانشجو و ارزیابی پاسخ.

شبیه‌سازی سه سطح دانشجو نیز یکی از نتایج مهم این پروژه است. با تعریف دانشجوی ضعیف، متوسط و قوی، سامانه می‌تواند تفاوت سطح یادگیری و پاسخ‌دهی را نمایش دهد. این تفاوت‌ها در جدول پاسخ‌ها، نمره‌ها و گراف دانش قابل مشاهده هستند. به‌ویژه نمایش گراف دانشجوی ضعیف به‌صورت جداگانه نشان می‌دهد که پروژه به تحلیل نقاط ضعف نیز توجه کرده است و فقط میانگین کلی کلاس را نمایش نمی‌دهد.

یکی دیگر از نتایج مهم پروژه، امکان مشاهده روند یادگیری در طول تکرارها است. با ذخیره مقدار تسلط هر مفهوم در پایان هر تکرار، سامانه می‌تواند نمودار پیشرفت یادگیری را تولید کند. این نمودار نشان می‌دهد که میزان تسلط مفاهیم چگونه در طول زمان تغییر کرده است. بنابراین نتیجه پروژه فقط یک تصویر نهایی نیست، بلکه مسیر یادگیری نیز قابل بررسی است.

از نظر رابط کاربری، سامانه توانسته است یک فرآیند نسبتاً پیچیده را در قالب یک صفحه وب ساده و قابل استفاده ارائه دهد. کاربر فقط موضوع، تعداد مفاهیم و تعداد تکرارها را وارد می‌کند و سپس برنامه مراحل مختلف شبیه‌سازی را انجام می‌دهد. وجود نوار پیشرفت باعث می‌شود کاربر در زمان اجرای برنامه از وضعیت آن مطلع باشد. این موضوع برای برنامه‌هایی که با مدل زبانی و تولید نمودار سروکار دارند، اهمیت زیادی دارد.

با وجود این، باید توجه داشت که دقت نتایج پروژه به کیفیت خروجی مدل زبانی وابسته است. مفاهیم، وابستگی‌ها، پاسخ‌های دانشجویان و نمره‌های ارزیابی همگی توسط مدل زبانی تولید یا ارزیابی می‌شوند. بنابراین اگر مدل در تولید مفاهیم یا ارزیابی پاسخ‌ها خطا داشته باشد، نتایج سامانه نیز تحت تأثیر قرار می‌گیرند. با این حال، ساختار کد به‌گونه‌ای طراحی شده است که خروجی‌ها پاک‌سازی شوند و خطاهای قالب‌بندی تا حد امکان مدیریت گردند.

در نتیجه، می‌توان گفت پروژه به هدف اصلی خود رسیده است. سامانه توانسته است یک مدل آموزشی مبتنی بر گراف دانش ایجاد کند، چند دانشجوی مجازی را در سطوح مختلف شبیه‌سازی کند، پاسخ‌های آنان را ارزیابی نماید، میزان تسلط را به‌روزرسانی کند و خروجی‌های تحلیلی و تصویری تولید کند. این ساختار می‌تواند پایه‌ای برای توسعه سامانه‌های پیشرفته‌تر آموزشی و تحلیل یادگیری باشد.

۶-۱-۱ نوآوری‌ها

- یکی از مهم‌ترین دستاوردهای این پروژه، ترکیب مدل زبانی با گراف دانش آموزشی است. در بسیاری از برنامه‌های ساده آموزشی، مفاهیم به‌صورت فهرست یا متن نمایش داده می‌شوند، اما در این پروژه مفاهیم به‌صورت گراف جهت‌دار مدل‌سازی شده‌اند. این گراف نه‌تنها مفاهیم را نمایش می‌دهد، بلکه روابط وابستگی میان آن‌ها را نیز نشان می‌دهد. به همین دلیل، ساختار آموزشی موضوع برای کاربر قابل مشاهده‌تر و قابل تحلیل‌تر شده است.

دستآورد دوم پروژه، تولید خودکار مفاهیم و روابط وابستگی از روی موضوع وارد شده است. کاربر نیازی ندارد خودش مفاهیم را به‌صورت دستی وارد کند. سامانه با استفاده از مدل زبانی، مفاهیم اصلی را استخراج می‌کند و روابط پیش‌نیازی را پیشنهاد می‌دهد. سپس برنامه این خروجی را پاک‌سازی می‌کند و آن را به ساختار گراف تبدیل می‌نماید. این قابلیت باعث می‌شود سامانه برای موضوعات مختلف قابل استفاده باشد.

دستآورد سوم، شبیه‌سازی دانشجویان با سطح‌های مختلف است. در این پروژه، سه دانشجو با ویژگی‌های رفتاری متفاوت تعریف شده‌اند. دانشجوی ضعیف، متوسط و قوی هرکدام بر اساس پیام سیستمی و مقدار دمای متفاوت پاسخ تولید می‌کنند. این روش باعث می‌شود تفاوت سطح یادگیری در خروجی‌ها قابل مشاهده باشد. به‌جای آنکه همه دانشجویان رفتار یکسان داشته باشند، سامانه تلاش می‌کند تفاوت عملکرد را در شبیه‌سازی وارد کند.

دستآورد چهارم، طراحی مکانیزم ارزیابی پاسخ‌ها است. پاسخ هر دانشجو توسط مدل زبانی ارزیابی می‌شود و یک نمره بین صفر تا یک دریافت می‌کند. این نمره سپس برای به‌روزرسانی مقدار تسلط دانشجو روی مفهوم استفاده می‌شود. به این ترتیب، پاسخ‌های متنی به مقدار عددی قابل تحلیل تبدیل می‌شوند. این تبدیل برای رسم نمودارها و تحلیل یادگیری ضروری است.

دستآورد پنجم، ثبت و نمایش روند یادگیری است. در این پروژه، فقط مقدار نهایی تسلط ذخیره نمی‌شود، بلکه مقدار تسلط عمومی هر مفهوم در پایان هر تکرار در دیکشنری history ثبت می‌شود. سپس این داده‌ها به نمودار خطی تبدیل می‌شوند. این قابلیت به کاربر اجازه می‌دهد روند تغییر یادگیری را مشاهده کند و فقط به وضعیت نهایی اکتفا نکند.

دستآورد ششم، تولید تحلیل معلم است. سامانه با استفاده از آستانه ۰.۶، پاسخ‌ها را به دو دسته یادگرفته‌شده و یادگرفته‌نشده تقسیم می‌کند. سپس درصد یادگیری هر مفهوم و توزیع کلی یادگیری را محاسبه و به‌صورت جدول، نمودار ستونی و نمودار دایره‌ای نمایش می‌دهد. این بخش خروجی پروژه را برای استفاده آموزشی و تصمیم‌گیری معلم مناسب‌تر می‌کند.

دستآورد هفتم، نمایش جداگانه گراف دانشجوی ضعیف است. این بخش از پروژه نشان می‌دهد که تحلیل فقط بر اساس میانگین کلاس انجام نشده است. با نمایش گراف دانشجوی ضعیف، نقاط ضعف احتمالی یک یادگیرنده با عملکرد پایین‌تر قابل بررسی می‌شود. این موضوع می‌تواند برای طراحی آموزش جبرانی یا تمرین‌های تکمیلی مفید باشد.

دستآورد هشتم، پیاده‌سازی سامانه به‌صورت وب و تعاملی است. کاربر از طریق یک فرم ساده می‌تواند شبیه‌سازی را اجرا کند و خروجی‌ها را در همان صفحه مشاهده نماید. استفاده از نوار پیشرفت و اجرای پس‌زمینه نیز باعث شده است تجربه کاربری بهتری ایجاد شود. برنامه در زمان اجرای عملیات طولانی، وضعیت خود را به کاربر اطلاع می‌دهد و پس از پایان، صفحه نتیجه را نمایش می‌دهد.

دستآورد نهم، ذخیره خروجی‌ها به‌صورت تصویر است. گراف‌های دانش، نمودار روند یادگیری، نمودار ستونی و نمودار دایره‌ای همگی به‌صورت فایل تصویری در پوشه static/generated ذخیره می‌شوند. این موضوع باعث می‌شود نتایج قابل مشاهده، قابل استفاده در گزارش و قابل بررسی مجدد باشند.

در مجموع، نوآوری و دستآورد اصلی پروژه در ایجاد یک سامانه یکپارچه است که تولید مفاهیم، مدل‌سازی گرافی، شبیه‌سازی دانشجویان، ارزیابی پاسخ‌ها، تحلیل یادگیری و نمایش وب را در کنار یکدیگر قرار داده است.

۶-۱-۲ محدودیت‌ها

پروژه حاضر با وجود آنکه توانسته است یک نمونه اولیه کاربردی از سامانه شبیه‌سازی یادگیری مبتنی بر مدل زبانی، گراف دانش و ردیابی تسلط ارائه دهد، با محدودیت‌هایی همراه بوده است. این محدودیت‌ها هم از جنبه فنی و هم از جنبه اجرایی قابل بررسی هستند و توجه به آن‌ها برای توسعه نسخه‌های آینده سامانه ضروری است.

نخستین محدودیت، وابستگی سامانه به مدل زبانی بزرگ و سرویس خارجی API است. در این پروژه، استخراج مفاهیم، تشخیص روابط میان آن‌ها، تولید پاسخ‌های دانشجویان شبیه‌سازی‌شده و ارزیابی پاسخ‌ها همگی با کمک مدل زبانی انجام می‌شوند. بنابراین کیفیت خروجی نهایی تا حد زیادی به کیفیت پاسخ‌های مدل وابسته است. اگر مدل زبانی مفهومی نادرست، رابطه‌ای نامعتبر یا نمره‌ای نامتناسب تولید کند، نتایج شبیه‌سازی نیز تحت تأثیر قرار می‌گیرند. اگرچه در برنامه تلاش شده است خروجی JSON پاک‌سازی شود و روابط نامعتبر حذف گردند، اما صحت علمی مفاهیم و روابط به‌صورت مستقل و توسط یک منبع تخصصی بررسی نمی‌شود. همچنین به دلیل استفاده از API، عملکرد سامانه به اتصال پایدار اینترنت، در دسترس بودن سرویس مدل زبانی و محدودیت‌های احتمالی آن وابسته است.

محدودیت دوم، عدم اجرای پروژه در محیط آموزشی واقعی با عامل‌های یادگیرنده واقعی است. در این پژوهش، دانشجویان واقعی و کلاس آموزشی واقعی مورد استفاده قرار نگرفته‌اند و فرایند یادگیری به‌صورت شبیه‌سازی‌شده انجام شده است. دانشجویان ضعیف، متوسط و قوی از طریق تنظیمات مدل زبانی و دستورهای سیستمی تعریف شده‌اند، نه از طریق داده‌های واقعی یادگیرندگان. بنابراین نتایج به‌دست‌آمده بیشتر نشان‌دهنده توانایی سامانه در شبیه‌سازی و تحلیل اولیه هستند و نمی‌توان آن‌ها را به‌طور مستقیم معادل رفتار واقعی دانشجویان در یک کلاس آموزشی دانست. برای اعتبارسنجی دقیق‌تر، لازم است سامانه در آینده روی داده‌های واقعی و در یک محیط آموزشی عملی آزمایش شود.

محدودیت سوم، عدم پیاده‌سازی پروژه روی یک درس واقعی و کامل است. در نسخه فعلی، سامانه می‌تواند با دریافت یک موضوع آموزشی، مفاهیم مرتبط با آن را تولید و گراف دانش مربوط به آن را بسازد؛ اما این فرایند بر اساس یک برنامه درسی رسمی، سرفصل معتبر دانشگاهی یا محتوای آموزشی تأییدشده انجام نشده است. همچنین محتوای آموزشی تولیدشده در تابع `build_curriculum` ساده و عمومی است و برای همه مفاهیم ساختاری تقریباً مشابه دارد. این محتوا بیشتر برای اهداف شبیه‌سازی مناسب است و هنوز به سطح محتوای آموزشی عمیق، تخصصی و منطبق با استانداردهای آموزشی یک درس واقعی نرسیده است.

محدودیت چهارم، سادگی روش به‌روزرسانی میزان تسلط دانشجو است. در این پروژه، مقدار جدید `mastery` بر اساس ترکیبی ساده از مقدار قبلی تسلط و نمره جدید پاسخ محاسبه می‌شود. این روش قابل فهم و مناسب برای نمونه اولیه است، اما همه عوامل مؤثر در یادگیری را در نظر نمی‌گیرد. عواملی مانند سختی مفهوم، تعداد دفعات تمرین، فاصله زمانی بین پاسخ‌ها، احتمال فراموشی، تأثیر روابط پیش‌نیازی، و تفاوت وزن پاسخ‌های جدید و قدیم در این مدل به‌صورت دقیق لحاظ نشده‌اند. بنابراین مقدار `mastery` در این سامانه یک شاخص ساده برای نمایش روند یادگیری است و نباید به‌عنوان یک مدل کامل شناختی از یادگیری انسان در نظر گرفته شود.

محدودیت پنجم، وابستگی فرایند ارزیابی پاسخ‌ها به قضاوت مدل زبانی است. تابع `evaluate` از مدل زبانی می‌خواهد که پاسخ دانشجو را به یک نمره عددی بین صفر تا یک تبدیل کند. اگرچه این روش انعطاف‌پذیر است و امکان ارزیابی پاسخ‌های متنی را فراهم می‌کند، اما ممکن است در اجراهای مختلف نمره‌دهی کاملاً یکسان نباشد. علاوه بر این، معیارهای ارزیابی به‌صورت یک `rubric` آموزشی دقیق و چندسطحی تعریف نشده‌اند. در نتیجه، برای افزایش دقت و تکرارپذیری ارزیابی، لازم است در نسخه‌های بعدی معیارهای مشخص‌تری برای نمره‌دهی پاسخ‌ها طراحی شود.

محدودیت ششم، نبود پایگاه داده دائمی و سیستم مدیریت کاربران است. در نسخه فعلی، اطلاعات مربوط به شبیه‌سازی‌ها در حافظه برنامه و در قالب دیکشنری `obs` ذخیره می‌شود. این داده‌ها با توقف سرور از بین می‌روند و فقط برخی خروجی‌ها مانند تصاویر در پوشه `static/generated` باقی می‌مانند. همچنین سامانه فاقد احراز هویت، حساب کاربری، ذخیره‌سازی نتایج هر کاربر و مدیریت دسترسی است. بنابراین این نسخه بیشتر برای اجرای محلی و آزمایشی مناسب است و برای استفاده در محیط آموزشی واقعی باید قابلیت‌هایی مانند پایگاه داده، ثبت سوابق یادگیری، مدیریت کاربران و حفظ حریم خصوصی اضافه شود.

محدودیت هفتم، محدود بودن رابط کاربری و امکانات تحلیلی تعاملی است. در نسخه فعلی، نتایج بیشتر به‌صورت تصویر، جدول و نمودارهای ثابت نمایش داده می‌شوند. کاربر نمی‌تواند به‌صورت تعاملی روی گره‌های گراف کلیک کند، جزئیات هر مفهوم را مشاهده کند، مفاهیم خاص را فیلتر کند یا روند یادگیری را به‌صورت پویا بررسی نماید. بنابراین، اگرچه خروجی‌ها برای تحلیل اولیه قابل استفاده هستند، اما برای تحلیل‌های آموزشی عمیق‌تر بهتر است در نسخه‌های آینده از گراف‌ها و داشبوردهای تعاملی استفاده شود.

در مجموع، محدودیت‌های اصلی این پروژه شامل وابستگی به مدل زبانی و اینترنت، نبود داده‌های واقعی از یادگیرندگان، ساده بودن مدل `mastery`، نبود پایگاه داده دائمی، و محدود بودن امکانات تعاملی سامانه است. با این حال، این محدودیت‌ها با ماهیت نمونه اولیه پروژه سازگار هستند و می‌توانند مسیر توسعه آینده سامانه را مشخص کنند. در نسخه‌های بعدی، استفاده از داده‌های واقعی، طراحی `rubric` دقیق برای ارزیابی، توسعه مدل پیشرفته‌تر ردیابی دانش، افزودن پایگاه داده و ایجاد رابط کاربری تعاملی می‌تواند دقت، اعتبار و کاربردپذیری سامانه را افزایش دهد.

برای توسعه آینده این پروژه، چند پیشنهاد اصلی قابل ارائه است. نخست، می‌توان مدل ردیابی دانش را از حالت ساده فعلی خارج کرد و از مدل‌های پیشرفته‌تر مانند مدل‌های احتمالاتی ردیابی دانش یا روش‌های مبتنی بر یادگیری ماشین استفاده نمود. در نسخه فعلی، مقدار تسلط دانشجو با روشی ساده و بر اساس میانگین مقدار قبلی و نمره جدید به‌روزرسانی می‌شود. در آینده می‌توان عواملی مانند سختی مفهوم، تعداد دفعات تمرین، اثر فراموشی، فاصله زمانی پاسخ‌ها و روابط پیش‌نیازی میان مفاهیم را نیز در محاسبه mastery وارد کرد. این کار باعث می‌شود مدل یادگیری به رفتار واقعی دانشجویان نزدیک‌تر شود.

پیشنهاد دوم، استفاده از روش‌های گراف‌ی پیشرفته‌تر، به‌ویژه Graph Neural Networks است. در نسخه فعلی، گراف دانش بیشتر برای نمایش مفاهیم و روابط پیش‌نیازی استفاده می‌شود، اما در نسخه‌های بعدی می‌توان از ساختار گراف برای پیش‌بینی دقیق‌تر یادگیری، تشخیص مفاهیم دشوار و شناسایی گلوگاه‌های آموزشی استفاده کرد. شبکه‌های عصبی گراف‌ی می‌توانند اثر یک مفهوم بر مفاهیم وابسته را بهتر مدل کنند و نشان دهند ضعف در یک گره چگونه بر بخش‌های دیگر گراف اثر می‌گذارد.

پیشنهاد سوم، بهبود کیفیت محتوای آموزشی و ارزیابی پاسخ‌ها است. در نسخه فعلی، برای هر مفهوم توضیحی عمومی و یک سؤال ساده تولید می‌شود و ارزیابی پاسخ نیز به یک نمره بین صفر تا یک محدود است. در آینده می‌توان برای هر مفهوم توضیح اختصاصی‌تر، مثال‌های واقعی، تمرین‌های چندسطحی، منابع کمکی و سؤال‌های متنوع تولید کرد. همچنین بهتر است ارزیابی پاسخ‌ها بر اساس rubric دقیق‌تری انجام شود؛ برای مثال معیارهایی مانند صحت علمی، کامل بودن پاسخ، وضوح بیان، استفاده از مثال و ارتباط پاسخ با مفهوم اصلی به‌صورت جداگانه بررسی شوند.

پیشنهاد چهارم، توسعه سامانه به سمت یک real-time adaptive tutoring system است. در چنین نسخه‌ای، سامانه می‌تواند در زمان واقعی وضعیت یادگیری دانشجو را تحلیل کند و بر اساس سطح تسلط او، سؤال بعدی، توضیح تکمیلی یا تمرین مناسب را پیشنهاد دهد. این قابلیت باعث می‌شود سیستم از یک ابزار شبیه‌سازی به یک سامانه آموزشی تطبیقی تبدیل شود که مسیر یادگیری هر دانشجو را به‌صورت شخصی‌سازی‌شده تنظیم می‌کند.

پیشنهاد پنجم، بهبود امکانات فنی و کاربری سامانه است. افزودن پایگاه داده برای ذخیره نتایج، مدیریت بهتر خطاها، افزایش امنیت پروژه از جمله اقداماتی هستند که می‌توانند سامانه را برای استفاده جدی‌تر آماده کنند. همچنین استفاده از گراف‌ها و نمودارهای تعاملی با ابزارهایی مانند Plotly یا Cytoscape.js می‌تواند تحلیل نتایج را برای کاربر ساده‌تر و قابل فهم‌تر کند.

در نهایت، پیشنهاد می‌شود در نسخه‌های آینده امکان بازبینی مفاهیم و روابط توسط معلم یا متخصص آموزشی نیز اضافه شود. در نسخه فعلی، مفاهیم و وابستگی‌ها توسط مدل زبانی تولید می‌شوند، اما اگر کاربر بتواند پیش از شروع شبیه‌سازی آن‌ها را ویرایش یا تأیید کند، دقت علمی گراف دانش افزایش می‌یابد. به طور کلی، این پروژه می‌تواند در آینده از یک نمونه اولیه به سامانه‌ای کامل‌تر برای تحلیل یادگیری، شناسایی گلوگاه‌های آموزشی و پشتیبانی از آموزش تطبیقی تبدیل شود.

- [1] Piech C, Spencer J, Huang J, Ganguli S, Sahami M, Guibas L, et al. Deep Knowledge Tracing. arXiv preprint arXiv:1506.05908. 2015.
- [2] Ghosh A, Heffernan N, Lan AS. Context-Aware Attentive Knowledge Tracing. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2020.
- [3] Nakagawa H, Iwasawa Y, Matsuo Y. Graph-based Knowledge Tracing: Modeling Student Proficiency Using Graph Neural Network. 2019.
- [4] Yang Y, Shen J, Qu Y, Liu Y, Wang K, Zhu Y, et al. GIKT: A Graph-based Interaction Model for Knowledge Tracing. arXiv preprint arXiv:2009.05991. 2020.
- [5] Qu K, Li KC, Wong BTM, Wu MMF, Liu M. A Survey of Knowledge Graph Approaches and Applications in Education. Electronics. 2024;13(13):2537.
- [6] Li Z, Yazdanpanah V, Wang J, Gu W, Shi L, Cristea AI, et al. TutorLLM: Customizing Learning Recommendations with Knowledge Tracing and Retrieval-Augmented Generation. arXiv preprint arXiv:2502.15709. 2025.
- [7] Bian H. LLM-Empowered Knowledge Graph Construction: A Survey. arXiv preprint arXiv:2510.20345. 2025.
- [8] Li R, Dai Q, Zhang Z, Chen X, Dong Z, Wen JR. KnowTrace: Bootstrapping Iterative Retrieval-Augmented Generation with Structured Knowledge Tracing. arXiv preprint arXiv:2505.20245. 2025



University of Tehran



College of Engineering

School of Electrical and Computer Engineering

Thesis Title

Detecting Learning Bottlenecks in a Large Language Model-Based
Multi-Agent System Using Knowledge Tracing and Knowledge
Graphs

By:

Shahzad Momayez

Supervisor:

Dr. Fattane Taghiyare

Spring 2026